

ÓBUDAI EGYETEM

[Kar neve, pl. Neumann János Informatikai Kar]

[Intézet neve]

TUDOMÁNYOS DIÁKKÖRI DOLGOZAT

Ellenőrizhető, adatvédelmet biztosító vizsgakérdés-generálás fogyasztói GPU-n futó kis nyelvi modellekkel

*A MIMIR tervezésvezérelt, forráshivatkozással RAG-módszere és többkarú
kísérleti kiértékelése*

Szerzők:

[Szerző 1 neve]

[szak, évfolyam]

[Szerző 2 neve]

[szak, évfolyam]

Konzulens:

[Konzulens neve]

[beosztás, intézet]

Budapest, 2026

64. Tudományos Diákköri Konferencia – 2026. őszi félév

Tartalmi összefoglaló

A feleletválasztós vizsgakérdések írása lassú és hibalehetőségekkel teli szakmai munka: a kérdésnek egyértelműnek kell lennie, pontosan egy válaszlehetőségnek kell a tananyag szerint helyesnek lennie, a disztraktoroknak pedig hihetőnek, de igazolhatóan hibásnak. A nagy nyelvi modellek (LLM) megjelenésével a legelterjedtebb gyakorlat az lett, hogy az oktató feltölti a jegyzetét egy felhőben futó csevegő asszisztensbe, és „tíz feleletválasztós kérdést” kér. Ez kényelmes, de négy lényeges hiányossága van: (P1) semmi sem ellenőrzi, hogy a megjelölt helyes válasz valóban következik-e a dokumentumból; (P2) a kérdések száma, típusa és nehézsége csak kérdés, a hosszú bemenetet a modellek egyenetlenül használják, így a kérdések néhány szakaszra tömörülnek; (P3) a tananyag egy harmadik félhez kerül; (P4) a módszer nagy, távoli modellekre és hosszú kontextusra támaszkodik.

Dolgozatunkban bemutatjuk a MIMIR rendszert, amely a fenti egyetlen promptot explicit, ellenőrizhető *módszerrel* váltja fel, és egyetlen fogyasztói GPU-n futó, 7 milliárd paraméteres, 4 bitre kvantált nyelvi modellel működik. A módszer lépései: szemantikus darabolás ablakozott mondatbeágyazással; lefedettségvezérelt *vizsgaterv* (blueprint), amely a fogalmakat, a kérdéstípusokat és a Bloom-szinteket a legnagyobb maradék elvén osztja ki a slotokra; slotonkénti visszakeresés (sűrű vagy BM25 + sűrű, reciprok rangfúzióval); egyenként generált, forrásrészletekre hivatkozó kérdések; valamint a legolcsóbbtól a legdrágábbig rendezett ellenőrző lánc (forma, meta-hivatkozás, hivatkozás, alátámasztottság és disztraktorok, vak megoldhatóság, válaszszivárgás, duplikátum), amely hiba esetén visszacsatolással újrapróbálkozik. Egy fogalomgráf-alapú változat (GraphRAG-lite) a tervezőt, a disztraktorjavaslatokat és a kontextust gazdagítja. A rendszer mikroszolgáltatás-architektúrája zéró adatmegőrzést valósít meg: a feltöltött tartalom csak a feladat idejére létezik, a naplóban csak kriptográfiai ujjlenyomatok maradnak.

Második hozzájárulásunk egy reprodukálható, nyílt kiértékelési *módszertan*: tizenöt, egyszerre egy tényezőt változtató kísérleti kar; 39 magyar és angol dokumentumból, 326 referencia-kérdésből álló adatkészlet; a generátoroktól eltérő modelleszaládba tartozó LLM-bíró; formálisan definiált mutatók (formai megfelelés, alátámasztottság, vak megoldhatóság, disztraktor-validitás, lefedettség, szivárgás); valamint dokumentumszintű statisztika (bootstrap konfidenciaintervallum, páros Wilcoxon-próba Holm-korrekcióval, rangbiszeriális hatásméret, Krippendorff- α).

Egy magyar nyelvű dokumentumon, tíz karral és három ismétléssel végzett pilot vizsgálat (300 kérdés) szerint a tervezés a helyi 7B modell alátámasztottságát 67-ről 73 %-ra, vak megoldhatóságát 85-ről 93 %-ra, disztraktor-validitását 80-ról 87 %-ra, lefedettségét 50-ről 58 %-ra javította; egy nagy szervermodell ugyan a legmagasabb minőségi indexet (0,99) érte el, de kérdéseit a dokumentum darabjainak mindössze negyedéből merítette; a helyi modell a GPU-memóriából 5,4–5,9 GB-ot foglalt, tehát 8 GB-os kártyán is fut. A pilot két, korábbi méréseket érvénytelenítő mérési hibát és egy új hibamódot is feltárt: az azonos 7B modellel végzett önellenőrzés a bíró ítéletével csak véletlenszerű egyezést mutat (Cohen- $\kappa \in [-0,14; 0,07]$), a

visszacsatolási hurok pedig a modellt a forrásmondat szó szerinti átvételére, azaz a helyes válasz kérdésszövegbe szivároztatására ösztönzi, amit a bíró sem büntet. Az egyetlen dokumentumon mért futásról futásra jelentkező zaj ($\approx 0,07$) indokolja a 39 dokumentumos, dokumentumszintű statisztikára épülő fő vizsgálatot, amelynek protokollját részletesen rögzítjük.

Kulcsszavak: automatikus kérdésgenerálás, visszakereséssel kiegészített generálás (RAG), kis nyelvi modellek, verifikáció, LLM-bíró, adatvédelem, oktatástechnológia, magyar nyelv

Abstract

The most common way to obtain an exam from teaching material today is to upload the document to a hosted chat-based large language model (LLM) and ask for a test. This offers no evidence that the keyed answers are supported by the document, no control over which parts of the material are covered, and it discloses the material to a third party. We present MIMIR, an open system that generates multiple-choice exams from a teacher’s own documents with a 4-bit, 7-billion-parameter model running on a single consumer GPU. MIMIR replaces the single prompt by an explicit method: semantic chunking with windowed sentence encoding, a coverage-driven exam *blueprint* that allocates concepts, question types and Bloom levels to slots by the largest-remainder rule, per-slot dense or hybrid (BM25 + dense, reciprocal rank fusion) retrieval, per-question generation with chunk citations, and a cheapest-first verifier chain with feedback-driven retries, optionally enriched by a session-scoped concept graph. The microservice architecture retains no uploaded content beyond the lifetime of a job.

We also contribute an evaluation methodology and open harness: fifteen single-factor experiment arms, a 39-document Hungarian–English dataset with 326 reference questions, a cross-family LLM judge, formally defined measures and document-level statistics. A pilot on one Hungarian document (ten arms, three seeds, 300 questions) shows that planning raised the local model’s grounding from 67 % to 73 %, blind answerability from 85 % to 93 %, distractor validity from 80 % to 87 % and section coverage from 50 % to 58 %; that a large server model scored highest (0.99) but drew its questions from a quarter of the document; and that the local model occupies 5.4–5.9 GB of GPU memory. The pilot also exposed two measurement defects, a same-model verifier at chance-level agreement with the judge (Cohen’s κ from -0.14 to 0.07) and an answer-leakage failure mode that inflates both automatic and LLM-judge scores. Run-to-run noise of about 0.07 on a single document motivates the 39-document main study, whose protocol we specify in full.

Keywords: automatic question generation, retrieval-augmented generation, small language models, verification, LLM-as-a-judge, privacy, educational technology, Hungarian

Tartalomjegyzék

Tartalmi összefoglaló	ii
Abstract	iv
Tartalomjegyzék	v
Rövidítések és jelölések	viii
1. Bevezetés	1
1.1. Motiváció	1
1.2. Problémafelvetés	1
1.3. Kutatási kérdések és hipotézisek	2
1.4. Hozzájárulások	3
1.5. A dolgozat felépítése	3
2. Szakirodalmi áttekintés	4
2.1. Automatikus kérdésgenerálás	4
2.2. Disztraktorgenerálás és tesztelméleti követelmények	4
2.3. A kognitív szint: Bloom taxonómiája	5
2.4. Visszakereséssel kiegészített generálás	5
2.5. Hosszú kontextus: az egylépéses alternatíva korlátai	6
2.6. Hallucináció, ellenőrzés és önjavítás	6
2.7. Kis nyelvi modellek és kvantálás	7
2.8. Nyelvi modell mint bíró	7
2.9. Magyar nyelvi erőforrások és szabályozási környezet	8
2.10. Összegzés és a kutatási rés	8
3. A feladat formális modellje	9
3.1. Dokumentum, mondatok és darabok	9
3.2. Vizsgaspecifikáció és kérdés	9
3.3. Forrásalapú minőségi predikátumok	10
3.4. Vizsgaszintű mutatók	10
3.5. A tervezési probléma	11
3.6. A vizsgaterv szerkezeti tulajdonságai	12
3.7. A modellhívások számának korlátai	13
4. Rendszerarchitektúra és helyi üzemeltetés	14
4.1. Tervezési célok	14
4.2. Mikroszolgáltatás-architektúra	14
4.2.1. Feladatkezelés és párhuzamosság	14
4.3. Adatvédelem: zéró megőrzés	16
4.4. Erőforrás-modell: a GPU-memória költségvetése	17

5. A MIMIR generálási módszere	19
5.1. Áttekintés: naiv RAG és tervezett csővezeték	19
5.2. Szövegkinyerés	20
5.3. Szemantikus darabolás	20
5.3.1. Mondatbeágyazás ablakozással	20
5.3.2. A vágási szabály	22
5.3.3. A csonkolt kódoló hibájának elemzése	22
5.4. Indexelés és sűrű visszakeresés	22
5.5. Az alapvonalak	23
5.6. A vizsgaterv	23
5.6.1. Áttekintés és fogalomtervezés	23
5.6.2. Bloom-szintek kiosztása	23
5.6.3. Mohó, lefedettségvezérelt slotkiosztás	24
5.7. Slotonkénti visszakeresés	24
5.7.1. Hibrid visszakeresés: BM25 és reciprok rangfúzió	25
5.8. Forráshoz kötött kérdésgenerálás	25
5.9. Az ellenőrző lánc	25
5.9.1. Ellenőrzések a költség szerinti sorrendben	25
5.9.2. Slotkitöltés visszacsatolással és tartalékokkal	26
5.10. Munkamenet-hatókörű fogalomgráf	26
5.11. Összeállítás és emberi felügyelet	28
6. Kiértékelési módszertan	29
6.1. A kiértékelő keretrendszer	29
6.2. Adatkészlet	30
6.3. Kísérleti karok	31
6.4. Mutatók	31
6.4.1. Automatikus mutatók	32
6.4.2. Tartalmi mutatók	32
6.4.3. Költségmutatók	33
6.5. A bíró	33
6.6. Oktatói értékelés	34
6.7. Statisztikai elemzés	34
6.7.1. Az elemzés egysége	34
6.7.2. Konfidenciaintervallum	34
6.7.3. Páros összehasonlítás	34
6.7.4. Többszörös összehasonlítás	35
6.7.5. Egyezési mutatók	35
6.7.6. Az ellenőrző mint osztályozó	35
6.8. Mérési validitás: a korábbi mérések hibái	36
6.9. Hardver	36
7. Eredmények	37
7.1. A pilot beállításai	37
7.2. Összesített eredmények	37
7.3. A tervezés hatása (RQ1, RQ2)	38
7.4. Az ellenőrzés hatása	39
7.4.1. A formai hiba és a tartalékkérdés	39

7.4.2. Az ellenőrző mint osztályozó	40
7.5. Válaszszivárgás: egy új hibamód	41
7.6. Szervermodell és helyi modell (RQ3)	42
7.7. Költség	42
7.8. Darabolási stratégiák	43
7.9. Futásról futásra jelentkező zaj	44
7.10. Példák	44
7.11. A hipotézisek előzetes értékelése	44
7.12. A fő vizsgálat eredményei	45
8. Diskusszió	46
8.1. Miért módszer, és nem prompt?	46
8.2. A mérés mint kutatási eredmény	46
8.3. Az érvényességet veszélyeztető tényezők	47
8.4. Etikai és jogi megfontolások	47
9. Összefoglalás és további munka	49
9.1. Összefoglalás	49
9.2. További munka	50
Irodalomjegyzék	51
Ábrajegyzék	56
Táblázatjegyzék	57
A Promptok	59
A.1. A naiv alapvonal (E0) és a teljes dokumentumos alapvonal promptja	59
A.2. A tervező promptja	59
A.3. A generátor promptja	60
A.4. Az ellenőrző promptjai	60
A.5. A fogalomgráf promptja (E3)	61
B A bíró és az oktatók rubrikája	62
C Konfigurációk és reprodukálás	63
C.1. Példakonfigurációk	63
C.2. A reprodukálás lépései	63
C.3. A fő vizsgálat futtatási terve	64
C.4. A közölt számok forrásai	64

Rövidítések és jelölések

Rövidítések

AQG	<i>Automatic Question Generation</i> , automatikus kérdésgenerálás
BM25	<i>Best Matching 25</i> , valószínűségi relevanciakeretre épülő lexikális rangsoroló függvény
CI	<i>Confidence Interval</i> , konfidenciaintervallum
E5	<i>EmbEddings from bidirectional Encoder representations</i> , kontrasztív szövegbeágyazó modellcsalád
GGUF	a <code>llama.cpp</code> kvantált modellformátuma
GQA	<i>Grouped-Query Attention</i> , csoportosított lekérdezésű figyelem
GPU	<i>Graphics Processing Unit</i> , grafikus feldolgozóegység
GDPR	az EU 2016/679 általános adatvédelmi rendelete
KV	<i>key-value</i> (kulcs-érték) gyorsítótár a transzformer dekódolásakor
LLM	<i>Large Language Model</i> , nagy nyelvi modell
MCQ	<i>Multiple-Choice Question</i> , feleletválasztós kérdés
MRR	<i>Mean Reciprocal Rank</i> , átlagos reciprok rang
Q4_K_M	4 bites, csoportos K-kvantálás közepes változata (<code>llama.cpp</code>)
RAG	<i>Retrieval-Augmented Generation</i> , visszakereséssel kiegészített generálás
RQ	<i>Research Question</i> , kutatási kérdés
RRF	<i>Reciprocal Rank Fusion</i> , reciprok rangfúzió
SLM	<i>Small Language Model</i> , kis nyelvi modell (itt: ≤ 8 milliárd paraméter)
VRAM	a GPU dedikált memóriája

Fontosabb jelölések

$\mathcal{D}$	a forrásdokumentum (karaktersorozat)
$S = (s_1, \dots, s_m)$	a dokumentum mondatai
$\mathcal{C} = (c_1, \dots, c_K)$	a dokumentum darabjai (<i>chunk</i>), $K = \mathcal{C} $
$\phi(\cdot) \in \mathbb{R}^d$	egységnyi normájú szövegbeágyazás ($d = 768$, multilingual-E5-base)
δ_i	a szomszédos mondatok koszinusz-távolsága, $\delta_i = 1 - \langle \phi(s_i), \phi(s_{i+1}) \rangle$
τ	a darabolási alapküszöb (percentilis vagy $\mu + k\sigma$)
$\sigma = (n, T, \lambda, \ell)$	vizsgaspecifikáció: kérdésszám, típusok, nehézség, nyelv
$q = (t, O, o^*, R)$	kérdés: törzs, válaszlehetőségek, kulcs, hivatkozott darabok
$\mathcal{X} = (q_1, \dots, q_n)$	a generált vizsga
$\mathcal{B} = \{(\kappa_j, T_j, b_j)\}$	a vizsgaterv slotjai: fogalom, típus, Bloom-szint
$G(q), A(q), V(q)$	alátámasztottság, vak megoldhatóság, disztraktor-validitás (bírói ítéletek)

$F(\mathcal{X})$	formai megfelelés indikátora
$\text{Cov}(\mathcal{X})$	darablefedettség
$Q(\mathcal{X})$	minőségi index, a négy ráta átlaga
$L(q)$	válaszszivárgás indikátora (kulcs a törzsben)
κ	Cohen-féle egyezési együttható (a szövegkörnyezetből egyértelmű, hogy nem fogalom)
α	Krippendorff-féle megbízhatósági együttható
r_{rb}	páros rangbiszeriális hatásméret

1. Bevezetés

1.1. Motiváció

A felsőoktatásban és a középiskolában a feleletválasztós kérdés (*multiple-choice question*, MCQ) a tudásmérés egyik legszélesebb körben használt eszköze: gépi úton javítható, nagy létszámú kurzusokon is skálázható, és jól illeszkedik a tanulásmenedzsment-rendszerek (pl. Moodle) kvízmoduljaihoz. Előállításában azonban drága. A pszichometriai szakirodalom szerint egy jó kérdés törzse önmagában is érthető és egyértelmű, pontosan egy válaszlehetősége helyes a tananyag szerint, a disztraktorok pedig azonos témájúak, formailag hasonlóak és egy felkészületlen hallgató számára hihetőek, ám igazolhatóan hibásak [1, 2]. Egy tapasztalt oktató egy-egy ilyen kérdéssel percek, egy teljes vizsgán órákat dolgozik, és a disztraktorok megírása a munka legnehezebb része.

A nagy nyelvi modellek (*large language model*, LLM) megjelenése [3] gyökeresen átalakította ezt a munkafolyamatot. Az oktatástechnológiai irodalom egyszerre hangsúlyozza a lehetőségeket és a kockázatokat [4]: a modellek képesek kérdéseket és disztraktorokat fogalmazni [5, 6], ugyanakkor hajlamosak folyékony, de a forrás által nem alátámasztott állításokat generálni [7]. A gyakorlatban a legelterjedtebb használati mód a legegyszerűbb: az oktató feltölti az előadásjegyzetét egy felhőben futó csevegő asszisztensbe, és „tíz feleletválasztós kérdést” kér. Ezt a munkafolyamatot a továbbiakban *status quo*-nak vagy *közvetlen megközelítésnek* nevezzük.

1.2. Problémafelvetés

A közvetlen megközelítés kényelmes, de az iskolai alkalmazás szempontjából négy, egymástól független gyengesége van. Ezek adják dolgozatunk kiindulópontját.

- (P1) A helyesség nincs ellenőrizve.** Semmi sem garantálja, hogy a megjelölt kulcsot a dokumentum alátámasztja, és hogy a disztraktorok valóban hamisak. Az LLM-ek hihetőek, de alá nem támasztott állításokat állítanak elő [7]; az önellenőrzés ezt csökkentheti, de nem szünteti meg [8–10].
- (P2) A szerkezet és a lefedettség nincs kontrollálva.** A kért darabszám, típus és nehézség csak kérdés a promptban. A hosszú bemenetet a modellek egyenetlenül használják: a kontextus elejére és végére jobban figyelnek, mint a közepére [11], a névleges kontextushossznál jóval rövidebb az a hossz, amelyen még megbízhatóan dolgoznak [12]. Ennek következménye, hogy a kérdések a dokumentum néhány szakaszára tömörülnek.
- (P3) Adatvédelem.** A tananyag személyes adatot (pl. esettanulmányban szereplő neveket, hallgatói munkákat) is tartalmazhat; a felhőszolgáltatás az adatkezelés címzettjévé válik [13], az intézmény pedig elveszíti a kontrollt a tartalom sorsa felett.
- (P4) Függés a nagy, távoli modellektől.** A teljes dokumentum egyetlen promptban hosszú

kontextust és nagy modellt igényel. Az iskola által saját hardveren futtatható kis modellek (néhány milliárd paraméter, 8 GB videomemória) kontextusa és memóriája korlátos.

Fontos látni, hogy a négy probléma nem oldható meg egyszerűen „jobb prompttal”. A P1 és a P2 a *módszer* hiányából fakad: ha a vizsga szerkezete, a kérdések forrása és az ellenőrzés nem explicit lépések, akkor nem is mérhetők és nem is kényszeríthetők ki. A P3 és a P4 pedig egymást erősítik: a helyi futtatás megoldaná az adatvédelmi problémát, de a helyben futtatható kis modellek éppen a hosszú kontextusú, egylépéses feladatban a leggyengébbek. Kutatásunk alapgondolata ezért az, hogy a feladatot rövid, jól specifikált, egyenként ellenőrizhető lépésekre bontva egy kis, helyben futó modell is képes lehet megbízható vizsgát előállítani, és ez a felbontás egyben a P1 és P2 problémát is kezeli.

1.3. Kutatási kérdések és hipotézisek

A fenti problémák alapján három kutatási kérdést fogalmaztunk meg.

- RQ1** A strukturált generálás – tervezés, kérdésenkénti forráshoz kötés és ellenőrzés – érvényesebb és jobban lefedő vizsgát eredményez-e, mint ha a modellnek a dokumentumot (vagy annak visszakeresett részleteit) egyetlen promptban adjuk át?
- RQ2** Mennyivel járul hozzá a minőséghez az egyes komponens (tervező, ellenőrző, fogalomgráf, hibrid visszakeresés, darabolási stratégia), és mennyibe kerül futási időben, modellszámokban és tokenben?
- RQ3** Mennyire közelíti meg egy 8 GB-os fogyasztói GPU-n futó, 4 bitre kvantált 7B modell egy nagy, szerveren futó modell teljesítményét?

A kutatási kérdésekhez a következő, a fő vizsgálatban tesztelendő hipotéziseket rendeljük. A pilot vizsgálat (7. fejezet) mindegyikhez szolgáltat előzetes, leíró jellegű bizonyítékot, szignifikanciavizsgálatra azonban egyetlen dokumentum nem elegendő.

1. Hipotézis (Tervezés). Azonos generátormodell mellett a vizsgaterv (E1) a naiv RAG alapvonalhoz (E0) képest növeli a kérdések alátámasztottságát és a vizsga darablefedettségét, a formai megfelelés romlása nélkül.

2. Hipotézis (Ellenőrzés). Az ellenőrző lánc (E2) a tervezéshez (E1) képest tovább növeli az alátámasztottságot és a vak megoldhatóságot; az ellenőrző ítélete pozitívan egyezik a független bíró ítéletével ($\kappa > 0$).

3. Hipotézis (Lefedettség). A teljes dokumentumot egyetlen promptban kapó alapvonal (B-doc) lefedettsége – modellszámtól függetlenül – alacsonyabb, mint a tervezett változatoké.

4. Hipotézis (Helyi vs. szerver). A helyi 7B modell a tervezett és ellenőrzött csővezetékben a szervermodell naiv változatának minőségi indexét megközelíti (különbség $< 0,1$), miközben a GPU-memóriaigénye 8 GB alatt marad.

5. Hipotézis (Mérés). A generátoroktól eltérő modelleszaládba tartozó LLM-bíró ítéletei az oktatói értékelésekkel legalább elfogadható megbízhatósággal egyeznek ($Krippendorff-\alpha \geq 0,667$).

1.4. Hozzájárulások

Dolgozatunk négy, egymásra épülő hozzájárulást tartalmaz.

- (1) **A MIMIR generálási módszer** (5. fejezet): ablakozott mondatbeágyazáson alapuló szemantikus darabolás, lefedettségvezérelt vizsgaterv a legnagyobb maradék elvén kiosztott Bloom-szintekkel, slotonkénti sűrű vagy hibrid visszakeresés, forrásrészletekre hivatkozó egyenkénti kérdésgenerálás, legolcsóbbtól a legdrágábbig rendezett ellenőrző lánc visszacsatolással és tartalékfogalmakkal, valamint egy munkamenet-hatókörű fogalomgráf (GraphRAG-lite). A módszert formálisan is megfogalmazzuk (3. fejezet), és algoritmikusan leírjuk.
- (2) **Adatvédelmet biztosító helyi architektúra** (4. fejezet): lazán csatolt mikroszolgáltatások, PostgreSQL-alapú zárolásmentes feladatsor, zéró adatmegőrzés és ujjlenyomat-alapú audit, valamint a GPU-memóriaigény analitikus modellje, amelyet mérésekkel validálunk.
- (3) **Kiértékelési módszertan és nyílt keretrendszer** (6. fejezet): tizenöt egytényezős kísérleti kar, kétnyelvű adatkészlet referenciakérdésekkel, formálisan definiált mutatók, modelleszaládon kívüli LLM-bíró, oktatói értékelési protokoll és dokumentumszintű statisztika.
- (4) **Empirikus eredmények** (7. fejezet): a pilot vizsgálat számszerű eredményei, köztük két korábbi méréseket érvénytelenítő mérési hiba feltárása, a válaszszivárgás mint új hibamód leírása és mérése, az önellenőrzés osztályozóként való validálása, valamint a futásról futásra jelentkező zaj becslése, amelyekből a fő vizsgálat terve közvetlenül következik.

1.5. A dolgozat felépítése

A 2. fejezet áttekinti az automatikus kérdésgenerálás, a visszakereséssel kiegészített generálás, az ellenőrzés és a nyelvimodell-alapú kiértékelés szakirodalmát, és azonosítja a kutatási rést. A 3. fejezet a feladatot formálisan definiálja, bevezeti a minőségi predikátumokat és a célfüggvényt. A 4. fejezet a rendszer architektúráját, adatvédelmi mechanizmusait és erőforrás-modelljét mutatja be. Az 5. fejezet a generálási módszer algoritmusait és azok költségelemzését tárgyalja. A 6. fejezet a kiértékelési módszertant, a 7. fejezet a pilot eredményeit ismerteti. A 8. fejezet az eredmények értelmezését és az érvényességet veszélyeztető tényezőket, a 9. fejezet az összefoglalást és a további munkát tartalmazza. A függelék a promptokat, a konfigurációkat és a reprodukálás lépéseit közli.

2. Szakirodalmi áttekintés

Ebben a fejezetben azokat a kutatási irányokat tekintjük át, amelyekre a MIMIR módszere és kiértékelése épül: az automatikus kérdés- és disztraktorgenerálást, a visszakereséssel kiegészített generálást, a hosszú kontextus problémáit, az ellenőrzést és önjavítást, a kis és kvantált modelleket, valamint a nyelvi modellekkel végzett kiértékelést. A fejezet végén összevetjük a meglévő megközelítéseket, és megfogalmazzuk a kutatási rést.

2.1. Automatikus kérdésgenerálás

Az automatikus kérdésgenerálás (*automatic question generation*, AQG) több évtizedes múltat tekint vissza. Kurdi és munkatársai szisztematikus irodalmi áttekintése [5] 93 oktatási célú AQG-rendszert elemzett, és három generációt különített el. Az *első* generáció szabály- és sablonalapú: szintaktikai elemzéssel azonosítja a kérdezhető mondatrészeket, és transzformációs szabályokkal kérdő mondatná alakítja őket. Mitkov és Ha rendszere [14] például főnévi csoportokat választ ki kulcsként, és a kulcshoz szemantikusan közeli fogalmakat (WordNet-szomszédokat) javasol disztraktornak. Heilman és Smith [15] a túlgenerálás és rangsorolás elvét vezette be: a szabályok sok jelöltet állítanak elő, amelyeket egy statisztikai modell rangsorol. A *második* generáció neurális szekvencia–szekvencia modelleket használ; Du és munkatársai [16] figyelemmechanizmusos kódoló–dekódoló hálózattal olvasásértési kérdéseket generáltak. A *harmadik* generációt az előtanított transzformerek [17, 18] és a nagy nyelvi modellek [3] jelentik, amelyek utasításkövetéssel, finomhangolás nélkül is képesek kérdés–válasz párokat írni.

Az áttekintések visszatérő megállapítása, hogy a kiértékelés a terület gyenge pontja. Mulla és Gharpure [19] szerint a rendszerek jelentős részét referencia-átfedésen alapuló metrikákkal (BLEU [20], ROUGE [21]) mérik, amelyek egy referenciakérdéshez való lexikális hasonlóságot jutalmaznak. Egy vizsgakérdés azonban akkor is kiváló lehet, ha egyetlen szava sem egyezik a referenciával, és akkor is használhatatlan, ha a kulcsa hibás. A humán értékelések jellemzően kis mintán, nem vakon történnek, és ritkán ellenőrzik a kulcs helyességét [5]. Munkánkban ezért a referenciakérdéseket nem célként, hanem csak a visszakeresés és a tematikus hasonlóság mérésére használjuk, a helyességet pedig explicit, forrásalapú predikátumokkal mérjük.

2.2. Disztraktorgenerálás és tesztelméleti követelmények

A feleletválasztós kérdés minőségét döntően a disztraktorok határozzák meg. Haladyna, Downing és Rodriguez [1] 31 kérdésírási irányelvet összegző munkája szerint a disztraktoroknak hihetőnek, egymást kölcsönösen kizárónak, a kulccsal azonos hosszúságúnak és nyelvtani formájúnak kell lenniük, és kerülni kell a „mindegyik fenti” típusú opciókat. Gierl és munkatársai [2] áttekintése a disztraktorok kidolgozásának, elemzésének és felhasználásának gyakorlatát foglalja össze, és

rámutat, hogy a nem funkcionáló (senki által nem választott) disztraktor gyakorlatilag csökkenti az opciók számát, így növeli a találgatás esélyét.

Alhazmi és munkatársai [6] a disztraktorgenerálás módszereit három csoportba sorolják: tudásbázis- és ontológiaalapú (hasonló fogalmak lexikai adatbázisból), korpuszalapú (disztribúciós hasonlóság) és generatív (neurális vagy LLM-alapú) módszerekre. Megállapításuk szerint a generatív módszerek a legfolyékonyabb disztraktorokat adják, ám gyakran előfordul, hogy egy disztraktor valójában szintén helyes, vagy nyilvánvalóan abszurd. A MIMIR ezt két ponton kezeli: a fogalomgráf-alapú változat a dokumentumból kinyert *testvérfogalmakat* (azonos szülőfogalom alá tartozó fogalmakat) ajánlja disztraktor-ötletként, az ellenőrző pedig minden disztraktorra külön ítéletet kér arról, hogy a forrás szerint hamis-e.

2.3. A kognitív szint: Bloom taxonómiája

A kérdések nehézségének szabályozására a legelterjedtebb keret Bloom taxonómiája [22] és annak Anderson és Krathwohl által átdolgozott változata [23]. A revideált taxonómia kognitív dimenziója hat, egymásra épülő szintet különít el: emlékezés, megértés, alkalmazás, elemzés, értékelés és alkotás. A feleletválasztós formátum az első öt szint mérésére alkalmas; az alkotás jellemzően nyitott feladatot igényel. A MIMIR a kért nehézséghez egy Bloom-szinteloszlást rendel (5.6. szakasz), és ezt az eloszlást a legnagyobb maradék elvén diszkrétizálja a vizsga slotjaira, így a nehézség a promptban tett kérdés helyett a terv szerkezeti tulajdonsága lesz.

2.4. Visszakereséssel kiegészített generálás

A visszakereséssel kiegészített generálás (RAG) Lewis és munkatársai [24] munkájában jelent meg: egy sűrű visszakereső (DPR [25]) a kérdéshez releváns szövegrészeket keres egy külső korpuszból, és a generátor ezekre támaszkodva válaszol. Azóta a RAG a nyelvi modellek tényszerűségének növelésére szolgáló legelterjedtebb technikává vált; Gao és munkatársai áttekintése [26] a „naiv”, a „fejlett” és a „moduláris” RAG paradigmáit különíti el. A naiv RAG egyetlen visszakeresést és egyetlen generálást végez; a fejlett RAG a visszakeresés előtt és után optimalizál (lekérdezés-átírás, újrarangsorolás); a moduláris RAG pedig tetszőleges sorrendben kombinálja a visszakeresési, generálási és ellenőrzési modulokat. A MIMIR tervezett csővezetéke ebben a terminológiában moduláris RAG: a visszakeresés a terv minden slotjára külön fut, és az ellenőrzés eredménye visszacsatol a visszakeresésre.

Sűrű és lexikális visszakeresés. A sűrű visszakeresés a szövegeket egy közös vektortérbe képezi. A mondatbeágyazások Sentence-BERT óta [27] színi hálózatokkal, kontrasztív tanítással készülnek; az E5 modelles család [28] gyengén felügyelt kontrasztív előtanítással készült, többnyelvű változata [29] a magyart is támogatja. A lexikális visszakeresés klasszikus módszere a BM25 [30], amely a kifejezés gyakoriságot telítő függvényrel és dokumentumhossz-normalizálással súlyozza. A két módszer hibái eltérnek: a sűrű visszakeresés a szinonimákat és parafrázisokat kezeli jól, a

lexikális a ritka szakkifejezéseket és tulajdonneveket. Az eredmények egyesítésére a reciprok rangfúzió (RRF) [31] egyszerű és robusztus módszer, amely csak a rangokat használja, így nem igényli a heterogén pontszámok kalibrálását.

Darabolás. A visszakeresés egysége a darab (*chunk*). A rögzített méretű ablakok egyszerűek, de mondatok és gondolatmenetek közepén vágnak. A szemantikus darabolás a szomszédos mondatok beágyazásainak távolságát figyeli, és ott vág, ahol a téma megváltozik. Qu, Tu és Bao [32] szisztematikusan vizsgálták, megéri-e a szemantikus darabolás számítási többletköltsége, és azt találták, hogy előnye adatkészletfüggő, és gyakran kisebb, mint várnánk. Ez indokolja, hogy a darabolási stratégiát mi is külön kísérleti karokkal (E5a–c) vizsgáljuk, ahelyett hogy eleve jobbnak tekintenénk.

Gráfalapú RAG. Edge és munkatársai GraphRAG módszere [33] a korpuszból nyelvi modellel entitás- és relációgráfot épít, a gráfot közösségekre bontja, és a közösségek összefoglalóival válaszol a korpusz egészére vonatkozó („globális”) kérdésekre. A módszer drága: a teljes korpuszt többször fel kell dolgozni. A MIMIR ennek egy könnyített, munkamenet-hatókörű változatát használja: a fogalomgráf egyetlen dokumentumra, kötegenként egy hívással készül, és csak a vizsga élettartamáig létezik.

2.5. Hosszú kontextus: az egylépéses alternatíva korlátai

A RAG alternatívája az, hogy a teljes dokumentumot egyetlen promptban adjuk át a modellnek – ez felel meg az oktatók napi gyakorlatának. Liu és munkatársai [11] kimutatták, hogy a modellek teljesítménye U alakú függvénye a releváns információ pozíciójának: a kontextus elején és végén elhelyezett információt jobban használják, mint a közepén lévő („lost in the middle”). Hsieh és munkatársai RULER benchmarkja [12] szerint a legtöbb modell „tényleges” kontextushossza jóval rövidebb a névlegesnél. Vizsgagenerálásra vetítve ez azt jelenti, hogy a teljes dokumentumot kapó modell kérdései várhatóan a dokumentum elejére és végére tömörülnek, ami a lefedettség mérésével közvetlenül tesztelhető (3. hipotézis).

2.6. Hallucináció, ellenőrzés és önjavítás

Ji és munkatársai [7] a hallucinációt úgy definiálják, mint a forrással ellentmondó (*intrinzikus*) vagy abból nem igazolható (*extrinzikus*) generált tartalmat. A kérdésgenerálásban mindkettő előfordul: a kulcs ellentmondhat a forrásnak, vagy olyan tényen alapulhat, amely a forrásban nem szerepel.

A hallucináció csökkentésére az irodalom két nagy stratégiát javasol. Az *önjavító* módszerek (Self-Refine [8]) a modell saját visszajelzésére építve iteratívan javítják a kimenetet; a Chain-of-Verification [9] ellenőrző kérdéseket generál, ezekre egymástól függetlenül válaszol, majd a válaszok alapján módosítja az eredeti kimenetet. A SelfCheckGPT [34] mintavételezett válaszok konzisztenciájából becsüli a hallucinációt, a FActScore [35] pedig atomi tényekre bontja a

szöveget, és ezeket egyenként ellenőrzi egy tudásforrással szemben. Huang és munkatársai [10] ugyanakkor kimutatták, hogy külső visszajelzés nélkül a modellek önjavítása érvelési feladatokban nem javít, sőt ronthat is. Snell és munkatársai [36] azt mutatták meg, hogy a következtetési időben elköltött számítás (több minta, ellenőrzés) bizonyos feladatokban helyettesítheti a modellméretet – ez adja az elméleti alapját annak, hogy egy kis modell ellenőrző láncsal megközelítheti egy nagy modell minőségét.

A MIMIR ellenőrző lánc a két irány metszetében helyezkedik el: külső, determinisztikus szabályokat (forma, meta-hivatkozás, hivatkozás, szivárgás, duplikátum) kombinál a forrással szembeni, nyelvi modellel végzett ítéletekkel (alátámasztottság, disztraktorok, vak megoldás). Pilot eredményeink (7.4. szakasz) Huang és munkatársai megállapításával összhangban azt mutatják, hogy a generátorral azonos kis modellel végzett ítélet nem hordoz érdemi információt.

2.7. Kis nyelvi modellek és kvantálás

A helyi futtatás kulcskérdése a memória. Egy P paraméteres modell súlyai 16 bites lebegőpontos ábrázolásban $2P$ bájtot foglalnak, ami 7 milliárd paraméternél 14 GB – ez nem fér el egy fogyasztói kártyán. A kvantálás a súlyokat alacsonyabb bitszámon tárolja: az LLM.int8() [37] 8 bites, a GPTQ [38] 3–4 bites utólagos kvantálást vezetett be. A llama.cpp projekt [39] GGUF formátuma és csoportos K-quantálása (pl. Q4_K_M) a helyi futtatás de facto szabványa lett; erre épül az általunk használt Ollama futtatókörnyezet [40]. Átfogó kiértékelések szerint a 4 bites súlykvantálás általános benchmarkokon csak kis pontosságvesztést okoz [41–43]. A memóriaigény másik összetevője a kulcs–érték (KV) gyorsítótár, amely a kontextushosszal lineárisan nő; a csoportosított lekérdezésű figyelem (GQA) [44] a KV-fejek számának csökkentésével ezt jelentősen mérsékli, a PagedAttention [45] pedig a töredezettséget szünteti meg; a helyi futtatás energiaigényét fogyasztói hardveren Zähl és Hennig mérte [46].

Az oktatási kérdés-generálásban Fawzi és munkatársai [47, 48] kimutatták, hogy kis modellek további előtanítással és finomhangolással a nagy modellekhez mérhető kérdéseket állíthatnak elő, Jaldi és munkatársai [49] pedig kis, privát modelleket hasonlítottak össze nagy modellekkel Bloom-szintekhez igazított értékelés tervezésében. A mi megközelítésünk ezekről abban tér el, hogy finomhangolás nélküli, általános célú modellt használunk, és a minőséget a *módszer* szerkezetével (terv, forráshoz kötés, ellenőrzés) igyekszünk biztosítani.

2.8. Nyelvi modell mint bíró

A generált szöveg minőségének skálázható mérésére egyre elterjedtebb az erős nyelvi modell bíróként való alkalmazása (*LLM-as-a-judge*). Zheng és munkatársai [50] szerint a GPT-4 bíró ítéletei az emberi preferenciákkal közel akkora mértékben egyeznek, mint az emberek egymással; a G-Eval [51] gondolatmenet-alapú, űrlapkitöltő értékeléssel javította az emberi egyezést. A bírók torzításai ugyanakkor jól dokumentáltak: pozíciós torzítás (az első vagy második válasz

2.1. táblázat. A kérdésgenerálási megközelítések összevetése a P1–P4 problémák szempontjából (✓ : megoldott, ○ : részben, – : nem kezelt).

Megközelítés	Kulcs ellenőrzése	Szerkezet, lefedettség	Helyi futás	Kis modell	Forrás-hivatkozás	Kulcs-validálás mérése
Szabály/sablon alapú [14, 15]	○	–	✓	✓	○	–
Neurális seq2seq [16]	–	–	✓	✓	–	–
Dokumentum → csevegő LLM (status quo)	–	–	–	–	–	–
Naiv RAG [24]	–	–	○	○	○	–
Finomhangolt kis modell [47, 48]	–	–	✓	✓	–	○
Önjavítás / CoVe [8, 9]	○	–	–	–	–	–
GraphRAG [33]	–	○	–	–	○	–
MIMIR (jelen munka)	✓	✓	✓	✓	✓	✓

preferálása) [52], a hosszabb válaszok előnyben részesítése, és az *önpreferencia*: Panickssery és munkatársai [53] kimutatták, hogy a modellek felismerik és előnyben részesítik saját generációikat. Ebből két tervezési döntésünk következik: a bíró más modelleszaládba tartozik, mint bármelyik generátor (gpt-oss [54] a Qwen- és Llama-generátorokkal szemben), a vak megoldási próbában pedig a válaszlehetőségek sorrendjét véletlenszerűen permutáljuk. A RAG-rendszerek referenciamentes kiértékelésére a RAGAS [55] hasonló, nyelvi modellel számolt hűség- és relevanciamutatókat javasol; mi ezeket a vizsgakérdés speciális szerkezetéhez (kulcs, disztraktorok, vak megoldás) igazítottuk.

2.9. Magyar nyelvi erőforrások és szabályozási környezet

A magyar nyelv agglutináló morfológiája és viszonylag kis erőforrás-háttér külön kihívás. A HuLU benchmark [56] a magyar nyelvértési feladatok egységes kiértékelési keretét adja, a MILQA [57] pedig magyar Wikipédia-cikkekre épülő kérdés–válasz adatbázis, amelyet a referenciakérdések egyik forrásaként használunk. Tudomásunk szerint magyar nyelvű vizsgakérdés-generálásra még nem publikáltak rendszerszintű, szisztematikus kiértékelést.

A tananyag felhőszolgáltatónak való átadása a GDPR [13] értelmében adattovábbítás; az AI Act [58] az oktatási MI-rendszerek egy részét magas kockázatúnak minősíti, és emberi felügyeletet, valamint az MI-tartalom átláthatóságát várja el.

2.10. Összegzés és a kutatási rés

A P1–P4 szempontok szerinti összevetésből (2.1. táblázat) három hiányosság rajzolódik ki: (i) nincs olyan nyílt rendszer, amely a kérdésgenerálást explicit tervvel, forráshivatkozással és ellenőrzéssel, fogyasztói GPU-n futó kis modellel végezné; (ii) a kiértékelés ritkán méri a kulcs helyességét és a lefedettséget, és hiányoznak az egytényezős (ablációs) tervek; (iii) a magyar nyelvű vizsgagenerálás kiértékelése feltáratlan. Ezekre rendre az 5., a 6. és a 7. fejezet ad választ.

3. A feladat formális modellje

A vizsgagenerálás minőségéről csak akkor lehet szigorúan beszélni, ha a „jó vizsga” fogalmát mérhető predikátumokra bontjuk. Ebben a fejezetben bevezetjük a dokumentum, a darabolás, a vizsgaspecifikáció és a kérdés formális modelljét, definiáljuk a minőségi predikátumokat és az ezekből képzett mutatókat, majd a módszer két szerkezeti tulajdonságát bizonyítjuk. A definíciók egyúttal a 6. fejezet mutatóinak pontos specifikációi is: a kiértékelő keretrendszer (`tests/eval/mimir_eval`) ezeket valósítja meg.

3.1. Dokumentum, mondatok és darabok

3.1. Definíció (Dokumentum és mondatfelbontás). A *dokumentum* egy $\mathcal{D} \in \Sigma^*$ karaktersorozat. A mondatfelbontás a dokumentumot diszjunkt, egymást követő intervallumokra bontja: $S = (s_1, \dots, s_m)$, ahol minden s_i egy $[a_i, b_i)$ karakterintervallumhoz tartozik és $b_i \leq a_{i+1}$.

3.2. Definíció (Darabolás). A *darabolás* a mondat sorozat egy összefüggő partíciója: $\mathcal{C} = (c_1, \dots, c_K)$, ahol $c_k = (s_{j_{k-1}+1}, \dots, s_{j_k})$ és $0 = j_0 < j_1 < \dots < j_K = m$. A darabolást a vágási pontok $J = \{j_1, \dots, j_{K-1}\}$ halmaza egyértelműen meghatározza.

Legyen $\phi : \Sigma^* \rightarrow \mathbb{S}^{d-1}$ egységnyi normájú beágyazás, és jelölje $\langle \cdot, \cdot \rangle$ a skaláris szorzatot, így a koszinusz-hasonlóság egységvektorokra $\cos(u, v) = \langle u, v \rangle$. A szomszédos mondatok *szemantikus távolsága*

$$\delta_i = 1 - \cos(\phi(s_i), \phi(s_{i+1})), \quad i = 1, \dots, m-1. \quad (3.1)$$

A szemantikus darabolás alapgondolata, hogy a témaváltásoknál δ_i nagy. A pontos vágási szabályt (küszöb és méretfüggő érzékenység) az 5.3. szakaszban adjuk meg.

3.2. Vizsgaspecifikáció és kérdés

3.3. Definíció (Vizsgaspecifikáció). A *vizsgaspecifikáció* egy $\sigma = (n, T, \lambda, \ell)$ négyes, ahol $n \in \mathbb{N}^+$ a kérdések száma, $T \subseteq \{\text{mcq}, \text{tf}, \text{open}\}$ a megengedett típusok sorozata, λ a nehézség (könnyű, közepes vagy nehéz), ℓ pedig a nyelv.

3.4. Definíció (Kérdés). Egy *kérdés* a $q = (\theta, t, O, R, b)$ ötös, ahol $\theta \in T$ a típus, t a kérdés törzse, $O = (o_1, \dots, o_{|O|})$ a válaszlehetőségek sorozata, amelyek közül a $\kappa(o) \in \{0, 1\}$ címke jelöli a helyeseket, $R \subseteq \{1, \dots, K\}$ a kérdés által hivatkozott darabok indexhalmaza, b pedig a tervezett Bloom-szint. Egyetlen helyes lehetőség esetén ezt *kulcsnak* nevezzük és o^* -gal jelöljük; a többi lehetőség a *disztraktorok* halmaza: $O^- = \{o \in O : \kappa(o) = 0\}$.

A típusonként elvárt opciószámot $m_{\text{mcq}} = 4$, $m_{\text{tf}} = 2$, $m_{\text{open}} = 1$ jelöli.

3.5. Definíció (Formai helyesség). A q kérdés *formailag helyes*, ha

$$\text{Shape}(q) = \mathbb{K} \left[|O| = m_\theta \wedge \sum_{o \in O} \kappa(o) = 1 \right] \quad (\theta \in \{\text{mcq}, \text{tf}\}), \quad (3.2)$$

nyitott kérdésnél pedig $\text{Shape}(q) = \mathbb{K}[\exists o \in O : \kappa(o) = 1 \wedge o \neq \varepsilon]$. Az $\mathcal{X} = (q_1, \dots, q_N)$ vizsga *formai megfelelése*

$$F(\mathcal{X}) = \mathbb{K} \left[N = n \wedge \forall i : \text{Shape}(q_i) = 1 \wedge \forall i : \theta_i \in T \right]. \quad (3.3)$$

3.3. Forrásalapú minőségi predikátumok

A kérdés tartalmi helyessége a forráshoz viszonyított fogalom. Jelölje $\text{Src}(q) \subseteq \mathcal{D}$ azt a szövegrészt, amelyből a kérdés készült (a hivatkozott vagy a kontextusként átadott darabok konkatenációja), és $\models$ a természetes nyelvi következményrelációt: $X \models y$, ha egy figyelmes olvasó X alapján y -t igaznak fogadja el. A $\models$ reláció nem számítható ki pontosan; a kiértékelésben egy független bíró (6.5. szakasz) közelíti, az oktatói értékelés pedig ezt a közelítést validálja.

3.6. Definíció (Alátámasztottság). A q kérdés *alátámasztott*, ha a forrás igazolja a kulcsot: $G(q) = \mathbb{K}[\text{Src}(q) \models o^* \text{ mint } t \text{ válasza}]$.

3.7. Definíció (Disztraktor-validitás). A q kérdés disztraktor-validitása a forrás szerint hamis és témába vágó disztraktorok aránya:

$$V(q) = \frac{1}{|O^-|} \sum_{o \in O^-} \mathbb{K}[\text{Src}(q) \models \neg o] \cdot \mathbb{K}[\text{onTopic}(o, t)]. \quad (3.4)$$

3.8. Definíció (Vak megoldhatóság). Legyen π az opciók egyenletes eloszlású véletlen permutációja, és $\mathcal{J}(t, \pi(O), \text{Src})$ egy megoldó választása, amely a kulcs ismerete nélkül, csak a forrás alapján dönt. A kérdés *vakon megoldható*, ha $A(q) = \mathbb{K}[\mathcal{J}(t, \pi(O), \text{Src}(q)) = o^*]$.

A vak megoldhatóság a kulcs helyességének és a kérdés egyértelműségének együttes, viselkedési mércéje: ha két opció is védhető, vagy a kulcs hibás, a megoldó várhatóan más opciót választ. A permutáció a bíró pozíciós torzítását [52] semlegesíti. Ha a megoldó nem ad értelmezhető választ, a kérdést a vak megoldhatóság átlagából kihagyjuk (nem nullának számítjuk).

3.4. Vizsgaszintű mutatók

3.9. Definíció (Minőségi index). Egy vizsga *minőségi indexe* a formai megfelelés és a három kérdésszintű ráta átlaga:

$$Q(\mathcal{X}) = \frac{1}{4} \left(F(\mathcal{X}) + \overline{G}(\mathcal{X}) + \overline{A}(\mathcal{X}) + \overline{V}(\mathcal{X}) \right), \quad \overline{X}(\mathcal{X}) = \frac{1}{|\mathcal{X}_X|} \sum_{q \in \mathcal{X}_X} X(q), \quad (3.5)$$

ahol $\mathcal{X}_X \subseteq \mathcal{X}$ azon kérdések halmaza, amelyekre az X predikátum értelmezett.

A Q index szándékosan egyszerű, súlyozatlan átlag: minden komponense $[0,1]$ -beli, a nagyobb érték jobb, és a formai megfelelés súlya egyenlő a tartalmi mutatókéval. Ennek következménye, hogy egyetlen hibás formájú kérdés egy tízkérdéses vizsgában a Q -t 0,25-tel csökkenti – a pilotban pontosan ez történt az ellenőrző karok egy-egy vizsgájában (7.4.1. szakasz).

3.10. Definíció (Lefedettségi). Legyen $v(q) = \arg \max_k \cos(\phi(t \oplus o^*), \phi(c_k))$ a kérdéshez (törzs és kulcs összefűzéséhez) legközelebbi darab. A vizsga *lefedettsége*

$$\text{Cov}(\mathcal{X}) = \frac{|\{v(q) : q \in \mathcal{X}\}|}{K}. \quad (3.6)$$

A lefedettség felső korlátja $\min(n, K)/K$; egy tízkérdéses vizsga egy négydarabos dokumentumon elvben teljes lefedettséget érhet el. A lefedettség nem minőségi, hanem *szerkezeti* mutató: azt méri, mennyire oszlik el a vizsga a tananyagon, ezért a Q indexbe nem számítjuk be, hanem mellette közöljük (a P2 probléma közvetlen mérőszáma).

3.11. Definíció (Válaszszivárgás és nem-kérdés törzs). Jelölje $W(x)$ az x szöveg szótokenjeinek halmazát. A kérdés *szivárogtatja a kulcsot*, ha

$$L(q) = \mathbb{K} \left[|W(o^*)| \geq 3 \wedge \frac{|W(o^*) \cap W(t)|}{|W(o^*)|} \geq 0,8 \right], \quad (3.7)$$

és a törzs *nem kérdés*, ha feleletválasztós kérdésnél sem kérdőjelre, sem kettőspontra nem végződik: $N(q) = \mathbb{K}[t \text{ utolsó írásjele} \notin \{?, : \}]$ (a záró idézőjeleket figyelmen kívül hagyva).

A háromszavas alsó korlát a rövid, egyszavas kulcsok (pl. „Arabica”) téves pozitív jelzését zárja ki: egy tulajdonnév természetesen szerepelhet a törzsben. Az L és N predikátumokat a pilot során feltárt hibamód (7.5. szakasz) motiválta; azóta mind mutatóként, mind az ellenőrző lánc szabályaként használjuk őket.

3.12. Definíció (Duplikátumráta). A vizsga duplikátumrátája azon kérdéspárok aránya, amelyek beágyazásainak koszinusz-hasonlósága meghaladja a 0,9 küszöböt:

$$\text{Dup}(\mathcal{X}) = \binom{N}{2}^{-1} \sum_{i < j} \mathbb{K}[\cos(\phi(q_i), \phi(q_j)) > 0,9].$$

3.5. A tervezési probléma

A fenti fogalmakkal a vizsgagenerálás egy korlátos optimalizálási feladatként írható fel. Jelölje Π a csővezetékek (módszerek) terét, $\mathcal{X}_\pi(\mathcal{D}, \sigma; \omega)$ a π csővezeték által ω véletlenmaggal előállított vizsgát, $C(\pi)$ a modellhívások számát, $\tau(\pi)$ a futási időt, $M(\pi)$ pedig a csúcs GPU-memóriát. A

cél

$$\max_{\pi \in \Pi} \mathbb{E}_{\mathcal{D}, \omega} [Q(\mathcal{X}_\pi)] \quad \text{f.h.} \quad \mathbb{E}[F(\mathcal{X}_\pi)] = 1, \quad M(\pi) \leq M_{\max}, \quad \mathbb{E}[\tau(\pi)] \leq \tau_{\max}, \quad (3.8)$$

másodlagos célként pedig a lefedettség, $\mathbb{E}[\text{Cov}(\mathcal{X}_\pi)]$ maximalizálása. A MIMIR esetében $M_{\max} = 8$ GB (egy fogyasztói kártya, amelyen az asztali környezet is fut), $\tau_{\max}$ pedig egy oktató számára elfogadható várakozási idő (néhány perc egy tízkérdéses vizsgára).

A kísérleti karok (6.3. szakasz) a Π tér egy-egy pontjai, amelyek egyszerre egy tényezőben különböznek. A (3.8) feladat *nem* oldható meg analitikusan, mert Q csak mintavétellel becsülhető; a kiértékelési módszertan éppen ennek a becslésnek a torzítatlanságát és pontosságát biztosítja.

3.6. A vizsgaterv szerkezeti tulajdonságai

A tervezett csővezeték két fontos tulajdonsága már a konstrukcióból következik, mérés nélkül is.

3.1. Állítás (Bloom-kvóta). Legyen $p : \mathcal{L} \rightarrow [0,1]$ a nehézséghez rendelt Bloom-eloszlás ($\sum_b p_b = 1$), és legyen c_b a legnagyobb maradék módszerével kapott darabszám: $c_b = \lfloor np_b \rfloor + \mathbb{1}_{[b \in \mathcal{T}]}$, ahol $\mathcal{T}$ a $\{np_b - \lfloor np_b \rfloor\}$ tört részek szerint csökkenő sorrendben első $n - \sum_b \lfloor np_b \rfloor$ szint. Ekkor $\sum_b c_b = n$ és minden b -re $|c_b - np_b| < 1$.

Bizonyítás. A $\lfloor np_b \rfloor$ értékek összege $n - r$, ahol $r = \sum_b (np_b - \lfloor np_b \rfloor)$ egész, mert $\sum_b np_b = n$; továbbá $0 \leq r < |\mathcal{L}|$. Pontosan r szint kap $+1$ -et, így az összeg n . Ha $b \in \mathcal{T}$, akkor $c_b - np_b = 1 - (np_b - \lfloor np_b \rfloor) \in (0,1]$, és az egyenlőség csak egész np_b esetén állna fenn, amikor a tört rész 0 – ilyen szint azonban csak akkor kerülhet $\mathcal{T}$ -be, ha minden tört rész 0 , ekkor viszont $r = 0$ és $\mathcal{T} = \emptyset$. Ha $b \notin \mathcal{T}$, akkor $c_b - np_b = -(np_b - \lfloor np_b \rfloor) \in (-1,0]$. $\square$

Az állítás a választási matematikából ismert Hamilton-módszer kvótatulajdonsága [59]. Példaként közepes nehézségnél ($p = (0,4; 0,4; 0,2)$ a megértés, alkalmazás, elemzés szintekre) és $n = 10$ esetén a kiosztás pontosan $(4,4,2)$; $n = 7$ esetén $(2,8; 2,8; 1,4) \rightarrow (3,3,1)$. A nehézség így nem kérés, hanem *garantált* szerkezeti tulajdonság.

3.2. Állítás (Tervezett lefedettség). Legyen $\mathcal{K} = \{\kappa_1, \dots, \kappa_P\}$ a tervező által javasolt fogalmak halmaza, $D(\kappa) \subseteq \{1, \dots, K\}$ a fogalmat alátámasztó darabok halmaza és $U = \bigcup_{\kappa \in \mathcal{K}} D(\kappa)$. A mohó slotkiosztás (5.2. algoritmus), amely minden lépésben olyan fogalmat választ, amely legalább egy még le nem fedett darabot érint, ha van ilyen, $n \leq P$ slot után legalább $\min(n, |U|)$ különböző darabot fed le.

Bizonyítás. Jelölje H_s az első s kiválasztott fogalom által lefedett darabok halmazát. Az $(s+1)$ -edik lépésben két eset lehetséges. Ha van olyan megmaradt κ , amelyre $D(\kappa) \setminus H_s \neq \emptyset$, akkor az algoritmus ilyet választ, így $|H_{s+1}| \geq |H_s| + 1$. Ha nincs ilyen, akkor minden megmaradt fogalom darabjai már H_s -ben vannak, és a már kiválasztottaké definíció szerint is, így $H_s \supseteq U$, azaz $|H_s| = |U|$. Indukcióval $|H_n| \geq \min(n, |U|)$. $\square$

Megjegyezzük, hogy a klasszikus maximális lefedési feladatra a határhaszon szerinti mohó algoritmus $(1 - 1/e)$ -approximációt ad [60]. A MIMIR kiosztása ettől szándékosan eltér: a választás lexikografikus kulcsa (*új darabot érint-e, fontosság*), vagyis az újdonság csak bináris feltétel, és azon belül a tervező által becsült fontosság dönt. Ezzel az approximációs garanciát a tartalmi relevanciáért cseréljük, a 3.2. állítás alsó korlátja azonban megmarad. A tervezett lefedettség (a kontextusként felhasznált darabok halmaza) és a (3.6) szerint mért lefedettség nem azonos: az utóbbi azt méri, miről szól ténylegesen a kész kérdés.

3.7. A modellhívások számának korlátai

A tervezett csővezeték költségét elsősorban a nyelvmodell-hívások száma határozza meg. Legyen r a slotonkénti újrapróbálkozások maximális száma, és tegyük fel, hogy minden hívás legfeljebb egyszer ismétlődik értelmezhetetlen JSON miatt. Egy *kísérlet* legfeljebb három sikeres hívásból áll (generálás, alátámasztottság-ellenőrzés, vak megoldás), egy slot legfeljebb $r + 1$ kísérletből, sikertelenség esetén pedig egy tartalékfogalommal újabb legfeljebb $r + 1$ kísérletből és egy javító hívásból. Így az ellenőrzött csővezeték hívásszáma

$$1 + 3n \leq C_{E2} \leq 2 + n(2 \cdot 2 \cdot 3(r + 1) + 2), \quad (3.9)$$

ahol a felső korlátban a tényezők rendre a tartalékfogalmat, a JSON-ismétlést és a kísérletenkénti három hívást, a $+2$ a javító hívást, a konstans 2 pedig a (legfeljebb egyszer ismételt) tervezőhívást jelöli; az alsó korlát az az eset, amikor minden kérdés elsőre átmegy (egy tervezőhívás, majd slotonként három hívás). Ha egy kísérlet p valószínűséggel megy át, függetlenül a többitől, akkor a slotonkénti kísérletek várható száma a csonkolt geometriai eloszlás várható értéke:

$$\mathbb{E}[a] = \sum_{j=0}^r (1-p)^j = \frac{1 - (1-p)^{r+1}}{p}. \quad (3.10)$$

Az egyhívásos karok (E0, E5a–c, B-doc) hívásszáma 1, az ellenőrzés nélküli E1 kar alsó korlátja $1 + n$, az E2f karé $1 + 2n$, a gráfalapú E3 karé pedig $G + 3n$, ahol $G = \lceil |\mathcal{D}|/3000 \rceil$ a gráfkegyek száma. A pilotban ($n = 10$, $r = 2$) az E2 kar átlagosan 75 hívást igényelt, ami slotonként 7,4 hívásnak felel meg – ez messze az alsó korlát (3) felett van, és jelzi, hogy a kis modell ellenőrző lánc a kérdések jelentős részét elutasította (7.7. szakasz). A naiv alapvonal ezzel szemben $C_{E0} = 1$ hívással dolgozik; a két módszer közötti költségkülönbség tehát két nagyságrend, amelyet a minőségi nyereségnek kell igazolnia.

4. Rendszerarchitektúra és helyi üzemeltetés

Ebben a fejezetben a MIMIR rendszer felépítését mutatjuk be abból a szempontból, amely a kutatási kérdések szempontjából releváns: hogyan valósítja meg a rendszer a helyi futtatást (P4), a zéró adatmegőrzést (P3) és a mérhetőséget. A felhasználói felület és az exportálók részleteit csak a teljesség kedvéért érintjük.

4.1. Tervezési célok

A P1–P4 problémákból és az AI Act emberi felügyeletre vonatkozó elvárásából [58] öt tervezési célt vezettünk le. A 4.1. táblázat ezeket a közvetlen megközelítéssel veti össze.

4.1. táblázat. Tervezési célok: a dokumentum csevegő LLM-nek való átadása és a MIMIR összevetése.

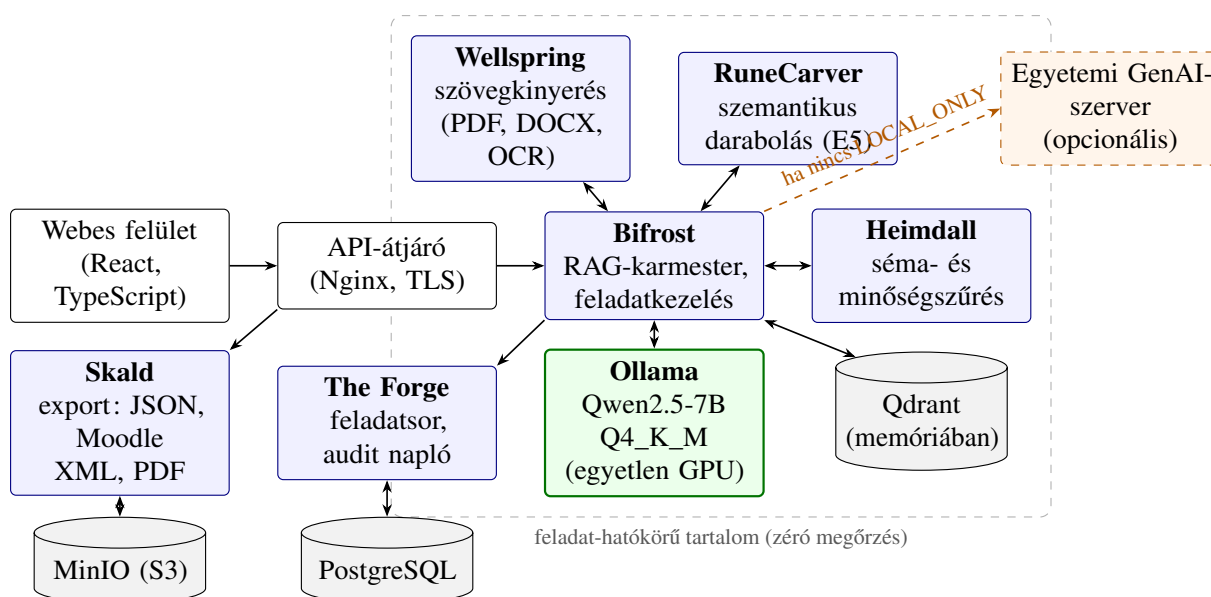
Cél	Dokumentum → csevegő LLM	MIMIR
G1 Ellenőrizhető	A kulcs nincs ellenőrizve, nincs forráshivatkozás	Minden kérdés darabokra hivatkozik; alátámasztottsági és vak megoldási ellenőrzés; az ellenőrzésen át nem jutott kérdés megjelölve
G2 Kontrollált	Darabszám, típus, nehézség csak kérdés	A terv rögzíti a darabszámot, típusokat, Bloom-szinteket; lefedettségvezérelt fogalomválasztás
G3 Adatvédő	Az anyag a szolgáltatóhoz kerül	Helyi modell; a tartalom a feladat végén törlődik; kapcsolható kizárólag helyi mód
G4 Megvalósítható	Hosszú kontextus, nagy modell	Rövid hívások 8 192–16 384 tokenes kontextusban; 7B modell 8 GB-on
G5 Felügyelt	A kimenet változtatás nélkül használható	Kötelező oktatói jóváhagyás szerkesztőben, a hivatkozott forrással; MI-közreműködés jelölése az exportban

4.2. Mikroszolgáltatás-architektúra

A rendszer lazán csatolt, konténerizált szolgáltatásokból áll, amelyeket egy Nginx-alapú API-átjáró fog össze (4.1. ábra). A szolgáltatások az északi mitológia tudással kapcsolatos alakjairól kapták nevüket; feladatukat a 4.2. táblázat foglalja össze. Minden szolgáltatás FastAPI-alapú Python-alkalmazás, egységes `/health` végponttal és verziózott `/api/v1` felülettel, így a kiértékelő keretrendszer ugyanazokat a végpontokat hívja, mint az éles frontend.

4.2.1. Feladatkezelés és párhuzamosság

A generálás percekig tarthat (7. fejezet), ezért a Bifrost aszinkron feladatmodellt használ: a `POST /api/v1/generate` kérés azonnal egy feladatazonosítót ad vissza, az állapot pedig a



4.1. ábra. A MIMIR szolgáltatásai és adatfolyama. A szaggatott keretben lévő komponensek a feltöltött tartalmat kizárólag a feladat élettartama alatt, memóriában tartják; a nyelvi modell az egyetlen GPU-t használó összetevő.

4.2. táblázat. A MIMIR szolgáltatásai és kutatási szempontból releváns felelősségeik.

Szolgáltatás	Felelősség
Wellspring	Szövegkinyerés PDF-ből (PyMuPDF), DOCX-ből és szövegfájlból. Oldalanként olvashatósági arányt számol (értelmes karakterek aránya a nem szóköz karakterek között); 0,85 alatt az oldalt képként dolgozza fel: helyi módban Tesseract OCR-rel, egyébként látás-nyelvi modellel.
RuneCarver	Mondatfelbontás, ablakozott mondatbeágyazás (multilingual-E5-base), szemantikus darabolás méretfüggő küszöbvel (5.3. szakasz).
Bifrost	A generálás karmestere: aszinkron feladatkezelés lejáratási idővel, indexelés a vektortárba, visszakeresés, promptépítés, modellválasztás (helyi vagy szerver), audit metaadatok küldése.
Heimdall	A modellválaszok JSON-sémájának és koherenciájának szűrése; a szervermodell elérhetetlensége esetén helyi tartalék.
The Forge	PostgreSQL-alapú feladatsor zárolásmentes kiosztással; audit napló kizárólag metaadatokkal és SHA-256 ujjlenyomatokkal; megőrzési idő utáni automatikus törlés.
Skald	Exportálás JSON, Moodle XML (CDATA, MathJax) és nyomtatható PDF formátumba; az MI-közreműködés és a modell nevének feltüntetése.

GET /api/v1/status/{job_id} végponton kérdezhető le, amely a haladást és a becsült hátralévő időt is közli. A feladatok egy memóriabeli tárban élnek, amely minden hozzáféréskor törli a lejáratú időn (TTL = 3600 s) túli bejegyzéseket.

A CPU- és GPU-igényes háttér munkát a The Forge osztja ki a munkásoknak egy PostgreSQL-táblán alapuló sorral. A kiosztás a `SELECT ... FOR UPDATE SKIP LOCKED` záradékra [61] épül: minden munkás egy tranzakcióban kiválasztja a legrégebbi, még nem zárolt, várakozó feladatot, zárolja, állapotát *feldolgozás alatt*-ra állítja, majd a befejezés után *kész*-re. A `SKIP LOCKED` miatt a párhuzamos munkások nem várnak egymás zárjaira, és egy feladatot pontosan egy munkás kap meg, külön üzenetközvetítő (pl. RabbitMQ) nélkül. Mivel a GPU egyetlen modellt futtat, a generálási feladatok párhuzamossága szándékosan egy; a párhuzamos munkások a CPU-igényes kinyerést és darabolást gyorsítják.

4.3. Adatvédelem: zéró megőrzés

A G3 cél megvalósítása az adatminimalizálás és a tárolási korlátozás GDPR-elvein [13] alapul. A 4.3. táblázat a feldolgozás során keletkező összes adatfajta életciklusát összegzi.

4.3. táblázat. Adatfajták életciklusa a MIMIR rendszerben.

Adat	Tárolás helye	Élettartam
Feltöltött fájl bájttjai	szolgáltatás memóriája	a kinyerési kérés végéig
Kinyert szöveg, darabok	szolgáltatás memóriája	a feladat végéig
Beágyazások	Qdrant, memóriában	a következő feladat indexelése előtt a gyűjtemény törlődik
Promptok, kontextus	szolgáltatás memóriája	a modellhívás végéig
Generált vizsga (vázlat)	feladattár	TTL = 60 perc, majd automatikus törlés
Jóváhagyott, mentett vizsga	MinIO/SQLite	beállítható megőrzési idő, oktatói törlés
Audit napló	PostgreSQL	csak metaadatok és ujjlenyomatok; megőrzési idő után törlés

Az audit napló kulcsszerepet játszik a reprodukálhatóságban és az elszámoltathatóságban, ugyanakkor nem tartalmazhat tananyagot. Ezért a napló egy bejegyzése a

$$a = (\text{job_id}, \text{modell}, \text{promptverzió}, \text{minőségi pontszám}, h(u), h(p), h(x), |u|, |x|) \quad (4.1)$$

rekord, ahol u a felhasználói kérés, p a teljes prompt, x a visszakeresett kontextus, h a SHA-256 lenyomatfüggvény, $|\cdot|$ pedig a karakterhossz. A lenyomat alapján utólag igazolható, hogy két futás ugyanazzal a kontextussal és prompttal dolgozott-e, a tartalom azonban nem állítható vissza

belőle. A rendszer induláskor a korábbi, még tartalmat is tároló naplósorok szöveges mezőit kiüríti, és óránként törli a megőrzési időn túli sorokat.

A `LOCAL_ONLY` kapcsoló minden külső modellhívást letilt: ilyenkor a látás–nyelvi modell helyett Tesseract OCR, a szervermodell helyett a helyi Ollama fut, a modellválasztó végpont pedig csak helyi modellt kínál. Kapcsoló nélkül a feltöltési képernyő megnevezi, hol történik a feldolgozás, és az oktató választhatja a helyi modellt.

4.4. Erőforrás-modell: a GPU-memória költségvetése

A G4 cél kvantitatív kérdés: befér-e a generátor egy 8 GB-os kártyára úgy, hogy mellette az asztali környezet is fut? A dekódolás közbeni memóriaigényt három tag határozza meg:

$$M(\text{ctx}) = W + \text{ctx} \cdot \underbrace{2LH_{kv}d_h\beta}_{m_{kv}} + M_0, \quad (4.2)$$

ahol W a kvantált súlyok mérete, L a rétegek, H_{kv} a kulcs–érték fejek száma, d_h a fejdimenzió, β a gyorsítótár elemeinek mérete bájtban (fp16 esetén $\beta = 2$), a 2-es szorzó a kulcs és az érték tárolását jelöli, M_0 pedig a futtatókörnyezet és a számítási pufferek többletköltsége. A csoportosított lekérdezésű figyelem [44] lényege éppen az, hogy H_{kv} a lekérdezőfejek számánál jóval kisebb. A Qwen2.5-7B [62] architektúrájára ($L = 28$, $H_{kv} = 4$, $d_h = 128$) a tokenenkénti gyorsítótár

$$m_{kv} = 2 \cdot 28 \cdot 4 \cdot 128 \cdot 2 \text{ B} = 57\,344 \text{ B} = 56 \text{ KiB}, \quad (4.3)$$

így 16384 tokenes kontextusnál a KV-gyorsítótár $56 \cdot 16384 \text{ KiB} = 896 \text{ MiB}$. A 4.4. táblázat a modell alapján becsült memóriaigényt mutatja több lehetséges helyi generátorra.

4.4. táblázat. Becsült GPU-memóriaigény 8192 tokenes kontextusnál, fp16 KV-gyorsítótárral és $M_0 \approx 0,6 \text{ GB}$ futtatókörnyezeti többlettel. A súlyok a GGUF-fájlok mérete, a gyorsítótár a publikált architektúrákból számolt érték.

Modell	Kvantálás	L/H_{kv}	Súlyok	m_{kv}	Összesen	Kártya
Qwen2.5-3B	Q4_K_M	36/2	1,9 GB	36 KiB	2,8 GB	8 GB
Qwen2.5-7B	Q4_K_M	28/4	4,7 GB	56 KiB	5,8 GB	8 GB
Llama-3.1-8B [63]	Q4_K_M	32/8	4,9 GB	128 KiB	6,6 GB	8 GB
Qwen2.5-7B	Q8_0	28/4	8,1 GB	56 KiB	9,2 GB	12 GB
Qwen2.5-14B	Q4_K_M	48/8	9,0 GB	192 KiB	11,2 GB	12 GB

A modell szerint a 3–8 milliárd paraméteres modellek 4 bites kvantálással elférnek egy 8 GB-os kártyán, a 14B modell vagy a 8 bites 7B modell viszont már 12 GB-ot igényel. A becslést a pilot vizsgálatban mért értékekkel validáltuk (4.5. táblázat): minden kar első vizsgájánál, amikor a modell még nem volt betöltve, az alapszint 1612–1931 MiB volt (asztali környezet), a generálás alatti csúcs pedig 7022–7808 MiB. A modell és a futtatókörnyezet tehát 5400–5950 MiB-ot foglalt 16384 tokenes kontextus mellett, ami a (4.2) modell alapján várt $4,7 \text{ GB} + 0,9 \text{ GiB} +$

+ M_0 értékkel összhangban van, és a legrosszabb esetben is ≈ 380 MiB tartalékot hagyott a 8 192 MiB-os kártyán.

4.5. táblázat. Mért GPU-memória a pilot vizsgálatban (MiB), karonként az első, hidegindítású vizsga alapján. A különbség a modell és a futtatókörnyezet foglalása.

Kar	Alapszint	Csúcs	Különbség
E1	1 612	7 025	5 413
E2	1 815	7 761	5 946
E2f	1 931	7 621	5 690
E2h	1 774	7 175	5 401
E3	1 739	7 134	5 395
E5a	1 896	7 369	5 473
E5b	1 899	7 326	5 427
E5c	1 892	7 292	5 400

A mérés egy fontos mellékhatásra is rámutat: a teljes dokumentumot egy promptban kapó alapvonal (B-doc-L) számára a kontextus növelése nem ingyenes. 32 768 tokenes kontextusnál a KV-gyorsítótár önmagában 1,75 GiB lenne, ami a mért csúcshoz adva már meghaladná a kártya kapacitását; ekkor a futtatókörnyezet a modell egy részét a CPU-ra helyezi, ami nagyságrendi lassulással jár. Ezért a helyi teljes dokumentumos alapvonal 16 384 tokenes kontextussal és 30 000 karakterre csonkolt dokumentummal fut, és a levágott rész arányát rögzítjük – éppen ez az a korlát, amelyet egy kis helyi modell nem tud megkerülni, a tervezett csővezeték viszont igen, mert minden hívása rövid.

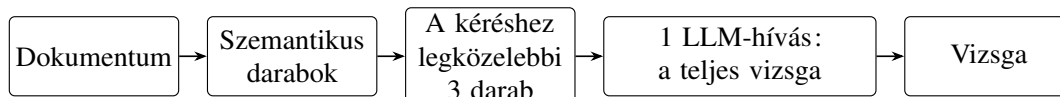
5. A MIMIR generálási módszere

Ez a fejezet a dolgozat módszertani magja. Először áttekintjük a két összehasonlított csővezetékét – a naiv RAG-alapvonalat és a tervezett (blueprint) csővezetékét –, majd lépésenként ismertetjük a darabolás, a tervezés, a visszakeresés, a generálás, az ellenőrzés és a fogalomgráf algoritmusait. A leírás a kiértékelő keretrendszer implementációját követi (`mimir_eval/blueprint`), amelynek paraméterei az 5.1. táblázatban szerepelnek.

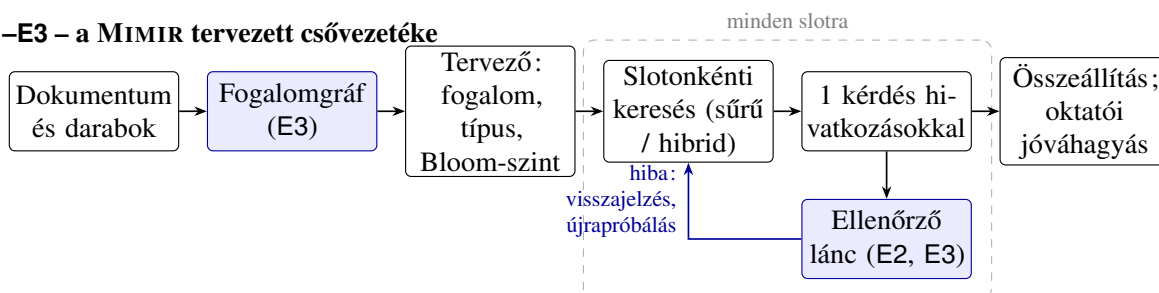
5.1. Áttekintés: naiv RAG és tervezett csővezeték

Az 5.1. ábra a két csővezetékét veti össze. A *naiv RAG* (E0) a kéreßszöveghez legjobban hasonlító három darabot keresi vissza, és egyetlen hívásban kéri a teljes vizsgát. Ez a rendszer eredeti (v1) működése, és egyben a legolcsóbb ésszerű alapvonal. A *tervezett csővezeték* (E1–E3) ezzel szemben először explicit vizsgatervet készít, majd a terv minden slotjára külön visszakeresést, egyetlen kérdés generálását és – az ellenőrző karokban – ellenőrzést végez.

E0 – naiv RAG (alapvonal)



E1–E3 – a MIMIR tervezett csővezetéke



5.1. ábra. A naiv RAG és a MIMIR tervezett csővezetéke. Kék: a kísérleti karokban ki- és bekapcsolt komponensek. A terv a vizsga szerkezetét még az első kérdés megírása előtt rögzíti, és minden kérdést a saját, hivatkozott kontextusával szemben ellenőriz.

A két megközelítés közötti lényegi különbség nem a modell, hanem a feladat felbontása. A naiv alapvonalban a modellnek egyetlen hívásban kell eldöntenie, hány kérdést, milyen témákról, milyen nehézségben és milyen disztraktorokkal ír; a tervezett csővezetékben ezek közül a szerkezeti döntéseket (darabszám, típus, Bloom-szint, téma) determinisztikus kód hozza meg, és a modell feladata egy-egy rövid, jól körülhatárolt kérdés megírására szűkül, amelyhez csak a releváns kontextust kapja meg.

5.1. táblázat. A tervezett csővezeték paraméterei és alapértékei.

Paraméter	Alapérték	Jelentés
retrieval_k	3	visszakeresett darabok száma slotonként
retrieval	dense	dense: Bifrost sűrű keresés; hybrid: BM25 + sűrű, RRF
spare_concepts	3	tartalékfogalmak száma ($n + 3$ tervezett fogalom)
max_retries (r)	1 (pilot: 2)	újrapróbálkozások slotonként visszajelzéssel
verifier	ki/be	ellenőrző lánc (E1 : ki; E2, E3: be)
blind_test	be	vak megoldási próba (E2f: ki)
leakage_checks	be	szivárgás- és kérdésforma-szabályok (a pilot után bevezetve)
repair	be	javító hívás hibás formájú végső kérdésre (a pilot után bevezetve)
dedupe_threshold	0,9	duplikátumküszöb (koszinusz-hasonlóság)
overview_chars	12 000	a tervezőnek átadott áttekintés karakterkerete
max_context_chars	6 000	slotonkénti kontextus karakterkerete
graph_batch_chars	3 000	fogalomgráf-kötegek mérete (E3)
generátor	Qwen2.5-7B	Q4_K_M, $T = 0,2, 8\,192\text{--}16\,384$ tokenes kontextus

5.2. Szövegkinyerés

A Wellspring szolgáltatás a PDF-oldalakat először szövegrétegük alapján olvassa ki. Minden oldalra kiszámítja az olvashatósági arányt,

$$\rho = \frac{|\{x \in \text{oldal} : x \in \Sigma_{\text{ért}}\}|}{|\{x \in \text{oldal} : x \text{ nem szóköz}\}|}, \quad (5.1)$$

ahol $\Sigma_{\text{ért}}$ a betűk, számjegyek és szokásos írásjelek halmaza. Ha $\rho < 0,85$ (hibás kódolású vagy szkennelt oldal), az oldalt képként dolgozza fel: helyi módban Tesseract OCR-rel, egyébként egy látás–nyelvi modellel, amely a táblázatokat és listákat is megőrzi. A kiértékelésben a dokumentumok szövegesek, így ez az ág nem befolyásolja a mérést.

5.3. Szemantikus darabolás

5.3.1. Mondatbeágyazás ablakozással

A darabolás első lépése a mondatfelbontás, amely a mondatvégi írásjelekre és sortörésekre illeszkedő reguláris kifejezéssel, a karakterpozíciók megőrzésével bontja a szöveget. A mondatbeágyazás a multilingual-E5-base [29] kódolóval, *kontextuális* módon történik: a modell nem az egyes mondatokat, hanem hosszabb szöveglablákat kódol, és egy mondat beágyazása a mondatot alkotó tokenek utolsó rejtett állapotainak átlaga (*mean pooling*). Formálisan, ha a w ablakban a tokenek rejtett állapotai $h_1, \dots, h_{|w|}$, és $\mathcal{T}(s_i)$ az s_i mondat karakterintervallumával

átfedő tokenek indexhalmaza, akkor

$$\phi(s_i) = \frac{1}{|\mathcal{T}(s_i)|} \sum_{t \in \mathcal{T}(s_i)} h_t. \quad (5.2)$$

Előnye, hogy egy mondat beágyazása a szomszédos mondatok kontextusát is tükrözi, így egy rövid, önmagában kétértelmű mondat is a témájához közel kerül.

A kódoló legfeljebb 512 tokenes bemenetet fogad. Az eredeti implementáció a teljes dokumentumot *egyetlen* menetben kódolta, csonkolással; minden mondat, amely a 512. token után kezdődött, üres tokenhalmazt, így nullvektort kapott. Az új, ablakozott kódoló (5.1. algoritmus) a mondatokat mondathatáron vágott, legfeljebb 510 tokenes ablakokba csoportosítja, és ablakonként futtatja a modellt. Rövid, 512 tokennél rövidebb szövegen a két kódoló azonos eredményt ad – ezt a pilotban ki is használtuk a futásról futásra jelentkező zaj becslésére (7.9. szakasz).

Algoritmus 5.1: Szemantikus darabolás ablakozott mondatbeágyazással

Bemenet: szöveg $\mathcal{D}$; módszer $\in \{\text{percentilis, szórás}\}$; küszöbparaméter θ ; célméret ℓ^* (karakter)

Kimenet: darabok $\mathcal{C} = (c_1, \dots, c_K)$

- 1 $S = (s_1, \dots, s_m) \leftarrow \text{MONDATOK}(\mathcal{D});$
- 2 **ha** $m \leq 1$ **akkor visszatér** ($\mathcal{D}$);
// 1. Ablakozás: mondathatáron, legfeljebb 510 token
- 3 $\mathcal{W} \leftarrow \emptyset; w \leftarrow (); \beta \leftarrow 0;$
- 4 **ciklus** $i = 1, \dots, m$:
 - 5 $\ell_i \leftarrow |\text{TOKENÉK}(s_i)|;$
 - 6 **ha** $w \neq ()$ **és** $\beta + \ell_i > 510$ **akkor** $\mathcal{W} \leftarrow \mathcal{W} \cup \{w\}; w \leftarrow (); \beta \leftarrow 0;$
 - 7 $w \leftarrow w \parallel (i); \beta \leftarrow \beta + \ell_i;$
- 8 $\mathcal{W} \leftarrow \mathcal{W} \cup \{w\};$
// 2. Kontextuális mondatbeágyazás
- 9 **minden** $w \in \mathcal{W}$ **elemre**
 - 10 $H \leftarrow \text{KÓDOLÓ}(\mathcal{D}[a_{w_1} : b_{w_{|w|}}]);$ // egy menet, ≤ 512 token
 - 11 **minden** $i \in w$ **elemre**
 - 12 $\phi(s_i) \leftarrow \text{ÁTLAG}\{H_t : t \in \mathcal{T}(s_i)\}$ (üres esetén $\mathbf{0}$)
- 13 // 3. Küszöb és vágás méretfüggő érzékenységgel
 $\delta_i \leftarrow 1 - \cos(\phi(s_i), \phi(s_{i+1}))$ minden $i < m$ esetén;
 $\tau \leftarrow P_\theta(\delta)$ ha percentilis, különben $\mu_\delta + \theta \sigma_\delta$;
 $c \leftarrow (s_1); \ell \leftarrow |s_1|; \mathcal{C} \leftarrow ();$
- 16 **ciklus** $i = 1, \dots, m - 1$:
 - 17 $\tau_{\text{eff}} \leftarrow \tau / \max(1, \ell / \ell^*);$
 - 18 **ha** $\delta_i > \tau_{\text{eff}}$ **akkor**
 - 19 $\mathcal{C} \leftarrow \mathcal{C} \parallel (c); c \leftarrow (s_{i+1}); \ell \leftarrow |s_{i+1}|$
 - 20 **különben**
 - 21 $c \leftarrow c \parallel (s_{i+1}); \ell \leftarrow \ell + |s_{i+1}|$
- 22 **visszatér** $\mathcal{C} \parallel (c);$

5.3.2. A vágási szabály

A vágás két összetevőből áll. Az *alapküszöb* τ vagy a távolságok θ -adik percentilise (alapérték: $\theta = 85$, vagyis a legnagyobb 15 % szomszédos távolságnál vágunk), vagy a $\mu_\delta + \theta\sigma_\delta$ érték (az E5b karban $\theta = 1,5$). A *méretfüggő érzékenység* a célméret (ℓ^*) túllépésekor arányosan csökkenti a küszöböt:

$$\tau_{\text{eff}}(\ell) = \frac{\tau}{\max(1, \ell/\ell^*)}. \quad (5.3)$$

Egy célméretnél rövidebb darab tehát csak valódi témaváltásnál zárul, a célméretet x -szeresen meghaladó darab viszont már τ/x távolságnál is. Ez a „puha korlát” megakadályozza a túlságosan hosszú darabokat anélkül, hogy a mondatok közepén vágna.

5.3.3. A csonkolt kódoló hibájának elemzése

A régi kódoló hibája a vágási szabályon keresztül nem nyilvánvaló módon torzít. A 512. token utáni mondatok nullvektort kapnak; a nullvektor normalizálás után is nulla, így bármely mondat-tal vett koszinusz-hasonlósága 0, a távolság tehát $\delta_i = 1$ minden érintett szomszédos párra. Ha a dokumentum nagyobbik része a határ után van, a távolságok több mint 15 %-a 1, így a percentilis küszöb $\tau = 1$ lesz, és a $\delta_i > \tau$ feltétel a célméret eléréséig soha nem teljesül. A célméret túllépése után viszont $\tau_{\text{eff}} < 1 = \delta_i$, így minden további mondatnál vágás történik. A dokumentum farka ezzel a tartalomtól független, *méretalapú* darabolássá degenerál, a darabok beágyazása pedig (a mondatbeágyazásoktól függetlenül számolva) már nem tükrözi a vágás logikáját. 4,0–4,5 karakter/token aránnyal becsülve a kiértékelő adatkészlet átlagos dokumentumának 68–71 %-a esett a határ után, így a hiba a hosszabb dokumentumokon végzett minden korábbi mérést érintett. Az E5a kar a régi kódolót szándékosan megtartja, hogy a hiba hatása a fő vizsgálatban számszerűsíthető legyen.

5.4. Indexelés és sűrű visszakeresés

A darabokat a Bifrost az E5 kódolóval beágyazza, és egy memóriában futó Qdrant [64] gyűjteménybe tölti koszinusz-metrikával. A gyűjtemény minden új dokumentum indexelése előtt törlődik, így a vektortár mindig csak az aktuális feladat tartalmát tartalmazza (4.3. szakasz). A sűrű keresés a q lekérdezésre a

$$\text{top}_k(q) = \arg \max_{I \subseteq \{1..K\}, |I|=k} \sum_{i \in I} \cos(\phi(q), \phi(c_i)) \quad (5.4)$$

indexhalmazt adja vissza, azaz a k leghasonlóbb darabot.

5.5. Az alapvonalak

Naiv RAG (E0). A lekérdezés a felhasználói kérdés szövege (pl. „10 feleletválasztós kérdés, könnyű nehézség”), a kontextus a top_3 darab. A prompt (A. függelék) a rendszer éles, v1.0-s promptja: szigorú szabályokat ír elő (zéró hallucináció, a kért darabszám és típus betartása, egy helyes és három hihető disztraktor, meta-hivatkozás tilalma) és a kimeneti JSON-sémát. A generálás egyetlen hívás.

Teljes dokumentum (B-doc-L, B-doc-S). A status quo modellje: ugyanaz a prompt, de a kontextus a teljes kinyert dokumentum. A helyi változat 16 384 tokenes kontextussal és 30 000 karakterre csonkolt dokumentummal, a szerverváltozat 80 000 karakterig fut; a csonkolt arányt (*truncated_share*) vizsgánként rögzítjük.

5.6. A vizsgaterv

5.6.1. Áttekintés és fogalomtervezés

A tervező egyetlen hívásban kapja meg a dokumentum *áttekintését*: minden darab azonosítóját ($C_0, C_1, \dots$) és a darab első $\max(120, \lfloor 12\,000/K \rfloor)$ karakterét. Az áttekintés így a dokumentum hosszától függetlenül korlátos méretű, és minden darab megjelenik benne – ez a „lost in the middle” jelenség [11] ellen hat, mert a modell nem egy hosszú, folytonos szöveget, hanem egy azonosítókkal tagolt, rövid felsorolást lát. A tervező $n + 3$ fogalmat ad vissza, mindegyikhez egy rövid leírást (mit kell kérdezni), a fogalmat alátámasztó darabok azonosítóit és egy fontossági értéket (1: részlet, 2: fontos, 3: központi). Ha a tervező válasza értelmezhetetlen, a csővezeték darabonként egy álfogalomra esik vissza (a darab első nyolc szava), így a terv mindig létrejön.

5.6.2. Bloom-szintek kiosztása

A nehézséghez rendelt Bloom-eloszlásokat az 5.2. táblázat mutatja. A diszkretizálás a 3.1. állítás szerinti legnagyobb maradék módszerrel történik; a kapott szinteket a véletlenmaggal megkeverjük (így a szintek nem a fogalmak fontossági sorrendjéhez kötődnek), a kész vizsgát pedig végül a Bloom-szintek szerint rendezzük, a könnyebbtől a nehezebb felé.

5.2. táblázat. A nehézségi szintekhez rendelt Bloom-eloszlások és a $n = 10$ kérdéses vizsga kiosztása.

Nehézség	Emlékezés	Megértés	Alkalmazás	Elemzés	Értékelés	Kiosztás ($n = 10$)
Könnyű	0,5	0,5	–	–	–	(5, 5, 0, 0, 0)
Közepes	–	0,4	0,4	0,2	–	(0, 4, 4, 2, 0)
Nehéz	–	–	0,3	0,4	0,3	(0, 0, 3, 4, 3)

5.6.3. Mohó, lefedettségvezérelt slotkiosztás

A slotkiosztás (5.2. algoritmus) a fogalmakat fontosság szerint rendezi, majd mohón választ: minden lépésben azt a fogalmat, amely (i) érint még le nem fedett darabot, és ezen belül (ii) a legfontosabb. A 3.2. állítás szerint ez legalább $\min(n, |U|)$ különböző darab lefedését garantálja. Ha kevesebb fogalom van, mint slot, a fogalmakat körbeforgatva újra felhasználjuk (eltérő Bloom-szinttel), így a slotok száma mindig pontosan n . A ki nem választott fogalmak alkotják a tartaléklistát.

Algoritmus 5.2: Vizsgaterv: Bloom-kiosztás és mohó, lefedettségvezérelt slotkiosztás

```

Bemenet: fogalmak  $\mathcal{K}$  (mindegyik  $D(\kappa)$  darabalmazzal és  $\iota(\kappa)$  fontossággal);  $n$ ; típusok  $T$ ;
           nehézség  $\lambda$ ; mag  $\omega$ 
Kimenet: slotok  $\mathcal{B} = ((\kappa_j, T_j, b_j))_{j=1}^n$ ; tartalékfogalmak  $\mathcal{S}$ 
// Bloom-szintek: legnagyobb maradék módszer
1  $p \leftarrow \text{BLOOMELOSZLÁS}(\lambda); c_b \leftarrow \lfloor np_b \rfloor$  minden  $b$ -re;
2 minden  $b$  a  $np_b - c_b$  törtrész szerint csökkenően, az első  $n - \sum_b c_b$  szintre elemre
3    $c_b \leftarrow c_b + 1$ 
4  $\mathbf{b} \leftarrow \text{KEVER}_\omega(\text{minden } b \text{ szint } c_b\text{-szer});$ 
   // Mohó lefedés
5  $\mathcal{P} \leftarrow \text{RENDEZ}(\mathcal{K}, -\iota); \mathcal{A} \leftarrow (); H \leftarrow \emptyset;$ 
6 amíg  $\mathcal{P} \neq \emptyset$  és  $|\mathcal{A}| < n$  :
7    $\kappa^+ \leftarrow \arg \max_{\kappa \in \mathcal{P}} (|\mathcal{K}[D(\kappa) \setminus H] \neq \emptyset|, \iota(\kappa));$  // lexikografikus
8    $\mathcal{P} \leftarrow \mathcal{P} \setminus \{\kappa^+\}; \mathcal{A} \leftarrow \mathcal{A} \parallel (\kappa^+); H \leftarrow H \cup D(\kappa^+);$ 
9  $i \leftarrow 0;$ 
10 amíg  $|\mathcal{A}| < n$  : // kevés fogalom: újrahasznosítás
11    $\mathcal{A} \leftarrow \mathcal{A} \parallel (\kappa_{(i \bmod |\mathcal{K}|)}); i \leftarrow i + 1;$ 
12 visszatér  $\mathcal{B} = ((\mathcal{A}_j, T_{(j \bmod |T|)}, \mathbf{b}_j))_{i=1}^n; \mathcal{S} = \mathcal{P};$ 

```

5.7. Slotonkénti visszakeresés

Minden slot saját kontextust kap. A lekérdezés a fogalom neve és leírása („*fogalom: leírás*”), a kontextus pedig a fogalomhoz a tervező által rendelt első két darab, kiegészítve a keresés top_k eredményével (ismétlődés nélkül). A kontextus karakterkerete 6 000 karakter, hogy a kis modell „kényelmi zónájában” maradjon: a darabokat sorrendben vesszük fel, amíg a keret engedi. Ha az ellenőrző azt jelzi, hogy a hivatkozott részlet nem igazolja a kulcsot, a következő kísérletben k kettővel nő, vagyis a csővezeték szélesebb hálójával tér vissza a visszakereséshez.

5.7.1. Hibrid visszakeresés: BM25 és reciprok rangfúzió

A hibrid kar (E2h) a sűrű rangsort egy lexikális BM25-rangsorral [30] egyesíti. A q lekérdezés és a c darab BM25-pontszáma

$$\text{BM25}(q, c) = \sum_{w \in q} \text{idf}(w) \cdot \frac{f(w, c) (k_1 + 1)}{f(w, c) + k_1 \left(1 - b + b \frac{|c|}{\overline{|c|}}\right)}, \quad \text{idf}(w) = \ln \left(1 + \frac{K - n_w + 0,5}{n_w + 0,5}\right), \quad (5.5)$$

ahol $f(w, c)$ a w szó gyakorisága a darabban, $|c|$ a darab tokenhossza, $\overline{|c|}$ az átlagos darabhossz, n_w a w -t tartalmazó darabok száma, $k_1 = 1,5$ és $b = 0,75$. A magyar nyelv agglutináló morfológiája miatt a tokenizálás egyszerű, nyelvfüggetlen *prefixtövesítést* alkalmaz: a hat karakternél hosszabb szavakat hatkarakteres előtagjukra vágja, így az „immunrendszer”, „immunrendszert”, „immunrendszerben” alakok közös kulcsot kapnak. A két rangsort a reciprok rangfúzió [31] egyesíti:

$$\text{RRF}(c) = \sum_{\rho \in \{\rho_{\text{sűrű}}, \rho_{\text{BM25}}\}} \frac{1}{k_{\text{rrf}} + \rho(c)}, \quad k_{\text{rrf}} = 60, \quad (5.6)$$

ahol $\rho(c)$ a darab (1-től számozott) rangja az adott rangsorban; a BM25-rangsorból a nulla pontszámú darabok kimaradnak.

5.8. Forráshoz kötött kérdésgenerálás

A generátor egyetlen kérdést ír a slot terve szerint. A prompt (A. függelék) tartalmazza a fogalmat, a kérdezendő tartalmat, a típusszabályt, a Bloom-szint magyar leírását, a nehézséget és a nyelvet; megismétli az éles prompt szabályait (zéró hallucináció, LaTeX és meta-hivatkozás tilalma); felsorolja a már elfogadott kérdéseket (legfeljebb az utolsó 12-t), hogy a modell ne ismételje őket; és megköveteli a `citations` mezőt, amelyben a modell megnevezi a kulcsot igazoló darabok azonosítóit. Ha az előző kísérlet hibás volt, a prompt a hibák listáját is tartalmazza („Az előző próbálkozás hibás volt, javítsd:”). A kérdések egyenkénti generálása kétféle előnnyel jár: a modellnek egyszerre csak egy döntést kell hoznia, és minden kérdés saját, ellenőrizhető forrásrészlethez kötődik (G1).

5.9. Az ellenőrző lánc

5.9.1. Ellenőrzések a költség szerinti sorrendben

Az ellenőrzések a legolcsóbbtól a legdrágábbig futnak (5.3. táblázat); egy olcsó ellenőrzés hibája esetén a drága, nyelvi modellt igénylő ellenőrzések el sem indulnak. A szabályalapú ellenőrzések determinisztikusak, a modellalapúak a hivatkozott darabokat (ennek hiányában a teljes slotkontextust) kapják forrásként.

Az alátámasztottsági ellenőrzés egyetlen hívásban három ítéletet kér: igazolja-e a forrás a

5.3. táblázat. Az ellenőrző lánc lépései végrehajtási sorrendben, a hiba esetén adott visszajelzéssel.

Ellenőrzés	LLM-hívás	Visszajelzés hiba esetén
Forma (Shape)	–	feleletválasztós: 4 válasz, 1 helyes
Meta-hivatkozás	–	fogalmazd meg „a szöveg szerint” jellegű hivatkozás nélkül
Szivárgás (L, N) (új)	–	a törzs nem tartalmazhatja a kulcsot, és kérdés legyen
Hivatkozás	–	hivatkozz a kontextus legalább egy darabjára
Alátámasztottság és disztraktorok	1	a kulcsot a részlet nem igazolja / egy disztraktor igaz lehet / kétértelmű
Vak megoldás	1	egy független megoldó más választ jelölt meg
Duplikátum	–	túl hasonló egy már elfogadott kérdéshez

kulcsot (`key_supported`), hamis-e minden disztraktor (`options_wrong`), és kétértelmű-e a kérdés (`ambiguous`). A vak megoldás külön hívás, amelyben a modell a kulcs ismerete nélkül, véletlen permutációban látja az opciókat. Ha az ellenőrző modell nem elérhető, a kérdést nem blokkoljuk, de a kihagyott ellenőrzések számát rögzítjük. Az E2x kar az ellenőrző hívásokat egy másik modelleszaládba tartozó modellel (Llama-3.1-8B [63]) végzi, hogy elválassa az önellenőrzés és az ellenőrzés általános hatását.

5.9.2. Slotkitöltés visszacsatolással és tartalékokkal

Az 5.3. algoritmus egy slot kitöltését írja le. Egy slot legfeljebb $r + 1$ kísérletet kap; minden kísérlet hibalistája a következő kísérlet promptjába kerül. Ha egyik kísérlet sem megy át, a slot egy tartalékfogalommal újraindul. Ha ez sem sikerül, a *legjobb* kísérlet marad meg, *nem ellenőrzöttként* megjelölve, hogy az oktató a szerkesztőben lássa. A „legjobb” a

$$\text{rang}(q) = (1 - \text{Shape}(q), L(q), |\text{hibák}(q)|) \quad (5.7)$$

lexikografikus kulcs szerint legkisebb kísérlet: a formailag helyes kérdés mindig előnyt élvez, ezen belül a szivárgásmentes, végül a kevesebb hibát tartalmazó. Ha a megmaradt feleletválasztós kérdés formája mégis hibás, egyetlen *javító* hívás pótolja a hiányzó disztraktorokat a tartalom és a kulcs megtartásával. Az (5.7) kulcs és a javító hívás a pilot egyik hibájára adott válasz: ott egy-egy vizsgában hibás formájú végső kérdés maradt meg, ami a vizsga formai megfelelését nullára állította (7.4.1. szakasz).

5.10. Munkamenet-hatókörű fogalomgráf

Az E3 kar a tervezőt egy fogalomgráffal helyettesíti. A gráfépítés (5.4. algoritmus) a darabokat legfeljebb 3 000 karakteres kötegekbe csoportosítja, és kötegenként egyetlen hívásban kinyeri a fogalmakat (definícióval, szülőfogalommal és darabazonosítókkal), az atomi tényeket és a tipizált relációkat (*is_a*, *part_of*, *causes*, *contrasts_with*, *used_for*, *related_to*). A csúcsokat normalizált név alapján, majd a névbeágyazások 0,92-es koszinusz-küszöbe szerint egyesítjük, ami a magyar

Algoritmus 5.3: Egy slot kitöltése ellenőrzéssel, visszacsatolással és tartalékfogalommal**Bemenet:** slot (κ, T, b) ; elfogadott kérdések $\mathcal{Q}$; tartalékok $\mathcal{S}$; r ; ellenőrzés be/ki**Kimenet:** kérdés q és ellenőrzött jelző

```

1 Függvény Kitölt( $\kappa$ ) :
2   visszajelzés  $\leftarrow \emptyset$ ;  $k' \leftarrow 0$ ;  $q^{\text{best}} \leftarrow \perp$ ;
3   ciklus  $a = 0, \dots, r$  :
4      $X \leftarrow \text{SLOTKONTEXTUS}(\kappa, k + k')$ ; //  $\leq 6000$  karakter
5      $q \leftarrow \text{GENERÁL}(\kappa, T, b, X, \mathcal{Q}, \text{visszajelzés})$ ;
6     ha  $q = \perp$  akkor visszajelzés  $\leftarrow$  „érvénytelen JSON”; folytat;
7      $E \leftarrow \text{ELLENŐRIZ}(q, X)$ ; // 5.3. táblázat, olcsótól drágáig
8     ha  $\max_{q' \in \mathcal{Q}} \cos(\phi(q), \phi(q')) \geq 0,9$  akkor  $E \leftarrow E \cup \{\text{duplikátum}\}$ ;
9     ha  $E = \emptyset$  akkor visszatér ( $q, \text{IGAZ}$ );
10    ha  $\text{rang}(q) < \text{rang}(q^{\text{best}})$  akkor  $q^{\text{best}} \leftarrow q$ ;
11    ha nincs ellenőrzés és Shape( $q$ ) és nem duplikátum akkor visszatér ( $q, \text{IGAZ}$ );
12    visszajelzés  $\leftarrow E$ ;
13    ha „nem igazolja”  $\in E$  akkor  $k' \leftarrow k' + 2$  // szélesebb visszakeresés;
14  visszatér ( $q^{\text{best}}, \text{HAMIS}$ );
15 ( $q, v$ )  $\leftarrow$  Kitölt( $\kappa$ );
16 ha  $\neg v$  és  $\mathcal{S} \neq \emptyset$  akkor
17   ( $q', v'$ )  $\leftarrow$  Kitölt( $\text{POP}(\mathcal{S})$ );
18   ha  $v'$  vagy  $q = \perp$  vagy  $(\text{Shape}(q') \wedge \neg \text{Shape}(q))$  akkor ( $q, v$ )  $\leftarrow$  ( $q', v'$ );
19 ha  $T = \text{mcq}$  és  $\neg \text{Shape}(q)$  akkor
20    $q \leftarrow \text{JAVÍT}(q)$ ; // egy hívás: hiányzó disztraktorok
21 visszatér ( $q, v$ );

```

ragozott alakokat (pl. „T-sejt”, „T-sejtek”) összevonja. A szülőfogalmak is csúcsok lesznek *is_a* éllel, így a közös szülő alá tartozó fogalmak egy lépésben elérhetők egymásból.

Egy v fogalom *centralitása* a foksám, a darabszám és a tényszám súlyozott összege:

$$\text{cent}(v) = \deg(v) + \frac{1}{2}|D(v)| + \frac{1}{2}|\text{Facts}(v)|, \quad (5.8)$$

amelyből a tervezési fontosság $\iota(v) = 3$, ha $\text{cent}(v) \geq 4$; $\iota(v) = 2$, ha $\text{cent}(v) \geq 2$; egyébként 1. A gráf három helyen gazdagítja a csővezetékét: (i) a tervező bemenete a centralitás szerint rendezett fogalomlista; (ii) a disztraktor-ötletek a *testvérfogalmak*, vagyis azok a w csúcsok, amelyekre $\text{parent}(w) = \text{parent}(v)$ vagy $N(w) \cap N(v) \neq \emptyset$, de $w \notin N(v)$ és w nem v szülője – ezek ugyanabba a kategóriába tartoznak, ezért hihetők, de nem azonosak a kulccsal; (iii) a kontextus kiegészül a szomszédos fogalmak legtöbbször előforduló, saját darabhalmazon kívüli két darabjával (1-ugrásos szomszédság).

A gráf csak a vizsga élettartamáig létezik a memóriában, összhangban a zéró megőrzéssel. Költsége $\lceil |\mathcal{Q}|/B \rceil$ nagyságrendű további hívás; egy 1 700 karakteres dokumentumon ez egyetlen hívás volt, amely a pilot három futásában 14, 18 és 31 fogalmat, illetve 3, 9 és 19 relációt adott – a szórás maga is jelzi, hogy a kis modell gráfkinyerése instabil.

Algoritmus 5.4: Munkamenet-hatókörű fogalomgráf építése

Bemenet: darabok $\mathcal{C}$; kötegméret $B = 3000$; hasonlósági küszöb $\eta = 0,92$
Kimenet: gráf $\mathcal{G} = (V, E)$ fogalmakkal, tényekkel és darabhalmazokkal

```

1  $V \leftarrow \emptyset; E \leftarrow \emptyset;$ 
2 minden  $\mathcal{J} \in \text{KÖTEGEK}(\mathcal{C}, B)$  elemre
3    $(Fog, Tény, Rel) \leftarrow \text{LLM}_{\text{gráf}}(\{[C_i]c_i : i \in \mathcal{J}\})$ ; // 1 hívás
4   minden  $(név, def, szülő, D) \in Fog$  elemre
5      $\text{HOZZÁAD}(V, \text{NORM}(név), def, szülő, D)$ ; // egyesít, ha létezik
6   minden  $(szöveg, fogalmak, i) \in Tény$  elemre
7      $\text{minden érintett fogalomhoz csatolja a tényt és az } i \text{ darabot}$ 
8   minden  $(u, típus, w) \in Rel$  elemre
9      $E \leftarrow E \cup \{\{u, w\}\}$ 
10  minden  $v \in V$  szülővel elemre
11     $E \leftarrow E \cup \{\{v, \text{parent}(v)\}\}$ ; // is_a él
12 minden  $u, w \in V, u \prec w$  elemre
13   ha  $\cos(\phi(v_u), \phi(v_w)) \geq \eta$  akkor  $\text{ÖSSZEVON}(u, w)$ ; //  $v_u$ :  $u$  neve; darabok,
   tények, élek uniója
14 ;
15 visszatér  $\mathcal{G}$ ;
```

5.11. Összeállítás és emberi felügyelet

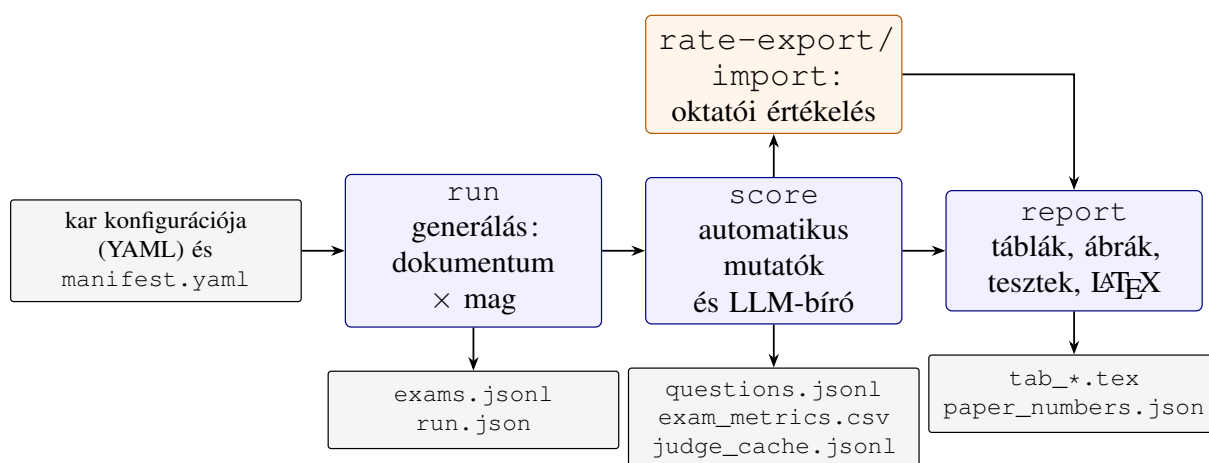
Az elfogadott kérdéseket a Bloom-szint szerint rendezzük, és minden kérdéshez rögzítjük a hivatkozott darabokat, a kontextus darabjait, a tervezett fogalmat és az *ellenőrzött* jelzöt. A vázlat egy szerkesztőbe kerül, ahol az oktató szerkesztheti, átrendezheti és törölheti a kérdéseket, módosíthatja a kulcsot, egyetlen kérdést újragenerálhat, és minden kérdés mellett megtekintheti a hivatkozott forrásrészletet. Jóváhagyás nélkül semmi sem exportálható, az exportált dokumentum pedig feltünteti, hogy a tartalom MI-rendszer közreműködésével készült, és melyik modellel (G5).

6. Kiértékelési módszertan

A 2. fejezetben láttuk, hogy a kérdésgenerálás kiértékelése jellemzően a referencia-átfedésre és kis, nem vak humán értékelésekre támaszkodik, és ritkán ellenőrzi a kulcs helyességét. Ebben a fejezetben olyan kiértékelési módszertant mutatunk be, amely (i) a 3. fejezetben definiált, forrásalapú predikátumokat méri, (ii) egytényezős kísérleti tervvel választja szét a komponensek hatását, (iii) a mérőeszközt (a bírót) maga is validálja, és (iv) a dokumentumot tekinti a statisztikai elemzés egységének. A módszertan minden eleme a nyílt forráskódú keretrendszerben (`tests/eval`) implementált.

6.1. A kiértékelő keretrendszer

A keretrendszer egy parancssori eszköz (`python -m mimir_eval`), amelynek fő lépései a 6.1. ábrán láthatók. A `run` lépés egy kar konfigurációja szerint minden dokumentum $\times$ véletlenmag párra vizsgát generál, és a vizsgákat a teljes nyers kimenettel, a visszakeresett kontextussal, a hívásstatisztikával és a GPU-memória-mintákkal együtt JSONL-fájlba írja. A `score` lépés kiszámítja az automatikus mutatókat és meghívja a bírót. A `report` lépés táblázatokat, ábrákat, szignifikanciasztesteket és a dolgozat $\text{\LaTeX}$ -táblázatait állítja elő, az `analyze` lépés pedig a karpárok összehasonlítását végzi. A `rate-export` és `rate-import` lépések az oktatói értékelőlapokat kezelik.



6.1. ábra. A kiértékelő keretrendszer lépései és kimenetei.

A keretrendszer a kinyerést, a darabolást és a visszakeresést a futó szolgáltatások HTTP-végpontjain keresztül végzi, így pontosan azt méri, ami éles üzemben fut; a naiv alapvonal promptját a Bifrost függőségmentes `prompts` moduljából importálja, így az bájtra azonos az éles prompttal; a tervezett csővezetékét pedig konfigurációs kulcsokkal kapcsolható komponensekkel valósítja meg. Minden hívás mellé rögzíti a hívás szakaszát, a tokenszámokat és a késleltetést, a GPU-memóriát pedig egy háttérzál mintavételezi. A reprodukálhatóságot négy mechanizmus biztosítja. (1) Minden futás rögzíti a feloldott konfigurációt, a git-verziót (a „písz-

kos” munkakönyvtár jelzésével), az adatkészlet verzióját, a generátor beállításait és a hardvert (GPU neve, teljes memóriája, meghajtóverzió). (2) A bíró válaszai a (bírómodell, promptverzió, gondolkodási szint, prompt) négyes kriptográfiai lenyomata szerint gyorsítótárba kerülnek, így az újrapontozás determinisztikus, és csak az új kérdésekért kell fizetni. (3) A futások megszakítás után folytathatók: a már elkészült dokumentum $\times$ mag párokat a keretrendszer kihagyja. (4) A teljes `run`→`score`→`report` folyamatot hamis szolgáltatásokkal és ál-nyelvi modellel futó végponttól végpontig tartó tesztek fedik le (a keretrendszer egységtesztjei, 40 tesztfüggvény).

6.2. Adatkészlet

A kiértékelő adatkészlet (`eval-v1`) 39 dokumentumból áll, három forrásból (6.1. táblázat). A forrásokat az döntötte el, hogy minden kérdéshez rendelkezésre álljon a forrásszakasz és a választ igazoló mondat (*evidence*), mert a visszakeresési és tematikus mutatók ezt igénylik.

6.1. táblázat. Az `eval-v1` kiértékelő adatkészlet. R/K/H: rövid ($\leq 6\,000$), közepes ($6\,000$ – $25\,000$) és hosszú ($25\,000$ – $90\,000$ karakter) dokumentumok száma.

Forrás	Nyelv	Dok.	R/K/H	Medián kar.	Ref. kérdés	Licenc
Saját oktatási szövegek	HU	3	3/0/0	1 923	12	saját
OpenStax, EduQG [65]	EN	12	4/4/4	16 752	74	CC BY 4.0
Wikipédia, MILQA [57]	HU	24	8/8/8	8 072	240	CC BY-SA 4.0
Összesen		39	15/12/12	7 919	326	

Saját szövegek. Három rövid magyar oktatási szöveg (a kávé története és feldolgozása; az immunrendszer; a játékelmélet alapjai), könnyű, közepes és nehéz vizsgaszinttel, dokumentumonként négy, kézzel írt referenciakérdéssel és kézzel megjelölt bizonyító mondatokkal. A pilot vizsgálat ezek közül az elsőn (`hu-coffee`) futott.

EduQG. Az EduQG adatkészlet [65] OpenStax-tankönyvekhez írt, szakértői feleletválasztós kérdéseket tartalmaz bizonyító mondatokkal és részben Bloom-címkéssel. Az importáló tárgyterületenként (élettudományok, társadalomtudományok, jog, közgazdaságtan) egy rövid, egy közepes és egy hosszú dokumentumot választ: összefüggő bekezdésablakot keres, amely a hossztartományba esik és a lehető legtöbb referenciakérdés bizonyító mondatát tartalmazza; csak azokat a kérdéseket tartja meg, amelyek bizonyító mondata szó szerint szerepel a dokumentumban.

MILQA. A MILQA [57] a legnagyobb magyar szövegértési adatbázis rövid és hosszú válaszszakaszokkal. A hosszú dokumentumok a Wikipédia azonos című, aktuális cikkéből készülnek; referenciakérdésként azokat tartjuk meg, amelyek bizonyító mondata a jelenlegi cikkben is megtalálható.

A referenciakérdéseket *nem* használjuk célként: egy jó vizsgakérdésnek nem kell a referenciakérdéshez hasonlítania. Szerepük kettős: bizonyító mondataik alapján mérjük a visszakeresés

pontosságát ($\text{recall}@k$, MRR), valamint azt, hogy a generált kérdések a referencia által fontosnak tartott tényeket érintik-e. Az adatkészletet az első éles futás előtt `eval-v1` verzióként befagyaszttjuk; minden későbbi változtatás új verziót kap, és a futások rögzítik a verziót, így eltérő verziókon készült eredmények nem keveredhetnek.

6.3. Kísérleti karok

A kísérleti terv *egytényezős*: minden kar egy referenciakarhoz képest pontosan egy tényezőben tér el, így a különbség az adott tényezőnek tulajdonítható. A 6.2. táblázat a karokat, a 6.3. táblázat a tervezett páros összehasonlításokat és azok kutatási kérdéseit sorolja fel. Minden helyi kar a Qwen2.5-7B-Instruct modellt használja Q4_K_M kvantálással, $T = 0,2$ hőmérséklettel; a szerverkarok az egyetemi GenAI-szerver nagy Qwen-családba tartozó modelljét (qwen38udq8x1).

6.2. táblázat. Kísérleti karok. Minden kar a zárójelben megadott referenciakartól egy tényezőben tér el.

Kar	Konfiguráció	Magok
B-doc-L	teljes dokumentum egy promptban, helyi 7B, 16k kontextus (status quo, helyi)	3
B-doc-S	teljes dokumentum egy promptban, szervermodell (status quo)	3
E0	naiv RAG: 3 darab, a teljes vizsga egy hívásban (alapvonal)	3
E0s	mint E0, de a Bifrost éles végpontján keresztül (implementációs kontroll)	3
E1	vizsgaterv: tervező, slotonkénti keresés, kérdésenként egy hívás (E0)	1
E2	E1 + ellenőrző lánc visszacsatolással (E1)	1
E2f	E2 vak megoldási próba nélkül (E2)	1
E2h	E2 hibrid (BM25 + sűrű, RRF) visszakereséssel (E2)	1
E2x	E2 más model családba tartozó ellenőrzővel, Llama-3.1-8B (E2)	1
E3	E2 + fogalomgráf (E2)	1
E4	E0 a szervermodellel (E0)	3
E4b	E2 a szervermodellel (E2)	1
E5a	E0 a régi, 512 tokenre csonkolt kódolóval (E0)	3
E5b	E0 $\mu + 1,5\sigma$ küszöbvel (E0)	3
E5c	E0 rögzített, 800 karakteres darabokkal (E0)	3

A tervezett karok (E1–E3, E4b) egy véletlenmaggal futnak, mert vizsgánként 3–8 percet igényelnek, és a variancia elsődleges forrása a dokumentum, nem a mag; az egyhívásos karok három maggal futnak, mert olcsók, és a pilot szerint (7.9. szakasz) egyetlen futás zaja jelentős.

6.4. Mutatók

A mutatók három csoportba sorolhatók: bírót nem igénylő *automatikus* mutatók, a bíró ítéletein alapuló *tartalmi* mutatók, valamint *költség* mutatók.

6.3. táblázat. Tervezett páros összehasonlítások és a hozzájuk tartozó kutatási kérdések, hipotézisek.

Összevetés	Vizsgált tényező	RQ	Hipotézis
E1 – E0	tervezés	RQ1, RQ2	1
E2 – E1	ellenőrzés	RQ2	2
E2x – E2	ellenőrző modelleszaládja	RQ2	2
E3 – E2	fogalomgráf	RQ2	–
E2h – E2	hibrid visszakeresés	RQ2	–
E2f – E2	vak megoldási próba	RQ2	–
E1, E2 – B-doc-L	módszer vs. status quo (helyi)	RQ1	3
E4b – B-doc-S	módszer vs. status quo (szerver)	RQ1	3
E2 – E4	helyi módszer vs. szerver alapvonal	RQ3	4
E4 – E0	modellméret	RQ3	4
E5a/b/c – E0	darabolási stratégia	RQ2	–

6.4.1. Automatikus mutatók

A formai megfelelés (F , (3.3)), a lefedettség (Cov , (3.6)), a szivárgás és a nem-kérdés arány (L , N , (3.7)), valamint a duplikátumráta definícióját a 3. fejezet adta meg. Ezeken felül mérjük a meta-hivatkozás arányát (a törzs „a szöveg szerint”, „a fenti részlet” jellegű fordulatot tartalmaz-e, reguláris kifejezéssel), a kulcs lexikális alátámasztottságát (a kulcs szótokenjeinek a kontextusban előforduló aránya), valamint két, a referenciakérdésekre épülő mutatót:

$$\text{GoldSim}(\mathcal{X}) = \frac{1}{|\mathcal{X}|} \sum_{q \in \mathcal{X}} \max_{g \in \mathcal{R}} \cos(\phi(q), \phi(g)), \quad (6.1)$$

$$\text{GoldRecall}(\mathcal{X}) = \frac{1}{|\mathcal{R}|} \sum_{g \in \mathcal{R}} \mathbb{1} \left[\max_{q \in \mathcal{X}} \cos(\phi(q), \phi(g)) \geq 0,85 \right], \quad (6.2)$$

ahol $\mathcal{R}$ a dokumentum referenciakérdéseinek halmaza. A visszakeresés minőségét egy *próba* méri: minden referenciakérdésre lefuttatjuk a visszakeresést, és megnézzük, hányadik helyen szerepel az első olyan darab, amely a bizonyító mondatot tartalmazza (rank_g). Ebből

$$\text{Recall}@k = \frac{1}{|\mathcal{R}|} \sum_g \mathbb{1}[\text{rank}_g \leq k], \quad \text{MRR} = \frac{1}{|\mathcal{R}|} \sum_g \frac{1}{\text{rank}_g}, \quad (6.3)$$

$k \in \{1,3,5,10\}$ mellett, ahol a meg nem talált bizonyítékhoz $1/\text{rank}_g = 0$ tartozik.

6.4.2. Tartalmi mutatók

A tartalmi mutatók a bírő (6.5. szakasz) ítéletein alapulnak: az alátámasztottság (G), a disztraktor-validitás (V), a vak megoldhatóság (A) és ezekből a minőségi index (Q , (3.5)), valamint négy 1–5 skálájú rubrikapontszám és egy bináris „felhasználnám”-ítélet (6.4. táblázat). A rubrika szó szerint azonos a bírő és az oktatók számára, ami a bírő validálásának (5. hipotézis) előfeltétele.

6.4. táblázat. A bíró és az oktatók közös értékelési rubrikája (rövidítve; a teljes szöveg a B. függelékben).

Szempont	Horgonyok
Helyesség	1 = hibás kulcs vagy két helyes opció; 3 = helyes, de pontatlan; 5 = teljesen helyes, egyértelmű
Érthetőség	1 = zavaros; 3 = nehezen érthető; 5 = vizsgakész megfogalmazás, nincs „a szöveg szerint”
Disztraktorok	1 = abszurd vagy szintén helyes; 3 = részben hihető; 5 = mind hihető és hibás
Szintillesztés	1 = egyértelműen rossz Bloom-szint; 3 = határeset; 5 = illik a kért nehézséghez
Felhasználnám	igen/nem: bekerülne-e egy valódi dolgozatba legfeljebb apró javítással

6.4.3. Költségmutatók

Vizsgánként rögzítjük a teljes és a szakaszonkénti futási időt (kinyerés, darabolás, indexelés, tervezés, generálás), a nyelvimodell-hívások számát szakaszonként, a prompt- és a választokenek számát, az értelmezhetetlen válaszok és a hálózati hibák számát, a GPU-memória alapszintjét és csúcsát, valamint azt, hogy a modell teljes egészében a GPU-n futott-e (az Ollama /api/ps végpontja alapján; ha nem, a modell egy része a CPU-ra került, ami a mérés érvényességét érinti).

6.5. A bíró

A bíró a gpt-oss-120b [54] modell az egyetemi GenAI-szerveren, $T = 0$ hőmérséklettel, alacsony gondolkodási szinttel („Reasoning: low”) és 16 384 tokenes kontextussal. A választás elsődleges szempontja, hogy más modellcsaládba tartozzon, mint bármelyik generátor (Qwen, Llama), így az önpreferencia torzítása [53] nem lép fel. Kérdésenként két hívást végez:

1. **Áttekintés** (látja a kulcsot): igazolja-e a forrás a megjelölt kulcsot, egy szó szerinti bizonyító mondat idézésével; minden disztraktorra, hogy a forrás szerint hamis-e és témába vágó-e; a négy rubrikapontszám; a kérdés Bloom-szintje; és a „felhasználnám”-ítélet, rövid indoklással.
2. **Vak megoldás** (nem látja a kulcsot): a forrás alapján válaszolnia kell a kérdésre, véletlenül permutált opciókkal.

A bíró forrásként alapértelmezésben a kérdéshez tartozó darabokat kapja, nem a teljes dokumentumot. Ezt a fő vizsgálat futási idejének csökkentése indokolja (a bíró egy hívása a megosztott szerveren kb. 14 másodperc, és ez a mérés szűk keresztmetszete); a választás korlátja, hogy a bíró a dokumentum más részeiben lévő ellentmondást nem láthatja (8.3. szakasz). A `context : document` beállítás a teljes dokumentumot adja át, a pilot így futott.

6.6. Oktatói értékelés

A bíró csak akkor tekinthető érvényes mérőeszköznek, ha ítéletei az emberi szakértőkével egyeznek. A fő vizsgálatban ezért három oktató vakon értékeli egy rétegzett mintát: a négy fő karból (B-doc-S, E0, E1, E2 vagy E2x) karonként 15 kérdést, azaz oktatónként 60 kérdést, valamint 10 közös kalibrációs kérdést. A lapok nem tartalmazzák a kar nevét, a kérdések sorrendje véletlen, a rubrika azonos a bíróéval. Az értékelő–értékelő egyezést Krippendorff- α -val (ordinális skálán), a bíró–oktató egyezést szempontonként Spearman- ρ -val és Krippendorff- α -val mérjük, a bináris ítéletekre Cohen- κ -t számolunk.

6.7. Statisztikai elemzés

6.7.1. Az elemzés egysége

A kérdések nem függetlenek: ugyanabból a dokumentumból, ugyanazon a futáson belül készülnek, így a kérdésszintű tesztek a szabadsági fokot mesterségesen felfújják (pszeudoreplikáció). Ezért minden mutatót először dokumentumon belül átlagolunk (a kérdésekre és a magokra), és a statisztikai következtetés egysége a *dokumentum*. Egy m mutató a karon mért értéke a d dokumentumon

$$\bar{m}_{a,d} = \frac{1}{|\Omega_a|} \sum_{\omega \in \Omega_a} m(\mathcal{X}_a(d, \omega)), \quad (6.4)$$

ahol Ω_a a kar magjainak halmaza.

6.7.2. Konfidenciaintervallum

A karonkénti átlag 95 %-os konfidenciaintervallumát percentilis bootstrappal [66] becsüljük, a dokumentumok visszatevéses újramintavételezésével, $B = 10000$ ismétléssel:

$$\hat{\mu}^{*(b)} = \frac{1}{D} \sum_{i=1}^D \bar{m}_{a,d_i^{*(b)}}, \quad \text{CI}_{95} = [\hat{\mu}_{(0,025)}^*, \hat{\mu}_{(0,975)}^*], \quad (6.5)$$

ahol $d_i^{*(b)}$ a b -edik mintában a D dokumentumból egyenletesen, visszatevéssel húzott elem.

6.7.3. Páros összehasonlítás

Két kar összevetésére a páros Wilcoxon-féle előjeles rangpróbát [67] használjuk, amely nem feltételez normalitást – ami a $[0,1]$ -be szorított, gyakran plafonhatást mutató rátáknál lényeges. A d dokumentumon mért különbség $\Delta_d = \bar{m}_{b,d} - \bar{m}_{a,d}$; a nullától különböző különbségeket abszolút értékük szerint rangsoroljuk (R_d), és

$$W^+ = \sum_{d:\Delta_d>0} R_d, \quad W^- = \sum_{d:\Delta_d<0} R_d. \quad (6.6)$$

A hatásméret a páros rangbiszeriális korreláció [68]:

$$r_{rb} = \frac{W^+ - W^-}{W^+ + W^-} \in [-1, 1], \quad (6.7)$$

amely az előjeles „egyszerű különbség” értelmezést adja: a kedvező és kedvezőtlen rangtömeg különbsége.

Megjegyzés (A minimális mintaméret). Egzakt kétoldali Wilcoxon-próbánál a legkisebb elérhető p -érték D nem nulla különbségű pár esetén $2 \cdot 2^{-D}$ (amikor minden különbség azonos előjelű). $D = 5$ -re ez 0,0625, $D = 6$ -ra 0,03125. Hat dokumentumnál kevesebből tehát $p < 0,05$ *elvileg* sem érhető el; az egyetlen dokumentumos pilot eredményei ezért kizárólag leíró jellegűek.

6.7.4. Többszörös összehasonlítás

A 6.3. táblázat összehasonlításainak p -értékeit Holm szekvenciális eljárásával [69] korrigáljuk. Ha a rendezett p -értékek $p_{(1)} \leq \dots \leq p_{(M)}$, a korrigált értékek

$$\tilde{p}_{(i)} = \max_{j \leq i} \min(1, (M - j + 1) p_{(j)}), \quad (6.8)$$

amely a családonkénti elsőfajú hibát α szinten tartja, a Bonferroni-korrekciónál nagyobb erővel.

6.7.5. Egyezési mutatók

Két értékelő (vagy az ellenőrző és a bíró) bináris ítéleteinek egyezését Cohen- κ -val mérjük [70]:

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \quad p_e = \sum_c \hat{\pi}_{1,c} \hat{\pi}_{2,c}, \quad (6.9)$$

ahol p_o a megfigyelt egyezési arány, $\hat{\pi}_{j,c}$ pedig a j -edik értékelő c kategóriájának relatív gyakorisága. Több értékelő és hiányzó értékek esetén Krippendorff- α -t használunk [71]:

$$\alpha = 1 - \frac{D_o}{D_e} = 1 - (n - 1) \frac{\sum_{c,k} o_{ck} \delta_{ck}^2}{\sum_{c,k} n_c n_k \delta_{ck}^2}, \quad (6.10)$$

ahol o_{ck} a koincidenciamátrix, n_c a c érték marginális gyakorisága, n a párosítható értékek száma, ordinális skálán pedig $\delta_{ck}^2 = (\sum_{g=c}^k n_g - \frac{n_c + n_k}{2})^2$. Az irodalmi konvenció szerint $\alpha \geq 0,8$ megbízható, $0,667 \leq \alpha < 0,8$ óvatos következtetésre elegendő.

6.7.6. Az ellenőrző mint osztályozó

Az ellenőrző lánc egy bináris osztályozó: minden kérdést *átengedett* vagy *megjelölt* kategóriába sorol. A bíró alátámasztottsági ítéletét referenciának tekintve az ellenőrző

$$\text{Prec} = \frac{|\{q : \text{átengedett} \wedge G(q) = 1\}|}{|\{q : \text{átengedett}\}|}, \quad \text{Rec} = \frac{|\{q : \text{átengedett} \wedge G(q) = 1\}|}{|\{q : G(q) = 1\}|} \quad (6.11)$$

pontossággal és felidézéssel, valamint a két bináris változó közötti Cohen- κ -val jellemezhető. Egy hasznos ellenőrzőtől $\kappa \gg 0$ várható; $\kappa \approx 0$ azt jelenti, hogy az ellenőrző ítélete a bírótól független, vagyis a jelzése az oktató számára nem hordoz információt.

6.8. Mérési validitás: a korábbi mérések hibái

A keretrendszer építése két olyan hibát tárt fel, amely a rendszer korábbi, végponttól végpontig tartó méréseit érvénytelenítette. Ezek bemutatása azért lényeges, mert pontosan azt a kockázatot illusztrálják, amely ellen a módszertan védekezik: egy mérés, amely „működik”, mégis mást mér, mint amit gondolunk.

Tartalékvizsgák eredményként. Ha minden modellhívás sikertelen volt, a háttérrendszer egy beégetett, egykérdéses tartalékvizsgát adott vissza hibaüzenet helyett. Egy 2026. május 3-i tesztfutás mindhárom tesztdokumentumot 4–9 másodperc alatt „generáltnak” jelentette, holott mindhárom „vizsga” ez a tartalék volt. A keretrendszer azóta a tartalékot hibajelzők alapján felismeri, és sikertelen futásként számolja.

Csonkolt darabolás. Az 5.3.3. szakaszban elemzett hiba miatt a dokumentumok első oldala utáni részét a darabolás gyakorlatilag a tartalomtól függetlenül vágta. A kódoló azóta ablakozva dolgozik; az E5a kar a régi viselkedést méri.

Válaszszivárgás. A harmadik hibát a pilot tárta fel: a kérdés törzsébe másolt kulcs a bíró és az automatikus mutatók értékét is felfelé torzítja (7.5. szakasz).

6.9. Hardver

A helyi karok egy fogyasztói GPU-n (NVIDIA GeForce RTX 3070 Ti, 8 GB) és – a hosszú futások miatt – egy egyetemi munkaállomáson (NVIDIA RTX A4500) futnak; mivel minden futás rögzíti a GPU típusát, a két gépen készült eredmények elkülöníthetők, és a futásidők csak azonos hardveren hasonlíthatók össze. A fő vizsgálat futtatási tervét (füstteszt, helyi és szerverkarok, pontozás, oktatói értékelés, jelentés) a C.3. függelék tartalmazza.

7. Eredmények

Ebben a fejezetben a pilot vizsgálat eredményeit közöljük. A pilot célja nem a hipotézisek eldöntése volt – egyetlen dokumentumon ez a 6.7. szakasz szerint elvileg sem lehetséges –, hanem (i) a teljes mérési lánc validálása, (ii) a komponensek hatásának és költségének előzetes becslése, és (iii) a fő vizsgálat tervének megalapozása. Minden közölt szám a `tests/eval/results_pilot` könyvtár nyers adataiból, a keretrendszer `report` lépésével készült; a táblázatok és a nyers adatok közötti megfeleltetést a C. függelék adja meg.

7.1. A pilot beállításai

A pilot a `hu-coffee` dokumentumon futott: egy 1 702 karakteres magyar oktatási szöveg a kávé eredetéről, fajtáiról, termesztéséről és feldolgozásáról, könnyű vizsgaszinttel. A szemantikus darabolás négy darabot adott (a szórásalapú és a rögzített darabolás hármát). Minden kar dokumentumonként három vizsgát generált (magok: 1, 2, 3), vizsgánként tíz feleletválasztós kérdéssel, így karonként 30, összesen 300 kérdés készült. A tíz kar: E0, E1, E2, E2f, E2h, E3, E4, E5a, E5b, E5c; az E4b kar futása nem fejeződött be, és a teljes dokumentumos alapvonalak, valamint az E2x kar a pilot után készültek el.

A helyi karok egy asztali PC fogyasztói GPU-ján (NVIDIA GeForce RTX 3070 Ti, 8 GB) futottak, a generátor 16 384 tokenes kontextussal. A pilot még az 5. fejezetben „új”-ként jelölt szabályok nélkül futott: a tervezett karokban $r = 2$ újrapróbálkozás volt engedélyezve, szivárgási szabályok és javító hívás nélkül. A bíró a teljes dokumentumot látta (`judge-v1`), ami egy ilyen rövid szövegnél lényegében azonos a kérdés kontextusával.

7.2. Összesített eredmények

A 7.1. táblázat a karonkénti fő mutatókat a minőségi index szerint csoportosítva mutatja. A 7.1. ábra ugyanezeket a mutatókat hő térképként, a visszakeresési találati aránnyal kiegészítve ábrázolja.

Három megfigyelés emelhető ki. *Először*, a helyi karok minőségi indexe 0,73 és 0,90 között szóródik, a szervermodell (E4) pedig 0,99-cel kiemelkedik. *Másodszor*, a tervezés (E1) a naiv alapvonalhoz képest minden tartalmi mutatón javított, az ellenőrző karok (E2, E2f, E2h, E3) viszont a formai megfelelésben 67 %-ra estek vissza, ami a Q indexüket jelentősen lerontotta. *Harmadszor*, a lefedettség a szervermodellnél a legalacsonyabb (25 %): a legjobb minőségű kar a dokumentum négy darabjából egyre koncentrált. A visszakeresési találati arány minden karon 100 % volt, ami egy négydarabos dokumentumon várható (a három visszakeresett darab a négyből szinte mindig tartalmazza a bizonyítékot); a visszakeresés mutatói csak a hosszú dokumentumokon lesznek informatívak.

7.1. táblázat. A pilot fő eredményei: egy magyar dokumentum, 3 mag, karonként 30 kérdés. Ráták %-ban. *Q*: minőségi index; Form.: formai megfelelés; Alát.: alátámasztottság; Vak: vak megoldhatóság; Diszt.: disztraktor-validitás; Lef.: lefedettség; Sziv.: kulcs a törzsben; Nell.: nem ellenőrzött kérdések száma vizsgánként; Hívás: átlagos modellhívás vizsgánként; Perc: medián futási idő vizsgánként.

Kar	<i>Q</i>	Form.	Alát.	Vak	Diszt.	Lef.	Sziv.	Nell.	Hívás	Perc
E0	0,83	100	67	85	80	50	0	–	1	0,3
E1	0,88	100	73	93	87	58	0	2,0	23	3,0
E2	0,76	67	73	77	89	58	3	4,3	75	6,2
E2f	0,73	67	53	82	90	58	7	3,3	59	5,3
E2h	0,76	67	67	79	93	67	10	4,0	76	6,7
E3	0,85	67	87	100	87	50	20	6,0	101	8,0
E4	0,99	100	100	100	97	25	0	–	1	6,6
E5a	0,90	100	80	86	93	50	0	–	1	0,3
E5b	0,90	100	80	90	91	56	0	–	1	0,3
E5c	0,81	100	60	79	83	67	0	–	1	0,3

	Formai megfelelés	Forrással igazolt válasz	Vakon megoldható	Jó disztraktorok	„A szöveg szerint” arány (kisebb a jobb)	Lefedettség	Keresési találat @3
E4 – Alapmódszer, szervermodell	100%	100%	100%	97%	0%	25%	100%
E5b – Szórás alapú darabolás	100%	80%	90%	91%	0%	56%	100%
E5a – Régi darabolás	100%	80%	86%	93%	0%	50%	100%
E1 – Tervezés	100%	73%	93%	87%	0%	58%	100%
E3 – Ellenőrzés + fogalomgráf	67%	87%	100%	87%	0%	50%	100%
E0 – Alapmódszer	100%	67%	85%	80%	0%	50%	100%
E5c – Fix darabolás	100%	60%	79%	83%	0%	67%	100%
E2h – Ellenőrzés + hibrid keresés	67%	67%	79%	93%	0%	67%	100%
E2 – Tervezés + ellenőrzés	67%	73%	77%	89%	0%	58%	100%
E2f – Ellenőrzés vak teszt nélkül	67%	53%	82%	90%	0%	58%	100%

PILOT – kevés dokumentum, nem végleges

7.1. ábra. Az egyes minőségi mutatók karonként (pilot). A „szöveg szerint” oszlop a meta-hivatkozások aránya, a keresési találat@3 a referenciakérdések bizonyító mondatának visszakeresési aránya.

7.3. A tervezés hatása (RQ1, RQ2)

A naiv alapvonal (E0) és a tervezett, de nem ellenőrzött csővezeték (E1) összevetése (7.1. és 7.6. táblázat) szerint a tervezés azonos modell és azonos formai megfelelés mellett javította az alátámasztottságot (+6 százalékpont), a vak megoldhatóságot (+8), a disztraktor-validitást (+7) és a lefedettséget (+8), a bíró helyességi pontszámát 3,5-ről 3,7-re, disztraktorpontszámát 3,5-ről 4,0-ra, a „felhasználnám” arányt 63-ról 67 %-ra emelte, az érthetőségi pontszám viszont 4,7-ről 4,0-ra csökkent. Az ára: 23-szoros hívásszám és 9-szeres futási idő (20 másodperc helyett 3 perc).

Az érthetőség csökkenése mögött a kérdések formájának megváltozása áll: az egyenkénti

generálás hosszabb törzseket és jóval hosszabb válaszlehetőségeket eredményezett (7.2. táblázat). A naiv alapvonal egy vizsgán belül rövid, tényfelidéző kérdéseket írt, 60 karakteres átlagos törzssel és négy opcióra összesen 60 karakterrel („9. század”, „10. század”, ...); a tervezett csővezeték kérdései átlagosan 88 karakteres törzsűek, az opciók együttes hossza pedig 315 karakter, vagyis tipikusan teljes mondatok. Ez a fogalmi, magyarázó kérdéseknek kedvez (a disztraktorpontszám nőtt), de a kérdés olvasási terhet is növeli.

7.2. táblázat. A kérdések hossza, változatossága és tematikus hasonlósága karonként (pilot, átlagok 30 kérdésen). Törzs: a törzs karakterszáma; Opciók: a négy opció együttes karakterszáma; Dup.: duplikátumráta; RefSim: átlagos maximális hasonlóság a referenciakérdésekhez.

Kar	Törzs	Opciók	Dup. (%)	RefSim
E0	60	60	16	0,895
E1	88	315	21	0,912
E2	84	348	21	0,909
E2f	95	405	17	0,911
E2h	93	352	14	0,907
E3	120	365	25	0,895
E4	54	87	4	0,890
E5a	65	88	16	0,898
E5b	58	73	10	0,891
E5c	57	36	45	0,885

A referenciakérdésekhez való hasonlóság (RefSim) a tervezett karokban a legmagasabb (0,907–0,912), ami arra utal, hogy a tervező által választott fogalmak jobban egyeznek a szakértő által fontosnak tartott tényekkel. A különbség azonban kicsi, és egyetlen dokumentumon nem értelmezhető megbízhatóan.

A tervezett Bloom-szint és a megvalósult szint. A könnyű vizsgaszinthez a terv 50–50 %-ban emlékezés és megértés szintű kérdéseket írt elő (5.2. táblázat), a bíró azonban az E1 kar 30 kérdéséből 29-et emlékezés szintűnek sorolt be (E2: 28, E2h: 24, E3: 30). A kis modell tehát a promptban megadott kognitív szintet nem valósítja meg megbízhatóan: a „megértés” szintű slotokból is jellemzően tényfelidéző kérdés lett. A szintillesztési pontszám ennek ellenére 5,0, mert a könnyű nehézséghez mindkét szint elfogadható. Ez a megfigyelés arra utal, hogy a Bloom-szint mint szerkezeti garancia (3.1. állítás) csak a *kért* szintre vonatkozik; a megvalósult szint ellenőrzése egy további, eddig hiányzó ellenőrzési lépés lenne.

7.4. Az ellenőrzés hatása

7.4.1. A formai hiba és a tartalékkérdés

Az ellenőrző karok mindegyikében (E2, E2f, E2h, E3) a három vizsgából egyben előfordult, hogy egy slot minden kísérlete, a tartalékfogalommal végzett kísérleteket is beleértve, elbukott,

és a megtartott végső kérdésnek csak két vagy három válaszlehetősége volt. Ez a vizsga formai megfelelését nullára, a minőségi indexét 0,25-tel csökkentette – a négy kar Q indexének csökkenése az E1-hez képest nagyrészt ebből ered. A hiba nem a módszer elvi korlátja, hanem implementációs hiányosság volt: a végső kérdés kiválasztása csak a hibák számát nézte, a formát nem. A javítás az (5.7) lexikografikus kulcs és a javító hívás (5.3. algoritmus); a fő vizsgát ezt már tartalmazza.

7.4.2. Az ellenőrző mint osztályozó

A formai hibától függetlenül az ellenőrzés a tartalmi mutatókon sem javított: a tervezésre épülő ellenőrzés (E2) alátámasztottsága az E1-ével azonos (73 %), vak megoldhatósága pedig 93-ról 77 %-ra csökkent. Az ok megértéséhez az ellenőrzőt osztályozóként értékeltük (7.3. és 7.4. táblázat).

7.3. táblázat. Egyezik-e a 7B ellenőrző ítélete a bíróéval? Az E2, E2f, E2h és E3 karok pilot kérdései az ellenőrző ítélete szerint; a bírói kimenetek %-ban.

Ellenőrző ítélete	n	Alátámasztott	Vakon helyes	Felhasználnám
Átengedett (ellenőrzött)	67	66	89	61
Megjelölt (nem ellenőrzött)	53	75	79	64

7.4. táblázat. Az ellenőrző pontossága, felidézése és Cohen- κ együtthatója a bíró alátámasztottsági ítéletéhez képest, karonként. Átengedett: az átengedett kérdések aránya (%).

Kar	n	Átengedett	Pontosság	Felidézés	κ
E2	30	57	0,71	0,55	–0,07
E2f	30	67	0,50	0,62	–0,09
E2h	30	60	0,61	0,55	–0,14
E3	30	40	0,92	0,42	0,07

Az eredmény egyértelmű: az ellenőrző által *átengedett* kérdéseket a bíró *ritkábban* (66 %) ítélte alátámasztottnak, mint a *megjelölteket* (75 %); a vak megoldhatóságban az átengedettek kissé jobbak (89 vs. 79 %), a „felhasználnám” arány pedig gyakorlatilag azonos. Az ellenőrző κ együtthatója minden karon –0,14 és 0,07 között van, vagyis az ellenőrző ítélete a véletlen egyezés szintjén áll. Az E3 kar magas pontossága (0,92) alacsony felidézéssel párosul: az ellenőrző ott a kérdések 60 %-át megjelölte, és az átengedettek zöme valóban alátámasztott volt – ez azonban részben a 7.5. szakaszban tárgyalt szivárgásnak köszönhető.

Az értelmezésünk szerint egy 7B modell, amely egyben a generátor is, nem tudja megbízhatóan eldönteni, hogy egy szövegrész következményként tartalmaz-e egy állítást; az ellenőrzés így a generátorral korrelált, nem független információforrás. Ez összhangban van azzal, hogy külső visszajelzés nélkül a modellek önjavítása nem javít [10], és azzal, hogy a modellek saját kimenetüket előnyben részesítik [53]. Két következménye van a fő vizsgálatra: az ellenőrző

jelzését nem szabad az oktátónak megbízhatóságként bemutatni, amíg osztályozóként nem validáltuk; és külön karban (E2x) kell tesztelni, hogy egy másik modelleszaládba tartozó ellenőrző független-e a generátortól.

7.5. Válaszszivárgás: egy új hibamód

A kérdések kézi átolvasása olyan hibát tárt fel, amelyet sem az automatikus mutatók, sem a bíró nem büntetett: egyes ellenőrzött kérdések egy forrásmondatot másolnak a törzsbe, és ugyanazt a mondatot ismétlik meg kulcsként. A 7.5. táblázat a szivárgás gyakoriságát mutatja.

7.5. táblázat. Válaszszivárgás karonként (pilot, 30 kérdés karonként, %). Kulcs a törzsben: legalább háromszavas kulcs, amelynek szótokenjei legalább 80 %-ban a törzsben is szerepelnek; nem kérdés: a törzs sem kérdőjelre, sem kettőspontra nem végződik; bírói helyesség: a szivárgó kérdések átlagos helyességi pontszáma (1–5).

Kar	<i>n</i>	Kulcs a törzsben	Nem kérdés	Bírói helyesség (szivárgó)
E0	30	0	0	–
E1	30	0	13	–
E2	30	3	17	5,0
E2f	30	7	10	5,0
E2h	30	10	10	2,3
E3	30	20	47	5,0
E4	30	0	0	–
E5a	30	0	33	–
E5b	30	0	0	–
E5c	30	0	0	–

A szivárgás kizárólag az ellenőrző karokban fordult elő, és ott a legerősebb, ahol a legtöbb újrapróbálkozás történt (E3: 20 % szivárgás, 47 % nem-kérdés törzs). A bíró a szivárgó kérdések többségének 5-ös helyességi pontszámot adott, ami érthető: a kulcs valóban alátámasztott, hiszen szó szerint a forrásból származik, és a kérdés „vakon” is megoldható, hiszen a válasz a kérdésben van. A szivárgás tehát *egyszerre* torzítja felfelé az alátámasztottságot, a vak megoldhatóságot és a bírói pontszámot – ez magyarázza, hogy az E3 kar miért érte el a legmagasabb helyi alátámasztottságot (87 %) és vak megoldhatóságot (100 %).

Értelmezésünk szerint az ellenőrző visszacsatolási hurka a kis modellt a forrásszöveg szó szerinti átvétele felé tolja, mert ez a legolcsóbb módja annak, hogy egy alátámasztottsági ellenőrzésen átjusson. Ez a jelenség a Goodhart-törvény egy példája: a mérőszám célponttá válva elveszíti mérő jellegét. A fő vizsgálatra két következménye van: a szivárgást és a nem-kérdés törzset az ellenőrző lánc szabályként elutasítja (5.3. táblázat), és mindkettő mutatóként is szerepel minden jelentésben.

7.6. Szervermodell és helyi modell (RQ3)

A nagy servermodell naiv RAG-gal (E4) minden tartalmi mutatón a legjobb volt: alátámasztottság és vak megoldhatóság 100 %, disztraktor-validitás 97 %, a bíró minden kérdését felhasználná (7.6. táblázat). A lefedettsége azonban a legalacsonyabb (25 %): mindhárom vizsgában az összes kérdés a dokumentum négy darabja közül ugyanahhoz az egyhez állt a legközelebb. Ez a P2 probléma – a kérdések egy szakaszra tömörülése – kis léptékben: a nagyobb modell sem terjeszti szét magától a kérdéseit, ha a feladat erre nem kényszeríti. A megosztott egyetemi serveren a medián futási idő 6,6 perc volt vizsgánként, ugyanannyi, mint az ellenőrzött helyi csővezetéké (6,2 perc), miközben a helyi naiv alapvonal 20 másodperc alatt végzett.

7.6. táblázat. A bíró rubrikapontszámai (1–5, átlagok 30 kérdésen) és a „felhasználnám” arány karonként (pilot).

Kar	Helyesség	Érthetőség	Disztraktorok	Szintillesztés	Felhasználnám
E0 (naiv)	3,5	4,7	3,5	4,8	63 %
E1 (terv)	3,7	4,0	4,0	5,0	67 %
E2 (+ ellenőrzés)	3,4	4,1	3,6	5,0	60 %
E2f (vak próba nélkül)	3,1	4,0	4,0	4,9	50 %
E2h (hibrid keresés)	3,5	4,1	4,2	4,8	63 %
E3 (+ gráf)	4,4	4,4	3,9	5,0	77 %
E4 (server, naiv)	5,0	5,0	4,6	5,0	100 %
E5a (régí kódoló)	3,8	4,3	3,9	5,0	70 %
E5b (szórásküszöb)	4,1	4,5	3,7	4,9	77 %
E5c (fix darabok)	3,3	4,0	3,0	5,0	57 %

A helyi modell erőforrásigényét a 4.4. szakasz részletezte: a modell és a futatókörnyezet 5 400–5 950 MiB-ot foglalt, a csúcs 7 808 MiB volt, így a helyi csővezeték egy 8 GB-os kártyán az asztali környezet mellett is fut. A 4. hipotézis szerinti $< 0,1$ különbséget a pilot nem támasztja alá (E4: 0,99; a legjobb tervezett helyi kar, E1: 0,88), a legjobb egyhívásos helyi karok (E5a, E5b: 0,90) azonban 0,09-re megközelítették.

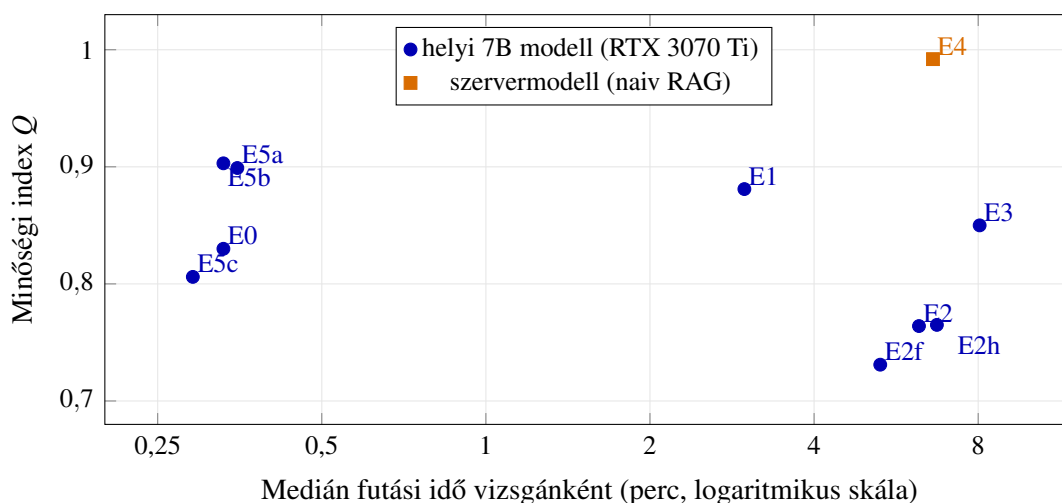
7.7. Költség

A 7.7. táblázat a hívásszámot szakaszonként, a tokenfelhasználást és a futási időt mutatja. A 7.2. ábra a minőséget a futási idő függvényében ábrázolja.

Az ellenőrző karok hívásszáma (59–101) messze meghaladja a (3.9) szerinti alsó korlátot (21–31), ami azt jelzi, hogy a kérdések jelentős része legalább egyszer elbukott. Az E2 kar slotonként átlagosan 3,1 generálást igényelt; a (3.10) képletet a tartalékfogalommal együtt alkalmazva ez $p \approx 0,3$ körüli kísérletenkénti átmenési valószínűségnek felel meg – azaz a kis modell ellenőrzője a kérdések kétharmadát első körben elutasította, miközben (az előző szakasz szerint) ítélete a bíróéval nem egyezett. Az ellenőrzés tehát a pilotban a hívásszámot 60–100-szorosára növelte

7.7. táblázat. Modellhívások szakaszonként, tokenfelhasználás és futási idő vizsgáknént (pilot, átlagok 3 vizsgán; az idő medián). Terv.: tervező vagy gráfépítés; Gen.: generálás; Alát.: alátámasztottsági ellenőrzés; Vak: vak megoldási ellenőrzés.

Kar	Terv.	Gen.	Alát.	Vak	Összes	Token (e)	Perc
E0	–	1	–	–	1	2,8	0,3
E1	1	22,0	–	–	23,0	45,2	3,0
E2	1	31,0	29,0	14,3	75,3	96,7	6,2
E2f	1	29,3	28,3	–	58,7	84,1	5,3
E2h	1	30,7	29,7	15,0	76,3	99,2	6,7
E3	1	43,7	37,3	18,7	100,7	119,6	8,0



7.2. ábra. Minőség és futási idő a pilotban. A tervezett karok egy nagyságrenddel lassabbak; a minőség-növekedést csak a tervezés (E1) hozta meg ellenőrzés nélkül.

anélkül, hogy információt adott volna. A tervezés (E1) ezzel szemben 23 hívással mérhető minőségjavulást hozott; a pilot alapján a fő vizsgálatban a tervezés bizonyítása, az ellenőrzés pedig érdemének igazolása a feladat.

7.8. Darabolási stratégiák

Az egyhívásos alapvonalat három darabolási változattal is összevetettük (7.1. és 7.2. táblázat). A rögzített, 800 karakteres ablakok (E5c) adták a leggyengébb eredményt: a kérdéspárok 45 %-a közel-duplikátum volt (a szemantikus darabolásnál 10–16 %), és a bíró disztraktorpontszáma 3,0-ra esett (a szemantikus változatoknál 3,5–3,9). A szórásalapú küszöb (E5b) és a régi kódoló (E5a) 0,90-es indexet ért el. Az utóbbi eredmény nem jelenti a régi kódoló előnyét: egy 512 tokennél rövidebb dokumentumon a régi és az új kódoló *azonos* darabokat ad, így az E0 és az E5a közötti különbség kizárólag a futásról futásra jelentkező zajból ered.

7.9. Futásról futásra jelentkező zaj

Az E0 és az E5a kar a pilotban azonos darabokat, azonos visszakeresett kontextust és azonos magokat kapott (ezt a nyers adatok, a `exams.jsonl` fájlok `retrieved` mezői igazolják), mégis eltérő kérdéseket, és 0,83, illetve 0,90 minőségi indexet adtak. Ez a különbség tehát egy természetes kontrollkísérlet: a teljes mérési láncot változatlanul hagyva, csak az ismétlés miatt 0,07-es eltérés adódott. Az ok az, hogy a GPU-n végzett következtetés alacsony hőmérsékleten és rögzített maggal sem bitre reprodukálható (a párhuzamos lebegőpontos redukciók sorrendje nem determinisztikus).

Ebből két következtetés adódik. *Egyrészt*, egyetlen dokumentumon a 0,07-nél kisebb különbségek zajnak tekintendők – a pilot szinte minden helyi karpárjának különbsége ebbe a sávba esik, a tervezés hatása (+0,05) is. *Másrészt*, a zaj csökkentésének hatékony módja nem a magok számának növelése egyetlen dokumentumon, hanem a dokumentumok számának növelése, mert a fő vizsgálat hipotézisei dokumentumok populációjára vonatkoznak. A fő vizsgálat ezért 39 dokumentumon, dokumentumszintű statisztikával fut.

7.10. Példák

A 7.8. táblázat négy pilotkérdést mutat eredeti, magyar nyelvűen, a bíró ítéletével. A példák a fenti eredmények tipikus eseteit illusztrálják: a naiv alapvonal téves kulcsát, a tervezett kar alá nem támasztott részletét, a szivárgó kérdést és a szervermodell jó kérdését.

7.8. táblázat. Pilotkérdések eredeti nyelven (* = megjelölt kulcs), a bíró ítéletével.

E0	<i>A kávé felfedezéséhez milyen században jutott el a velencei kereskedők révén? 9. század* / 10. század / 8. század / 11. század. Bíró: nem alátámasztott – a forrás szerint a kávé a 17. században jutott el Velencébe; a 9. század az etióp legenda ideje. Egyik opció sem helyes; helyesség 1/5.</i>
E1	<i>A pörkölés hőmérsékletének milyen hatással van az italt és a koffeintartalomra? Kulcs: A sötét pörkölés testesebb, füstösebb és keserűbb ízvilágot eredményez, de a koffeintartalom csökken.* Bíró: az ízhatást a forrás igazolja, a koffeintartalomról azonban nem szól, így a kulcs nem alátámasztott; a disztraktorok jók; helyesség 1/5. (A törzs nyelvtani hibája a 7B modell magyar nyelvi korlátját is mutatja.)</i>
E3	<i>„A kávéfogyasztás szokása Etiópiából hamar áterjedt a szomszédos Jemenbe, majd az arab világba, Európába pedig a 17. században jutott el a velencei kereskedők révén.” Kulcs: ugyanez a mondat*; disztraktorok: a mondat a 16., 18., illetve 15. századdal. Bíró: alátámasztott, vakon helyes, helyesség 5/5 – noha a „törzs” nem kérdés, és a választ szó szerint tartalmazza.</i>
E4	<i>Mennyit tesz ki az Arabica a globális kávétermelésből? Mintegy 70 százalékat* / Mintegy 30 százalékat / Mintegy 50 százalékat / Mintegy 90 százalékat. Bíró: a forrás a 70 %-ot közvetlenül kimondja; a többi arány hihető, de a szöveg cáfolja; helyesség 5/5.</i>

7.11. A hipotézisek előzetes értékelése

A 7.9. táblázat összefoglalja, hogy a pilot milyen irányú bizonyítékot szolgáltat az egyes hipotézisekhez. Hangsúlyozzuk, hogy ezek egyetlen dokumentumon, szignifikanciavizsgálat nélkül

tett, leíró megfigyelések; a hipotézisek eldöntése a fő vizsgálat feladata.

7.9. táblázat. A hipotézisek és a pilot által szolgáltatott előzetes, leíró bizonyíték.

Hipotézis	Pilot megfigyelés	Irány
1. Tervezés	alátámasztottság +6, lefedettség +8 pp, forma változatlan; a különbség a zajsávon belül	támogató, nem döntő
2. Ellenőrzés	alátámasztottság változatlan, vak megoldhatóság –16 pp; $\kappa \in [-0,14; 0,07]$	ellentmondó (önellenőrzésre)
3. Lefedettség	a szervermodell naiv RAG-gal 25 % lefedettséget ért el; a B-doc kar még nem futott	támogató (közvetett)
4. Helyi vs. szerver	különbség 0,09–0,11; memória 5,4–5,9 GiB	részben támogató
5. Mérés	oktatói értékelés még nincs; a bíró a szivárgó kérdéseket nem büntette	nyitott; kockázat azonosítva

7.12. A fő vizsgálat eredményei

A fő vizsgálat (`eval-v1`, 39 dokumentum, a 6.2. táblázat összes kara, oktatói értékeléssel) futása e dolgozat lezárásakor folyamatban van. Eredményei a keretrendszer `report` lépésével, a pilottal azonos formában, automatikusan kerülnek a 7.10. táblázatba és a hozzá tartozó szignifikanciátáblába. A táblázat szerkezetét itt rögzítjük; a piros mezők a fő futás adataira várnak.

7.10. táblázat. A fő vizsgálat eredményei (39 dokumentum; átlag és 95 %-os bootstrap-CI; páros Wilcoxon-próba Holm-korrekciónal a referenciakarhoz képest). **A fő futás után kitöltendő.**

Kar	Ref.	Q [95 % CI]	Alát.	Lef.	$\tilde{p}$ (Holm)	r_{rb}
B-doc-L	–	–	–	–	–	–
E0	–	–	–	–	–	–
E1	E0	–	–	–	–	–
E2	E1	–	–	–	–	–
E2x	E2	–	–	–	–	–
E3	E2	–	–	–	–	–
E4	E0	–	–	–	–	–
E4b	E2	–	–	–	–	–
B-doc-S	–	–	–	–	–	–

8. Diszkusszió

8.1. Miért módszer, és nem prompt?

A dolgozat központi állítása, hogy a vizsgagenerálás megbízhatóságát nem egy jobb prompt, hanem a feladat explicit, ellenőrizhető lépésekre bontása biztosítja. A pilot három ponton támasztja alá ezt az érvelést. *Először*, ugyanaz a 7B modell pusztán a tervezés hatására alátámasztottabb, vakon jobban megoldható, jobb disztraktorú és jobban lefedő vizsgát írt; a különbség nem a modell tudásából, hanem a feladat szerkezetéből ered. *Másodszor*, egy lényegesen nagyobb modell, amely a dokumentumot (visszakeresett formában) egyben kapta meg, szintén a dokumentum egyetlen részére koncentrált – a lefedettség tehát nem a modell méretével, hanem a feladat megfogalmazásával kontrollálható. *Harmadszor*, a kérdésenkénti forráshivatkozás lehetővé tette, hogy egy olvasó – a kiértékelésben a bíró, a gyakorlatban az oktató – minden kulcsot a saját forrásával szemben ellenőrizzen; éppen ez tárta fel a naiv alapvonal hibás kulcsait és az ellenőrző karok szivárgó kérdéseit.

A pilot a módszer határait is megmutatta. Az ellenőrzés a hívásszámot 60–100-szorosára növelte, és ha az ellenőrző ugyanaz a 7B modell, mint a generátor, ez a többletköltség nem jár többletinformációval. A fő vizsgálatban ezért nem a tervezés, hanem az ellenőrzés az a komponens, amelynek bizonyítania kell a helyét. Ha az E2x kar (eltérő modelleszámba tartozó ellenőrző) sem mutat pozitív egyezést a bíróval, akkor a helyi telepítés helyes konfigurációja a tervezett, ellenőrzés nélküli csővezeték (E1), a forráshivatkozás megjelenítésével és az oktatói ellenőrzéssel mint egyetlen ellenőrzési réteggel.

8.2. A mérés mint kutatási eredmény

A pilot talán legfontosabb eredménye módszertani: a mérés három különböző ponton torzított volna, ha nem validáljuk. A tartalékvizsga-hiba a generálási sikert, a csonkolt kódoló a darabolás minőségét, a szivárgás pedig a tartalmi mutatókat és a bíró ítéletét torzította – mindhárom esetben *felfelé*, vagyis jobb eredményt mutatva a valósnál. A három hiba közös vonása, hogy a mérőrendszer „működött”: nem adott hibaiüzenetet, és hihető számokat állított elő. Ebből az a módszertani tanulság következik, hogy a nyelvi modellekre épülő rendszerek kiértékelésében a nyers kimenetek kézi átolvasása és a mutatók szélsőértékeinek tudatos vizsgálata nem helyettesíthető automatizmussal; a mi esetünkben a legjobb helyi alátámasztottsági értéket (E3, 87 %) éppen egy hibamód okozta.

A futásról futásra jelentkező zaj ($\approx 0,07$) becslése hasonló tanulság: a kevés dokumentumon, kevés ismétléssel mért különbségek jelentős része zaj lehet.

8.3. Az érvényességet veszélyeztető tényezők

A tapasztalati szoftvertechnológiai kutatás szokásos felosztását [72] követve négy érvényességi dimenziót vizsgálunk.

Belső érvényesség. Az egytényezős kísérleti terv a karok közötti különbséget egyetlen tényezőhöz köti, ám a tényezők nem mindig függetlenek: az ellenőrzés például a kérdések hosszát és formáját is megváltoztatja, ami közvetve hat a bíró érthetőségi ítéletére. A GPU-következtetés nemdeterminisztikussága miatt azonos konfiguráció mellett is eltérő kimenetek születnek; ezt a dokumentumszintű aggregálás és a dokumentumok számának növelése kezeli. A pilot és a fő vizsgálat között a csővezeték változott (szivárgási szabályok, javító hívás, $r = 1$), így a két vizsgálat eredményei nem közvetlenül összevethetők; a pilot eredményei az akkori implementációra vonatkoznak.

Konstrukciós érvényesség. A minőségi index négy egyenlő súlyú komponense közül a formai megfelelés vizsgaszintű bináris változó, a másik három kérdésszintű ráta; ezért egyetlen formai hiba aránytalanul nagy hatású. Ezt tudatosan vállaljuk (egy hibás formájú kérdés a gyakorlatban használhatatlanná teszi a vizsgát), de minden komponenst külön is közlünk. A lefedettség az embedding-térbeli legközelebbi darabra épül, ami csak közelítése annak, hogy a kérdés „miről szól”. Az alátámasztottság és a vak megoldhatóság a bíró ítéletén alapul; a bíró a kérdés kontextusát látja, így a dokumentum más részeiben lévő ellentmondást nem veheti észre, és – amint a szivárgás megmutatta – a formailag hibás, de „igaz” kérdéseket nem bünteti. A bíró validálása az oktatói értékeléssel ezért a fő vizsgálat elengedhetetlen része.

Külső érvényesség. A pilot egyetlen rövid, könnyű magyar szövegen futott, amelyen a naiv alapvonal a négy darabból hármát lát – ez a tervezés számára a legkedvezőtlenebb helyzet, hosszabb dokumentumokon nagyobb különbség várható. A fő adatkészlet magyar és angol részhalmaza eltérő forrásból származik (Wikipédia és tankönyv), így a nyelv hatása a forrás hatásával keveredik. Csak feleletválasztós kérdéseket értékelünk; az igaz–hamis és a kifejtős kérdések minősége külön vizsgálatot igényel. Az eredmények a vizsgált modellekre (Qwen2.5-7B, Llama-3.1-8B, a szervermodell) vonatkoznak; egy másik modelleslád eltérően viselkedhet.

Következtetési érvényesség. A dokumentumszintű nemparaméteres tesztek konzervatívak; a 39 dokumentum közepes hatásméretű kimutatására elegendő, kis hatások kimutatására nem. A több összehasonlítást Holm-korrekcióval kezeljük. A bootstrap-intervallumok kis mintán alulfedhetnek, ezért a pontbecslések mellett mindig a dokumentumszintű eloszlást is közöljük.

8.4. Etikai és jogi megfontolások

A rendszer hallgatói adatot nem kezel, a feltöltött dokumentumok a feladat végén törölődnek (4.3. szakasz), minden generált kérdés oktatói jóváhagyást igényel, az export pedig feltünteteti

az MI-közreműködést és a modellt [58]. A kiértékelő dokumentumok saját vagy nyílt licenccű (CC BY 4.0, CC BY-SA 4.0) művek; a szerverkarok és a bíró az egyetem saját GenAI-infrastruktúráján futnak. A rendszer célja a vázlatírás gyorsítása és ellenőrizhetővé tétele, nem az oktatói döntés kiváltása.

9. Összefoglalás és további munka

9.1. Összefoglalás

Dolgozatunkban a vizsgakérdés-generálás négy gyakorlati problémájából indultunk ki: a kulcsok helyessége nincs ellenőrizve (P1), a vizsga szerkezete és lefedettsége nincs kontrollálva (P2), a tananyag egy harmadik félhez kerül (P3), és a módszer nagy, távoli modellekre támaszkodik (P4). Ezekre a MIMIR rendszerrel válaszoltunk, amely az egyetlen promptot explicit, ellenőrizhető módszerrel váltja fel, és egyetlen fogyasztói GPU-n, adatmegőrzés nélkül fut.

A munka eredményei a következőkben foglalhatók össze.

- (1) **Formális modell.** A „jó vizsga” fogalmát mérhető, forrásalapú predikátumokra bontottuk (formai helyesség, alátámasztottság, disztraktor-validitás, vak megoldhatóság, lefedettség, szivárgás), a generálást korlátos optimalizálási feladatként írtuk fel, és bizonyítottuk a vizsgaterv két szerkezeti tulajdonságát: a Bloom-kiosztás kvótatulajdonságát és a mohó slotkiosztás lefedettségi alsó korlátját.
- (2) **Generálási módszer.** Kidolgoztuk és implementáltuk a tervezésvezérelt, forráshivatkozással csővezetékkel: ablakozott szemantikus darabolás, lefedettségvezérelt vizsgaterv, slotonkénti sűrű vagy hibrid visszakeresés, egyenkénti forráshoz kötött generálás, legolcsóbbtól a legdrágábbig rendezett ellenőrző lánc visszacsatolással, tartalékfogalmakkal és javító hívással, valamint munkamenet-hatókörű fogalomgráf.
- (3) **Adatvédő architektúra.** A mikroszolgáltatás-architektúra zéró adatmegőrzéssel, újlenyomatokra épülő naplózással és kapcsolható, kizárólag helyi üzemmóddal működik; a GPU-memóriaigény analitikus modelljét mérésekkel validáltuk: a 7B generátor és futtató környezete 5,4–5,9 GiB-ot foglal, így 8 GB-os kártyán is fut.
- (4) **Kiértékelési módszertan.** Tizenöt egytényezős kísérleti kart, 39 dokumentumos kétnyelvű adatkészletet 326 referenciakérdéssel, model családon kívüli bírót, közös bírót–oktató rubrikát és dokumentumszintű statisztikát tartalmazó, nyílt és reprodukálható keretrendszert hoztunk létre.
- (5) **Pilot eredmények.** A tervezés a helyi modell alátámasztottságát 67-ről 73 %-ra, vak megoldhatóságát 85-ről 93 %-ra, disztraktor-validitását 80-ről 87 %-ra és lefedettségét 50-ről 58 %-ra javította; a nagy szervermodell a legmagasabb minőséget (0,99), de a legalacsonyabb lefedettséget (25 %) érte el; az azonos modellel végzett önellenőrzés a bíróval csak véletlenszerű egyezést mutatott; a visszacsatolási hurok a kulcs kérdésbe szivárgását idézte elő, amely az automatikus mutatókat és a bírót egyaránt félrevezette; a futásról futásra jelentkező zaj ($\approx 0,07$) pedig megmutatta, hogy egyetlen dokumentumon a legtöbb különbség nem értelmezhető.

A kutatási kérdésekre a pilot alapján a következő előzetes válaszokat adhatjuk. **RQ1:** a

tervezés érvényesebb és jobban lefedő vizsgát eredményezett; a hatás iránya a várt, nagysága azonban egyetlen dokumentumon a zajsávon belül van. **RQ2:** a tervezés 23 hívásért mérhető javulást hozott; az önellenőrzés 60–100 hívásért nem hozott, a fogalomgráf pedig csak a szivárgás torzításával együtt javított; a rögzített darabolás a duplikátumokat megháromszorozta. **RQ3:** a helyi 7B modell a servermodell minőségi indexét 0,09–0,11-re közelítette meg, 8 GB alatti memóriaigénnyel.

9.2. További munka

A közvetlen folytatás a fő vizsgát lefuttatása a 6. fejezetben rögzített protokoll szerint: mind a 15 kar a 39 dokumentumon, a bíró validálása három oktató vak értékelésével, és a 7.10. táblázat kitöltése. Ezen túl a következő irányokat tartjuk ígéretesnek.

- **Független ellenőrző.** Ha az eltérő modelleszaládba tartozó ellenőrző (E2x) sem független a generátortól, természetes nyelvi következtetésre (NLI) tanított kisebb osztályozó vagy a FActScore-hoz hasonló atomi tényellenőrzés [35] lehet a következő lépés.
- **A megvalósult Bloom-szint ellenőrzése.** A pilot szerint a kis modell a kért kognitív szintet nem valósítja meg megbízhatóan; egy szintosztályozó ellenőrzés az ellenőrző láncba illeszthető.
- **Adaptív ellenőrzés.** Az ellenőrzés költségét csökkentené, ha csak a bizonytalan kérdésekre futna (pl. a kulcs lexikális alátámasztottsága vagy a generátor bizonyossága alapján).
- **Nyelvi összevetés.** Az angol dokumentumok egy részének magyar fordítása lehetővé tenné a nyelv hatásának a forrástól független mérését.
- **További kérdéstípusok és tanórai kipróbálás.** Az igaz–hamis és a kifejtős kérdések kiértékelése, valamint a generált vizsgák pszichometriai elemzése (nehézségi és diszkriminációs index, nem funkcionáló disztraktorok) valós hallgatói válaszokon.

A MIMIR nyílt forráskódú: a kód, a konfigurációk, az adatkészlet-manifesztek és a pilot nyers kimenetei a reprodukáláshoz rendelkezésre állnak.

Irodalomjegyzék

- [1] T. M. Haladyna, S. M. Downing és M. C. Rodriguez. „A Review of Multiple-Choice Item-Writing Guidelines for Classroom Assessment”. *Applied Measurement in Education* 15.3 (2002), 309–333. old.
- [2] M. J. Gierl, O. Bulut, Q. Guo és X. Zhang. „Developing, Analyzing, and Using Distractors for Multiple-Choice Tests in Education: A Comprehensive Review”. *Review of Educational Research* 87.6 (2017), 1082–1116. old.
- [3] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal és tsai. „Language Models are Few-Shot Learners”. *Advances in Neural Information Processing Systems*. 33. köt. 2020, 1877–1901. old.
- [4] E. Kasneci, K. Sessler, S. Küchemann, M. Bannert, D. Dementieva, F. Fischer és tsai. „ChatGPT for Good? On Opportunities and Challenges of Large Language Models for Education”. *Learning and Individual Differences* 103 (2023), 102274. old.
- [5] G. Kurdi, J. Leo, B. Parsia, U. Sattler és S. Al-Emari. „A Systematic Review of Automatic Question Generation for Educational Purposes”. *International Journal of Artificial Intelligence in Education* 30.1 (2020), 121–204. old. DOI: 10.1007/s40593-019-00186-y.
- [6] E. Alhazmi, Q. Z. Sheng, W. E. Zhang, M. Zaib és A. Alhazmi. „Distractor Generation in Multiple-Choice Tasks: A Survey of Methods, Datasets, and Evaluation”. *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 2024, 14437–14458. old.
- [7] Z. Ji, N. Lee, R. Frieske és tsai. „Survey of Hallucination in Natural Language Generation”. *ACM Computing Surveys* 55.12 (2023), 1–38. old.
- [8] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe és tsai. „Self-Refine: Iterative Refinement with Self-Feedback”. *Advances in Neural Information Processing Systems*. 36. köt. 2023.
- [9] S. Dhuliawala, M. Komeili, J. Xu és tsai. „Chain-of-Verification Reduces Hallucination in Large Language Models”. *Findings of the Association for Computational Linguistics: ACL 2024*. 2024.
- [10] J. Huang, X. Chen, S. Mishra és tsai. „Large Language Models Cannot Self-Correct Reasoning Yet”. *International Conference on Learning Representations (ICLR)*. 2024.
- [11] N. F. Liu, K. Lin, J. Hewitt és tsai. „Lost in the Middle: How Language Models Use Long Contexts”. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. old.
- [12] C.-P. Hsieh, S. Sun, S. Krizan és tsai. „RULER: What’s the Real Context Size of Your Long-Context Language Models?”. *Conference on Language Modeling (COLM)*. 2024.
- [13] Európai Parlament és a Tanács. *Az Európai Parlament és a Tanács (EU) 2016/679 rendelete (általános adatvédelmi rendelet)*. Az Európai Unió Hivatalos Lapja, L 119. 2016.
- [14] R. Mitkov és L. A. Ha. „Computer-Aided Generation of Multiple-Choice Tests”. *Proceedings of the HLT-NAACL 03 Workshop on Building Educational Applications Using Natural Language Processing*. 2003, 17–22. old.

- [15] M. Heilman és N. A. Smith. „Good Question! Statistical Ranking for Question Generation”. *Human Language Technologies: The 2010 Annual Conference of the North American Chapter of the Association for Computational Linguistics*. 2010, 609–617. old.
- [16] X. Du, J. Shao és C. Cardie. „Learning to Ask: Neural Question Generation for Reading Comprehension”. *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*. 2017, 1342–1352. old.
- [17] A. Vaswani, N. Shazeer, N. Parmar és tsai. „Attention Is All You Need”. *Advances in Neural Information Processing Systems*. 30. köt. 2017.
- [18] J. Devlin, M.-W. Chang, K. Lee és K. Toutanova. „BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*. 2019, 4171–4186. old.
- [19] N. Mulla és P. Gharpure. „Automatic Question Generation: A Review of Methodologies, Datasets, Evaluation Metrics, and Applications”. *Progress in Artificial Intelligence* 12.1 (2023), 1–32. old.
- [20] K. Papineni, S. Roukos, T. Ward és W.-J. Zhu. „BLEU: A Method for Automatic Evaluation of Machine Translation”. *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. 2002, 311–318. old.
- [21] C.-Y. Lin. „ROUGE: A Package for Automatic Evaluation of Summaries”. *Text Summarization Branches Out*. 2004, 74–81. old.
- [22] B. S. Bloom, szerk. *Taxonomy of Educational Objectives: The Classification of Educational Goals. Handbook I: Cognitive Domain*. New York: David McKay, 1956.
- [23] L. W. Anderson és D. R. Krathwohl, szerk. *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom’s Taxonomy of Educational Objectives*. New York: Longman, 2001.
- [24] P. Lewis, E. Perez, A. Piktus és tsai. „Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. *Advances in Neural Information Processing Systems*. 33. köt. 2020, 9459–9474. old.
- [25] V. Karpukhin, B. Oğuz, S. Min és tsai. „Dense Passage Retrieval for Open-Domain Question Answering”. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2020, 6769–6781. old.
- [26] Y. Gao, Y. Xiong, X. Gao és tsai. *Retrieval-Augmented Generation for Large Language Models: A Survey*. 2023. arXiv: 2312.10997.
- [27] N. Reimers és I. Gurevych. „Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*. 2019, 3982–3992. old.
- [28] L. Wang, N. Yang, X. Huang és tsai. *Text Embeddings by Weakly-Supervised Contrastive Pre-training*. 2022. arXiv: 2212.03533.
- [29] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder és F. Wei. *Multilingual E5 Text Embeddings: A Technical Report*. 2024. arXiv: 2402.05672.
- [30] S. Robertson és H. Zaragoza. „The Probabilistic Relevance Framework: BM25 and Beyond”. *Foundations and Trends in Information Retrieval* 3.4 (2009), 333–389. old.

- [31] G. V. Cormack, C. L. A. Clarke és S. Buettcher. „Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods”. *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2009, 758–759. old.
- [32] R. Qu, R. Tu és F. Bao. *Is Semantic Chunking Worth the Computational Cost?* 2024. arXiv: 2410.13070.
- [33] D. Edge, H. Trinh, N. Cheng és tsai. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. 2024. arXiv: 2404.16130.
- [34] P. Manakul, A. Liusie és M. J. F. Gales. „SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models”. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2023, 9004–9017. old.
- [35] S. Min, K. Krishna, X. Lyu és tsai. „FACTScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation”. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2023, 12076–12100. old.
- [36] C. Snell, J. Lee, K. Xu és A. Kumar. „Scaling LLM Test-Time Compute Optimally Can be More Effective than Scaling Parameters for Reasoning”. *International Conference on Learning Representations (ICLR)*. 2025.
- [37] T. Dettmers, M. Lewis, Y. Belkada és L. Zettlemoyer. „LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale”. *Advances in Neural Information Processing Systems*. 35. köt. 2022.
- [38] E. Frantar, S. Ashkboos, T. Hoefler és D. Alistarh. „GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers”. *International Conference on Learning Representations (ICLR)*. 2023.
- [39] G. Gerganov és tsai. *llama.cpp: LLM inference in C/C++*. <https://github.com/ggml-org/llama.cpp>. Szoftver, GGUF formátum és K-kvantálás. 2023.
- [40] Ollama. *Ollama: Get up and running with large language models locally*. <https://github.com/ollama/ollama>. Szoftver. 2024.
- [41] S. Li, X. Ning, L. Wang és tsai. „Evaluating Quantized Large Language Models”. *Proceedings of the 41st International Conference on Machine Learning (ICML)*. 235. köt. PMLR. 2024, 28480–28524. old.
- [42] E. Kurtic, A. N. Marques, S. Pandit, M. Kurtz és D. Alistarh. „“Give Me BF16 or Give Me Death”? Accuracy-Performance Trade-Offs in LLM Quantization”. *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2025.
- [43] U. Kurt. *Which Quantization Should I Use? A Unified Evaluation of llama.cpp Quantization on Llama-3.1-8B-Instruct*. 2026. arXiv: 2601.14277.
- [44] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón és S. Sanghai. „GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints”. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2023, 4895–4901. old.
- [45] W. Kwon, Z. Li, S. Zhuang és tsai. „Efficient Memory Management for Large Language Model Serving with PagedAttention”. *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*. 2023, 611–626. old.

- [46] P. M. Zähl és A. Hennig. *Energy Efficiency of Locally Deployed LLMs: A Preliminary Quantitative GPU Power Benchmark on Consumer Hardware*. 2026. arXiv: 2608.00008.
- [47] F. Fawzi, S. Amini és S. Bulathwela. „Small Generative Language Models for Educational Question Generation”. *NeurIPS 2023 Workshop on Generative AI for Education (GAIED)*. 2023.
- [48] F. Fawzi, S. Balan, M. Cukurova, E. Yilmaz és S. Bulathwela. „Towards Human-Like Educational Question Generation with Small Language Models”. *Artificial Intelligence in Education. Posters and Late Breaking Results (AIED 2024)*. 2150. köt. Communications in Computer and Information Science. Springer, 2024.
- [49] C. D. Jaldi, A. Saini, S. Zhang, N. Schroeder, C. Shimizu és E. Ilkou. *Small, Private Language Models as Teammates for Educational Assessment Design*. 2026. arXiv: 2605.15015.
- [50] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang és tsai. „Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena”. *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*. 36. köt. 2023.
- [51] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu és C. Zhu. „G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment”. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2023, 2511–2522. old.
- [52] P. Wang, L. Li, L. Chen és tsai. „Large Language Models are not Fair Evaluators”. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. 2024, 9440–9450. old.
- [53] A. Panickssery, S. R. Bowman és S. Feng. „LLM Evaluators Recognize and Favor Their Own Generations”. *Advances in Neural Information Processing Systems*. 37. köt. 2024.
- [54] OpenAI. *gpt-oss-120b & gpt-oss-20b Model Card*. 2025. arXiv: 2508.10925.
- [55] S. Es, J. James, L. Espinosa-Anke és S. Schockaert. „RAGAs: Automated Evaluation of Retrieval Augmented Generation”. *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*. 2024, 150–158. old.
- [56] N. Ligeti-Nagy, G. Ferenczi, E. Héja és tsai. „HuLU: Hungarian Language Understanding Benchmark Kit”. *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*. 2024, 8360–8371. old.
- [57] A. Novák, B. Novák, D. Zombori, G. Szabó, Z. Szántó és R. Farkas. „A Question Answering Benchmark Database for Hungarian”. *Proceedings of the 17th Linguistic Annotation Workshop (LAW-XVII)*. 2023, 188–198. old.
- [58] Európai Parlament és a Tanács. *Az Európai Parlament és a Tanács (EU) 2024/1689 rendelete a mesterséges intelligenciára vonatkozó harmonizált szabályok megállapításáról (mesterséges intelligenciáról szóló rendelet)*. Az Európai Unió Hivatalos Lapja, L sorozat. 2024.
- [59] M. L. Balinski és H. P. Young. *Fair Representation: Meeting the Ideal of One Man, One Vote*. 2. kiad. Washington, D.C.: Brookings Institution Press, 2001.

- [60] G. L. Nemhauser, L. A. Wolsey és M. L. Fisher. „An Analysis of Approximations for Maximizing Submodular Set Functions – I”. *Mathematical Programming* 14.1 (1978), 265–294. old.
- [61] The PostgreSQL Global Development Group. *PostgreSQL 16 Documentation: SELECT – The Locking Clause (FOR UPDATE SKIP LOCKED)*. <https://www.postgresql.org/docs/16/sql-select.html>. 2024.
- [62] Qwen Team. *Qwen2.5 Technical Report*. 2024. arXiv: 2412.15115.
- [63] A. Grattafiori és tsai. *The Llama 3 Herd of Models*. 2024. arXiv: 2407.21783.
- [64] Qdrant. *Qdrant: Vector Similarity Search Engine*. <https://github.com/qdrant/qdrant>. Szoftver. 2024.
- [65] A. Hadifar, S. K. Bitew, J. Deleu, C. Develder és T. Demeester. „EduQG: A Multi-Format Multiple-Choice Dataset for the Educational Domain”. *IEEE Access* 11 (2023), 20885–20896. old.
- [66] B. Efron és R. J. Tibshirani. *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993.
- [67] F. Wilcoxon. „Individual Comparisons by Ranking Methods”. *Biometrics Bulletin* 1.6 (1945), 80–83. old.
- [68] D. S. Kerby. „The Simple Difference Formula: An Approach to Teaching Nonparametric Correlation”. *Comprehensive Psychology* 3 (2014).
- [69] S. Holm. „A Simple Sequentially Rejective Multiple Test Procedure”. *Scandinavian Journal of Statistics* 6.2 (1979), 65–70. old.
- [70] J. Cohen. „A Coefficient of Agreement for Nominal Scales”. *Educational and Psychological Measurement* 20.1 (1960), 37–46. old.
- [71] K. Krippendorff. *Content Analysis: An Introduction to Its Methodology*. 2. kiad. Thousand Oaks, CA: Sage, 2004.
- [72] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell és A. Wesslén. *Experimentation in Software Engineering*. Berlin, Heidelberg: Springer, 2012.

Ábrák jegyzéke

4.1.	A MIMIR szolgáltatásai és adatfolyama. A szaggatott keretben lévő komponensek a feltöltött tartalmat kizárólag a feladat élettartama alatt, memóriában tartják; a nyelvi modell az egyetlen GPU-t használó összetevő.	15
5.1.	A naiv RAG és a MIMIR tervezett csővezetéke. Kék: a kísérleti karokban ki- és bekapcsolt komponensek. A terv a vizsga szerkezetét még az első kérdés megírása előtt rögzíti, és minden kérdést a saját, hivatkozott kontextusával szemben ellenőriz.	19
6.1.	A kiértékelő keretrendszer lépései és kimenetei.	29
7.1.	Az egyes minőségi mutatók karonként (pilot). A „szöveg szerint” oszlop a meta-hivatkozások aránya, a keresési találat@3 a referenciakérdések bizonyító mondatának visszakeresési aránya.	38
7.2.	Minőség és futási idő a pilotban. A tervezett karok egy nagyságrenddel lassabbak; a minőségnövekedést csak a tervezés (E1) hozta meg ellenőrzés nélkül. . .	43

Táblázatok jegyzéke

2.1.	A kérdésgenerálási megközelítések összevetése a P1–P4 problémák szempontjából (✓ : megoldott, ○ : részben, – : nem kezelt).	8
4.1.	Tervezési célok: a dokumentum csevegő LLM-nek való átadása és a MIMIR összevetése.	14
4.2.	A MIMIR szolgáltatásai és kutatási szempontból releváns felelősségeik.	15
4.3.	Adatfajták életrajza a MIMIR rendszerben.	16
4.4.	Becsült GPU-memóriaigény 8 192 tokenes kontextusnál, fp16 KV-gyorsítótárral és $M_0 \approx 0,6$ GB futtatókörnyezeti többlettel. A súlyok a GGUF-fájlok mérete, a gyorsítótár a publikált architektúrákból számolt érték.	17
4.5.	Mért GPU-memória a pilot vizsgálatban (MiB), karonként az első, hidegindítású vizsga alapján. A különbség a modell és a futtatókörnyezet foglalása.	18
5.1.	A tervezett csővezeték paraméterei és alapértékei.	20
5.2.	A nehézségi szintekhez rendelt Bloom-eloszlások és a $n = 10$ kérdéses vizsga kiosztása.	23
5.3.	Az ellenőrző lánc lépései végrehajtási sorrendben, a hiba esetén adott visszajelzéssel.	26
6.1.	Az eval-v1 kiértékelő adatkészlet. R/K/H: rövid ($\leq 6\,000$), közepes ($6\,000$ – $25\,000$) és hosszú ($25\,000$ – $90\,000$ karakter) dokumentumok száma.	30
6.2.	Kísérleti karok. Minden kar a zárójelben megadott referenciakartól egy tényezőben tér el.	31
6.3.	Tervezett páros összehasonlítások és a hozzájuk tartozó kutatási kérdések, hipotézisek.	32
6.4.	A bírósági és az oktatók közös értékelési rubrikája (rövidítve; a teljes szöveg a B. függelékben).	33
7.1.	A pilot fő eredményei: egy magyar dokumentum, 3 mag, karonként 30 kérdés. Ráták %-ban. Q : minőségi index; Form.: formai megfelelés; Alát.: alátámasztottság; Vak: vak megoldhatóság; Diszt.: disztraktor-validitás; Lef.: lefedettség; Sziv.: kulcs a törzsben; Nell.: nem ellenőrzött kérdések száma vizsgánként; Hívás: átlagos modellhívás vizsgánként; Perc: medián futási idő vizsgánként.	38
7.2.	A kérdések hossza, változatossága és tematikus hasonlósága karonként (pilot, átlagok 30 kérdésen). Törzs: a törzs karakterszáma; Opciók: a négy opció együttes karakterszáma; Dupl.: duplikátumráta; RefSim: átlagos maximális hasonlóság a referenciakérdésekhez.	39
7.3.	Egyezik-e a 7B ellenőrző ítélete a bírósággal? Az E2, E2f, E2h és E3 karok pilot kérdései az ellenőrző ítélete szerint; a bírósági kimenetek %-ban.	40
7.4.	Az ellenőrző pontossága, felidézése és Cohen- κ együtthatója a bírósági alátámasztottsági ítéletéhez képest, karonként. Átengedett: az átengedett kérdések aránya (%).	40

7.5.	Válaszszivárgás karonként (pilot, 30 kérdés karonként, %). Kulcs a törzsben: legalább háromszavas kulcs, amelynek szótokenjei legalább 80 %-ban a törzsben is szerepelnek; nem kérdés: a törzs sem kérdőjelre, sem kettőspontra nem végződik; bírói helyesség: a szivárgó kérdések átlagos helyességi pontszáma (1–5).	41
7.6.	A bíró rubrikapontszámai (1–5, átlagok 30 kérdésen) és a „felhasználnám” arány karonként (pilot).	42
7.7.	Modellhívások szakaszonként, tokenfelhasználás és futási idő vizsgámként (pilot, átlagok 3 vizsgán; az idő medián). Terv.: tervező vagy gráfépítés; Gen.: generálás; Alát.: alátámasztottsági ellenőrzés; Vak: vak megoldási ellenőrzés. .	43
7.8.	Pilotkérdések eredeti nyelven (* = megjelölt kulcs), a bíró ítéletével.	44
7.9.	A hipotézisek és a pilot által szolgáltatott előzetes, leíró bizonyíték.	45
7.10.	A fő vizsgálat eredményei (39 dokumentum; átlag és 95 %-os bootstrap-CI; páros Wilcoxon-próba Holm-korrektcióval a referenciakarhoz képest). A fő futás után kitöltendő.	45
C.1.	A fő vizsgálat futtatási terve.	64
C.2.	A dolgozat táblázatainak és ábráinak adatforrásai a tárolóban.	64

A. Promptok

Ez a függelék a csővezetékek promptjait közli (rövidítve, a JSON-sémák elhagyásával ott, ahol a törzsszövegből egyértelműek). A belső lépések (tervező, gráf, ellenőrző) promptjai angol nyelvűek, mert a kis helyi modellek ezeket megbízhatóbban követik; a generátor promptja magyar, és megismétli az éles prompt szabályait, így az E0 és az E1+ karok szerkezetben, nem szabályokban különböznek.

A.1. A naív alapvonal (E0) és a teljes dokumentumos alapvonal promptja

```
Te egy kiemelkedő tudású oktatásmódszertani szakértő és professzionális vizsgakészítő vagy.

KÖTELEZŐ SZABÁLYOK, AMIKET SZIGORÚAN BE KELL TARTANOD:
1. ZÉRÓ HALLUCINÁCIÓ: KIZÁRÓLAG a megadott KONTEXTUS alapján dolgozz! Ha a kontextus nem tartalmazza a választ, ne találj ki semmit!
2. FELHASZNÁLÓI UTASÍTÁS KÖVETÉSE: ... Pontosán a kért mennyiséget és típust generáld le!
3. DISZTRAKTOROK: Feleletválasztós (mcq) kérdés esetén 1 helyes és 3 hihető, de helytelen válasz.
4. NO LATEX: Szigorúan TILOS LaTeX formázást vagy dollárjeleket ($) használni!
5. META-REFERENCIA TILALOM: Szigorúan TILOS a dokumentum szerkezetére kérdezni.

KONTEXTUS:
{context_text}

FELADAT (A felhasználó pontos kérése):
"{query}"

KIMENETI FORMÁTUM: kizárólag egy érvényes JSON objektum:
{"title": "Mimir AI Vizsga", "format": "pdf", "questions": [
  {"type": "mcq", "text": "...?", "answers": [{"text": "...", "is_correct": true}, ...]}]}
```

A.2. A tervező promptja

```
[planner] You design exam blueprints. Reply with one valid JSON object only.

Below is a document split into numbered chunks (C0, C1, ...). Some chunks are shortened.
{overview}
List the {n_concepts} most important, clearly distinct concepts, facts or ideas a student should
be tested on. Cover the whole document: every major part should have at least one concept.
Prefer
central ideas over trivia. Write concept names and descriptions in {language}.
JSON shape:
{"concepts": [{"concept": "short name", "description": "one sentence: what exactly should be
  tested", "chunk_ids": ["C0"], "importance": 3}]}
importance: 3 = central, 2 = important, 1 = detail.
```

A.3. A generátor promptja

```
[generator] Te egy kiemelkedő tudású oktatásmódszertani szakértő és professzionális vizsgakészítő vagy. Kizárólag érvényes JSON formátumban válaszolj, markdown formázás nélkül!

Készíts PONTOSAN EGY vizsgakérdést az alábbi terv szerint.
TERV:
- Fogalom: {concept}
- Mit kérdezz: {description}
- Kérdéstípus: feleletválasztós ("mcq"): pontosan 4 válasz, ebből pontosan 1 helyes és 3 hihető, de a kontextus szerint egyértelműen helytelen disztraktor; a válaszok hossza és formája legyen hasonló.
- Bloom-szint: {pl. Megértés (magyarázat, összehasonlítás, saját szavakkal)}
- Vizsga nehézsége: {difficulty}
- Nyelv: a kérdést és a válaszokat {magyar} nyelven írd.
KÖTELEZŐ SZABÁLYOK:
1. ZÉRÓ HALLUCINÁCIÓ: KIZÁRÓLAG a KONTEXTUS alapján dolgozz. A helyes válasznak a kontextusból igazolhatónak kell lennie.
2. HIVATKOZÁS: a "citations" mezőbe írd azoknak a részleteknek az azonosítóját (pl. "C3"), amelyek igazolják a helyes választ.
3. NO LATEX. 4. META-REFERENCIA TILALOM.
5. NE ISMÉTELD a már elkészült kérdéseket: {existing}
[DISZTRAKTOR-ÖTLETEK (rokon fogalmak a dokumentumból) - csak E3]
[AZ ELŐZŐ PRÓBÁLKOZÁS HIBÁS VOLT, JAVÍTSD: {feedback}]
KONTEXTUS:
[C3] ...
[C1] ...
KIMENETI FORMÁTUM (csak ez a JSON objektum):
{"type": "mcq", "text": "A kérdés szövege?", "answers": [...], "citations": ["C0"], "bloom": "understand", "explanation": "egy mondat: miért ez a helyes válasz"}
```

A.4. Az ellenőrző promptjai

```
[verifier] You check exam questions against source text. Reply with one valid JSON object only.

SOURCE: "{source}"
QUESTION (mcq): {text}
A. ... <-- marked correct
B. ...
Check strictly against the SOURCE only:
1. key_supported: is the marked-correct answer clearly supported by the SOURCE?
2. For every option NOT marked correct: is it clearly wrong according to the SOURCE? (options_wrong, in order)
3. ambiguous: could a careful reader argue that more than one option is correct?
4. problems: one short sentence describing any problem (" " if none).
JSON shape: {"key_supported": true, "options_wrong": [true, true, true], "ambiguous": false, "problems": ""}

--- vak megoldás ---
SOURCE: "{source}"
Answer the question using only the SOURCE. Exactly one option is correct.
{text}
```

A. ... B. ... C. ... D. ... (véletlen permutáció)
JSON shape: {"answer": "A"}

A.5. A fogalomgráf promptja (E3)

[graph] You extract knowledge graphs from teaching material. Reply with one valid JSON object only.

Extract the knowledge in these chunks as a small concept graph. Chunk ids are in brackets.
{chunks_block}

Rules:

- concepts: the key terms, entities, processes or ideas (max 8 per chunk). "parent" is the broader category the concept belongs to (e.g. "T-sejt" -> "limfocita"), or "" if none.
- facts: short atomic statements from the text (max 6 per chunk), with the concepts they mention.
- relations: typed links between concepts: is_a, part_of, causes, contrasts_with, used_for, related_to.

JSON shape: {"concepts": [...], "facts": [...], "relations": [...]}

B. A bíró és az oktatók rubrikája

A rubrika szövege a bíró és az oktatói értékelőlap számára azonos (a bíró az angol, az oktatók a magyar változatot kapják).

Helyesség

A megjelölt válasz a forrás szerint helyes, és más opció nem helyes. 1 = hibás kulcs vagy két helyes opció; 3 = helyes, de pontatlan; 5 = teljesen helyes, egyértelmű.

Érthetőség

A kérdés önmagában érthető, nyelvtanilag helyes, egyértelmű, nem hivatkozik „a szövegre” vagy a dokumentum szerkezetére. 1 = zavaros; 3 = nehezen érthető; 5 = vizsgakész megfogalmazás.

Disztraktorok

A rossz válaszok a forrás szerint egyértelműen rosszak, de egy felkészületlen diák számára hihetők (azonos téma, hasonló hossz és forma). 1 = abszurd vagy szintén helyes; 3 = részben hihető; 5 = mind hihető és rossz. Kifejtős kérdésnél üresen marad.

Szintillesztés

A kognitív szint megfelel a kért nehézségnek (könnyű: felidézés/megértés; közepes: alkalmazás/megértés; nehéz: elemzés/értékelés). 1 = egyértelműen rossz szint; 3 = határeset; 5 = illik.

Felhasználnám

Beraknád ezt a kérdést egy valódi dolgozatba legfeljebb apró javítással? (igen/nem)

A bíró áttekintő hívása ezen felül kéri a kulcsot igazoló mondat szó szerinti idézését (key_evidence mező), disztraktoronként a is_wrong és on_topic ítéletet, a kérdés Bloom-szintjét és egy-egy mondatos indoklást kér.

C. Konfigurációk és reprodukálás

C.1. Példakonfigurációk

Minden kísérleti kar egy YAML-fájl a `tests/eval/configs` könyvtárban. Az alábbi két konfiguráció az E2 és az E3 kart írja le; a két fájl egyetlen kulcsban (`graph`) különbözik, ami az egytényezős tervet a konfiguráció szintjén is láthatóvá teszi.

```
# E2 - E1 + verifier chain ("Thorough" mode): schema, meta-reference, citations, grounding +
# distractor check, blind answer test; max 1 retry per slot with feedback, then 1 replacement.
name: E2
description: "Blueprint + verifier (max 1 retry, blind answer test on)"
seeds: [1]
exam: {n_questions: 10, types: [mcq]}
pipeline: {kind: blueprint, retrieval: dense, retrieval_k: 3, verifier: true, blind_test: true,

            max_retries: 1, graph: false}
chunking: {mode: service, method: percentile, threshold: 85, target_size: 1000}
generator: {provider: ollama, model: "qwen2.5:7b", temperature: 0.2, num_ctx: 8192}

# E3 - E2 + concept graph
name: E3
pipeline: {kind: blueprint, retrieval: dense, retrieval_k: 3, verifier: true, blind_test: true,

            max_retries: 1, graph: true}
```

C.2. A reprodukálás lépései

```
# 1. Szolgáltatások és helyi modell
ollama pull qwen2.5:7b
docker-compose up -d --build

# 2. Kiértékelő környezet
cd tests/eval
pip install -r requirements.txt
python -m mimir_eval check configs/e0_naive_local.yaml # adatkészlet és szolgáltatások

# 3. Egy kar futtatása, pontozása és a jelentés
python -m mimir_eval run configs/e1_blueprint.yaml
python -m mimir_eval score results/E1_<időbélyeg>
python -m mimir_eval report --results results --name main

# 4. A teljes mátrix (Windows PowerShell)
./run_all.ps1
```

C.3. A fő vizsgálat futtatási terve

C.1. táblázat. A fő vizsgálat futtatási terve.

Lépés	Tartalom	Erőforrás	Becsült idő
R0	fűstteszt minden karon 2 dokumentumon (egy hosszú)	helyi GPU + szerver	2–3 óra
R1	helyi karok mind a 39 dokumentumon	helyi GPU	2–3 nap
R2	szerverkarok mind a 39 dokumentumon	egyetemi szerver	kb. 1 nap
R3	pontozás a bíróval	egyetemi szerver	kb. 1 nap
R4	oktatói értékelés: 3 oktató $\times$ (60 + 10) kérdés	oktatók	1–2 hét
R5	report: táblázatok, ábrák, tesztek	–	1–2 nap

C.4. A közölt számok forrásai

C.2. táblázat. A dolgozat táblázatainak és ábráinak adatforrásai a tárolóban.

Dolgozatbeli elem	Forrás
7.1. táblázat, 7.1. ábra	results_pilot/*/exam_metrics.csv, questions.jsonl; report_pilot/pilot
7.3., 7.4. táblázat	results_pilot/{E2,E2f,E2h,E3}*/exams.jsonl (verified) és questions.jsonl (grounded, blind_correct, judge_would_use)
7.5. táblázat	questions.jsonl; schema.key_in_stem, schema.stem_is_question
7.6. táblázat	questions.jsonl, judge_* mezők, 30 kérdés átlaga
7.2. táblázat	questions.jsonl (text, options), exam_metrics.csv (duplicate_rate, gold_max_sim_mean)
7.7. táblázat	exams.jsonl (llm.calls_by_stage, llm.tokens), exam_metrics.csv
4.5. táblázat	exams.jsonl (vram_baseline_mib, vram_peak_mib), karonként az első vizsga
4.4. táblázat	publikált modellarchitektúrák és GGUF-fájlméretek; M_0 becslés
6.1. táblázat	tests/eval/dataset/manifest*.yaml
7.8. táblázat	results_pilot/{E0,E1,E3,E4}*/questions.jsonl, l. mag
Futásról futásra zaj	results_pilot/{E0,E5a}*/exams.jsonl (retrieved)